

INDUSTRIAL PROJECT REPORT ON STOCKGRID: AI-POWERED INVENTORY MANAGEMENT SYSTEM

*Submitted in partial fulfillment of the requirements for the
degree of*

Master of Computer Applications (MCA)

in

DEPARTMENT OF COMPUTER APPLICATION

Engineering College Bikaner

Session-2024-2026

Submitted By:
Shubham Panwar

MCA Final Year
Semester: 4th

Engineering College Bikaner,
Pugal Road, RIICO Karni Industrial Area,
Bikaner, Rajasthan-33004
web: www.ecb.ac.in, Telephone: +91-
1512253404

Guided by:
Dr. Ankur Goswami
sir

MCA Department
Engineering College Bikaner
Bikaner Technical University

Submitted to:
Mr. Sanjay Tejaswi
sir

HOD Of MCA Department
Engineering College Bikaner
Bikaner Technical University

Bikaner Technical University,
UCET Campus
Pugal Road, RIICO Karni Industrial Area,
Bikaner, Rajasthan-33004
web: www.btu.ac.in

Certificate from Company

[This is a placeholder for the certificate provided by the organization where the industrial training/project was conducted.]



Certificate from Guide

[This is a placeholder for the certificate from your internal mentor/guide.]



Acknowledgement

The successful completion of this industrial project, "**StockGrid: An AI-Powered Inventory Management System**," is the result of the collective effort, guidance, and support of several individuals and the academic environment provided by the institute.

Gratitude to the Internal Mentor and Department

I wish to express my deepest sense of gratitude to my **Dr. Ankur Goswami Sir Internal**

Mentor/Guide, whose technical expertise and consistent encouragement were vital throughout the project lifecycle. Their insights into full-stack architecture, particularly regarding the integration of Node.js and PostgreSQL, helped bridge the gap between theoretical knowledge and practical application.

I am also sincerely grateful to the **Head of Department (HoD)** for granting the necessary

permissions and providing the administrative framework required to undertake this project. Their vision for academic excellence and insistence on "industrial significance" pushed me to develop a solution that addresses real-world inventory challenges.

Institutional and Industrial Support

I would like to acknowledge the **Educational Institute** for providing a conducive learning environment. A special thanks goes to the **Incubation Center**, where the development and laboratory work for this project were conducted. The facilities provided there were instrumental in testing the AI-driven demand prediction models and smart mapping features of StockGrid.

I also extend my thanks to our **industry collaborators**. Their feedback regarding existing manual system drawbacks—such as "dead stock" and SKU tracking errors—provided the foundational requirements for this software. Interacting with professionals in the field allowed me to understand the operational assignments

and functional areas essential for a modern MCA project.

Peers and Family

I am indebted to my **peers and classmates** for their constructive criticism and for providing a collaborative atmosphere for brainstorming complex coding logic. Our discussions often led to more efficient ways to handle JWT-based authentication and real-time stock calculations.

Finally, I would like to thank my **family** for their unwavering emotional support and patience during the long hours of development and documentation. Their belief in my capabilities was the primary motivation behind the successful submission of this project within the prescribed timeline

Index

1. CHAPTER 1: INTRODUCTION

- 1.1 Company/Educational Institute Profile
- 1.2 Existing System and Need for System
- 1.3 Scope of Work
- 1.4 Operating Environment – Hardware and Software

2. CHAPTER 2: PROPOSED SYSTEM

- 2.1 Proposed System
- 2.2 Objectives of System
- 2.3 User Requirements

3. CHAPTER 3: ANALYSIS & DESIGN

- 3.1 Entity Relationship Diagram (ERD)
- 3.2 System Architecture
- 3.3 Database Requirements & User Interfaces
- 3.4 Data Flow Diagram (DFD)
- 3.5 Data Dictionary
- 3.6 Table Design
- 3.7 Code Design
- 3.8 Menu Screens / Input Screens
- 3.9 Test Procedures and Implementation

4. CHAPTER 4: USER MANUAL

- 4.1 User Manual
- 4.2 Operations Manual / Menu Explanation
- 4.3 Forms and Report Specifications

5. Drawbacks and Limitations

- 6. Proposed Enhancements
- 7. Conclusions
- 8. Bibliography
- 9. Annexures



CHAPTER 1: INTRODUCTION

1.1 Company/Educational Institute Profile

StockGrid was developed as part of an industrial project within the college incubation center. The project follows the standards of the Master of Computer Applications (MCA) curriculum, focusing on full-stack application development and AI integration for business automation.

1.2 Existing System and Need for System

Currently, many small and medium-sized businesses rely on manual spreadsheets or outdated offline software to track their inventory. These 'Existing Systems' suffer from several drawbacks:

- - Lack of real-time stock updates across different roles (Admin/Staff).
- - High risk of manual entry errors leading to 'dead stock' or stockouts.
- - No automated alerting system for low stock levels.
- - Difficulty in analyzing historical data for future demand prediction.

StockGrid addresses these needs by providing a structured, scalable, and intelligent system that automates tracking and leverages AI for insights.

1.3 Scope of Work

The scope of this project encompasses the design and development of a full-stack SaaS application. The primary functional areas include:

- - Product CRUD and Category Management.
- - Real-time IN/OUT transaction tracking with an audit trail.
- - Smart low-stock notifications and AI-driven demand prediction.
- - Secure multi-role authentication (Admin, Manager, Staff).

1.4 Operating Environment – Hardware and Software

Software Requirements:

- - Frontend: React (Vite), Tailwind CSS
- - Backend: Node.js, Express.js
- - Database: PostgreSQL (via Supabase)
- - Authentication: JWT (JSON Web Tokens)
- - Email Service: Gmail SMTP

Hardware Requirements:

- - Processor: Intel i5 / AMD Ryzen 5 or higher
- - RAM: 8GB minimum
- - Storage: 256GB SSD

CHAPTER 2: PROPOSED SYSTEM

2.1 Proposed System

The proposed 'StockGrid' system is an AI-powered SaaS platform. Unlike traditional inventory tools, it uses a centralized PostgreSQL database for real-time synchronization and employs AI models to suggest reorder points and predict stock demand based on historical trends.

2.2 Objectives of System

- - Efficiency: Reduce manual labor in tracking stock movements.
- - Accuracy: Provide real-time inventory levels with automated audit logs.
- - Intelligence: Utilize AI for demand forecasting and reorder suggestions.
- - Security: Implement Role-Based Access Control (RBAC) for data integrity.

2.3 User Requirements

The system identifies three primary users with specific requirements:

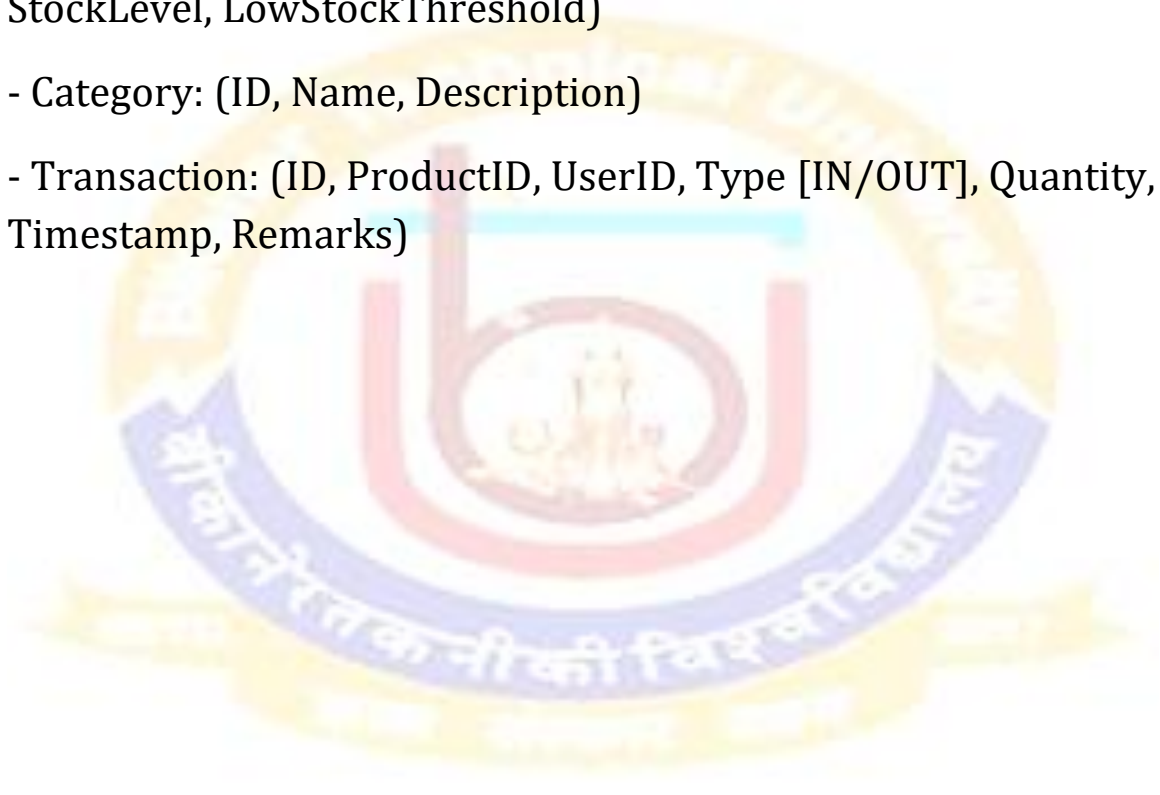
- 1. Admin: Full control over system configuration, user management, and advanced analytics.
- 2. Manager: Authorized to handle SKU-based product tracking, category management, and data imports/exports.
- 3. Staff: Responsible for logging IN/OUT transactions and monitoring daily stock alerts.

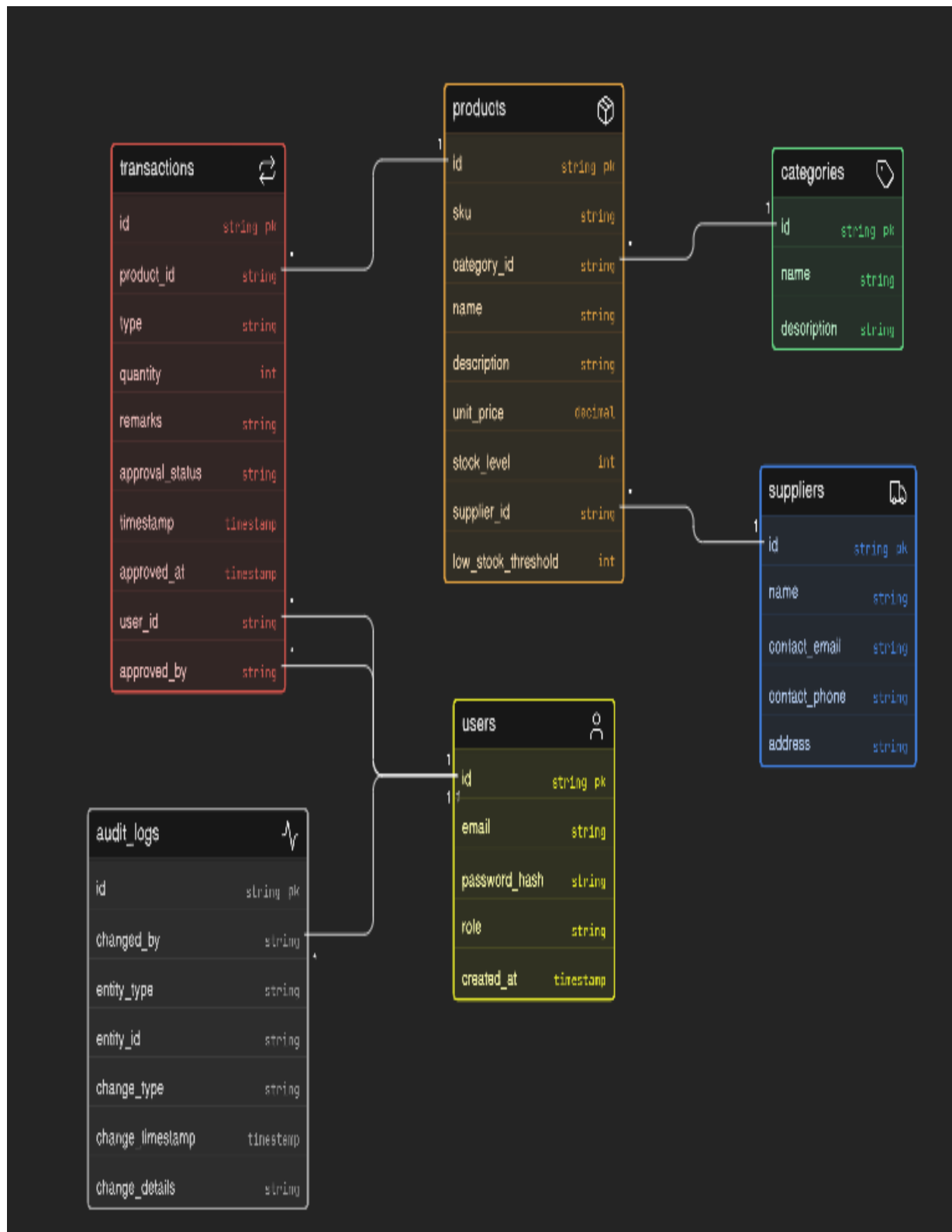
CHAPTER 3: ANALYSIS & DESIGN

3.1 Entity Relationship Diagram (ERD)

The ERD for StockGrid consists of following key entities:

- User: (ID, Email, Password Hash, Role, Timestamp)
- Product: (ID, SKU, Name, Description, CategoryID, UnitPrice, StockLevel, LowStockThreshold)
- Category: (ID, Name, Description)
- Transaction: (ID, ProductID, UserID, Type [IN/OUT], Quantity, Timestamp, Remarks)





3.2 System Architecture

The architectural framework of StockGrid is built upon the Three-Tier Architecture model. This design pattern is strategically implemented to ensure a clean Separation of Concerns, allowing the user interface, business logic, and data storage to operate and scale independently.

1. Presentation Tier (Frontend)

The Presentation Tier is the topmost layer of the application, serving as the primary interface through which users interact with the system.

- **React (Vite):** StockGrid utilizes React to function as a Single Page Application (SPA). This ensures a fluid, highly responsive user experience by updating only the necessary components of the page rather than refreshing the entire browser. Vite is employed as the build tool to provide a high-performance development environment with instant Hot Module Replacement (HMR).
- **Tailwind CSS:** For the visual layer, Tailwind CSS provides a utility-first approach to styling. This ensures that the interface remains lightweight and fully responsive across various devices (desktops, tablets, and mobiles), which is critical for real-time stock monitoring.

- **Responsibility:** This tier is strictly responsible for capturing user inputs via forms, rendering data visualizations (like stock trends), and initiating asynchronous requests to the backend via the RESTful API.

2. Logic Tier (Application/Backend)

Often referred to as the "Brain" of the system, this middle tier processes all business rules and coordinates the application's core functionality.

- **Node.js & Express.js:** The backend is architected using Node.js due to its non-blocking, event-driven I/O model, which is ideal for handling numerous concurrent inventory transactions. Express.js is used to design a structured RESTful API that facilitates seamless communication between the frontend and the database.
- **Business Logic & AI Integration:** This layer handles the critical "heavy lifting," including stock level calculations, transaction logging, and AI-driven insights like demand prediction. It also manages JWT-based authentication to ensure that data access is restricted based on user roles (Admin, Manager, or Staff).
- **Responsibility:** It receives HTTP requests from the Presentation Tier, validates user permissions, processes the data according to business rules, and returns structured JSON responses.

3. Data Tier (Database)

The Data Tier is the foundational layer responsible for the secure and persistent storage of all application data.

- **PostgreSQL:** As a relational database, PostgreSQL ensures ACID compliance (Atomicity, Consistency, Isolation, Durability). This is vital for inventory management to prevent data corruption—for example, ensuring that a stock "OUT" transaction is only recorded if the product actually exists in the database.
- **Supabase:** Supabase provides the hosted PostgreSQL infrastructure. It offers advanced features like Row Level Security (RLS) and real-time database listeners, which allow StockGrid to push instant notifications when stock levels hit a critical threshold.
- **Responsibility:** This layer handles complex query execution, data indexing, and the maintenance of relational integrity between entities such as Users, Products, and Categories.

Visualizing the Flow: When a user adds a new product, the Presentation Tier collects the details; the Logic Tier validates the SKU and user permissions; finally, the Data Tier commits the entry to the Products table, ensuring the system remains the single source of truth for the business.

3.3 Database Requirements & User Interfaces

3.3.1 Database Requirements

The database serves as the persistent backbone of StockGrid. Because the system manages financial-grade transaction data and complex relationships between products, categories, and users, a high-performance Relational Database Management System (RDBMS) is mandatory.

1. **Data Integrity and ACID Compliance** Inventory management requires strict adherence to ACID (Atomicity, Consistency, Isolation, Durability) properties. For instance, when a "Stock OUT" transaction occurs, the system must simultaneously decrement the stock count and create an audit log entry. PostgreSQL ensures that if one part of this process fails, the entire transaction is rolled back, preventing data corruption or "phantom" stock.
2. **Relational Mapping and Constraints** The database is designed to enforce business rules through schema constraints:
 - **Foreign Key Integrity:** Ensures that a transaction cannot be logged for a product that does not exist in the catalog.
 - **Check Constraints:** Programmatic rules prevent stock_level from becoming a negative value, ensuring physical reality is reflected in the data.

- **Unique Indexing:** Enforces unique SKU (Stock Keeping Unit) codes to prevent duplicate product entries.

3. Performance for Analytics and AI To support "Dead Stock Detection" and "Sales Insights," the database must handle complex aggregation queries (e.g., calculating moving averages of sales over 30 days). PostgreSQL's advanced indexing and query optimization allow the AI module to fetch historical data rapidly for demand prediction.

4. Real-time Capabilities with Supabase By utilizing PostgreSQL via Supabase, the system gains "Real-time Listeners." This allows the database to push instant updates to the User Interface the moment a stock threshold is hit, triggering "Low Stock Notifications" without requiring a page refresh.

3.3.2 User Interfaces

The User Interface (UI) of StockGrid is engineered for speed, clarity, and accessibility. The goal is to provide a "Spreadsheet-free" experience that remains intuitive for both warehouse staff and business administrators.

1. 'Mobile-First' Design Philosophy Recognizing that inventory tracking often happens on the warehouse floor rather than at a desk, the UI is built with a Mobile-First approach. Using Tailwind CSS's responsive breakpoints, the interface automatically adapts:

- **On Mobile:** Large touch-friendly buttons for "Quick Stock Adjustments" and a simplified list view.

- **On Desktop:** A comprehensive multi-column dashboard featuring data-rich charts and side-by-side analytics.

2. Tailwind CSS Utility-First Styling Tailwind CSS was selected to create a lightweight, high-performance UI. This utility-first framework allows for:

- **Consistency:** Standardized spacing and color palettes (e.g., Red for "Low Stock," Green for "In-Stock") ensure a professional aesthetic.
- **Low Latency:** Tailwind generates minimal CSS files, which, combined with Vite's rapid build system, results in nearly instantaneous page loads.

3. Core Interface Components

- **Smart Dashboard:** Features "AI-Powered Insights" through visual widgets. It uses interactive charts to display top products and demand trends.
- **Inventory CRUD Portal:** A centralized hub for product management. It includes "Smart Column Mapping" interfaces that use AI to help users import legacy CSV/Excel data into the system.
- **Audit Trail Feed:** A chronological, read-only list of all transactions, providing transparency and accountability for every SKU movement.

- **Role-Based Views:** The UI dynamically changes based on the JWT token; for example, "Staff" users may only see the transaction forms, while "Admins" gain access to the "Role-Based Access Control" and "Financial Export" menus.



3.4 Data Flow Diagram (DFD)

3.4.1 DFD Level 0: Context Diagram

The Level 0 DFD, also known as the Context Diagram, defines the boundary of the StockGrid system. It represents the entire application as a single process and identifies the external entities that exchange information with it.

- **External Entities:**
 - **Admin:** Provides system configurations, user management data, and receives high-level sales and "Dead Stock" analytics.
 - **Staff/Manager:** Inputs product details, SKU information, and transaction logs (IN/OUT). They receive low-stock alerts and transaction confirmations.
 - **AI Module (External Logic):** While part of the backend, logically it acts as a processor that receives historical transaction data and returns "Demand Predictions" and "Reorder Suggestions."
 - **Email Service (Gmail SMTP):** Receives alert triggers from the system to dispatch notifications to the Admin.

3.4.2 DFD Level 1: Detailed Data Flow

The Level 1 DFD breaks down the "StockGrid System" process into sub-processes, showing the internal movement of data between the application logic and the data stores (PostgreSQL).

Process 1.0: Authentication & Authorization

- **Input: User credentials (email/password).**
- **Logic: The system verifies credentials against the User Data Store.**
- **Output: A JWT (JSON Web Token) is issued, which determines the user's role (Admin/Staff) for subsequent processes.**

Process 2.0: Product & Category Management

- **Input: New product details (Name, SKU, Category, Threshold).**
- **Logic: Validates SKU uniqueness and maps categories.**
- **Output: Data is committed to the Products Table.**

Process 3.0: Transaction Processing (Stock IN/OUT)

- **Input: Transaction type, quantity, and product ID.**
- **Logic: This is a critical process where the system updates the stock_level in the Inventory Data Store. It also writes a record to the Audit Trail Store to ensure every movement is traceable.**
- **Output: Updated stock status and success confirmation to the UI.**

Process 4.0: AI-Driven Insights & Reporting

- **Data Source:** Fetches historical data from the Transaction Store.
- **Logic:** The system applies predictive logic to analyze frequency of "OUT" transactions.
- **Output:** Generates a "Reorder Report" and "Smart Insights" (e.g., identifying items that haven't moved in 60 days).

Process 5.0: Notification Engine

- **Data Source:** Monitors the Products Table.
- **Logic:** Compares current_stock ,low_stock_threshold.
- **Output:** If the threshold is breached, it sends data to the Email Service for an automated alert.

3.5 Data Dictionary

The data dictionary for StockGrid ensures that all modules—from the AI insight engine to the frontend transaction forms—interpret data fields identically. Below are the detailed specifications for the primary entities.

3.5.1 User Entity (Table: users)

This table handles authentication and Role-Based Access Control (RBAC).

Field Name	Data Type	Constraints	Description
user_id	UUID	Primary Key, Not Null	Unique identifier for each user.
email	VarChar(255)	Unique, Not Null	Registered email used for login and notifications.
password_hash	Text	Not Null	Encrypted user password for security.
role	Enum	Default: 'Staff'	Access level: 'Admin', 'Manager', or 'Staff'.

Field Name	Data Type	Constraints	Description
created_at	Timestamp	Default: Current_Time	Date and time the account was created.

3.5.2 Product Entity (Table: products)

The core repository for inventory details and AI-assisted tracking.

Field Name	Data Type	Constraints	Description
product_id	UUID	Primary Key, Not Null	Unique identifier for each product.
sku	VarChar(50)	Unique, Not Null	Stock Keeping Unit (e.g., SG-102-BLUE).
name	VarChar(100)	Not Null	Descriptive name of the item.
category_id	UUID	Foreign Key	Links the product to a specific category.
stock_level	Integer	Not Null, ≥ 0	Current physical count of the SKU.
low_threshold	Integer	Default: 10	The count that triggers a "Low Stock" alert.

Field Name	Data Type	Constraints	Description
unit_price	Decimal(10,2)	Not Null, > 0	Unit cost for valuation and analytics.

3.5.3 Transaction Entity (Table: transactions)

The audit trail of every inventory movement within the system.

Field Name	Data Type	Constraints	Description
txn_id	UUID	Primary Key, Not Null	Unique identifier for the movement log.
product_id	UUID	Foreign Key, Not Null	The item involved in the transaction.
user_id	UUID	Foreign Key, Not Null	The person who logged the entry (Audit).
txn_type	Enum	'IN' or 'OUT'	Direction of stock movement.
quantity	Integer	Not Null, > 0	Amount of stock moved.
txn_date	Timestamp	Default: Current_Time	Precise time of the stock update.
remarks	Text	Optional	Comments for auditing

Field Name	Data Type	Constraints	Description
			(e.g., "Supplier Restock").

Significance of the Data Dictionary in StockGrid

1. **Data Validation:** By defining `stock_level` as Not Null and ≥ 0 , the system prevents "ghost inventory" errors where stock counts could mathematically drop into negative numbers.
2. **AI Accuracy:** The AI Demand Prediction model relies on the `txn_date` and `quantity` fields. Standardizing these formats in the data dictionary ensures the AI module receives clean, predictable data for processing.
3. **Scalability:** When adding new features like "Multi-Warehouse Tracking," the Data Dictionary provides a clear blueprint of existing relationships, preventing naming conflicts or data redundancy.
4. **Security:** Defining `password_hash` as a Text field instead of plain text ensures that the system architecture inherently supports industry-standard encryption (like bcrypt or Argon2).

3.6 Table Design

3.6.1 Table: Users

This table acts as the central repository for identity management and access control.

- **Primary Key:** user_id (UUID)
- **Design Rationale:** The table is designed to support Role-Based Access Control (RBAC). By storing a role enum, the system can dynamically adjust the UI and API permissions.
- **Columns:**
 - **user_id:** Unique identifier for the user.
 - **full_name:** The display name of the staff or admin.
 - **email:** Used for authentication and system alerts.
 - **password_hash:** Stores the bcrypt-hashed password (never plain text).
 - **role:** Defines access level (Admin, Manager, Staff).
 - **created_at:** Audit timestamp for account creation.

3.6.2 Table: InventoryItems (Products)

This is the core table of the application. It stores the static and dynamic attributes of every product tracked by StockGrid.

- **Primary Key:** item_id (UUID)

- **Design Rationale:** This table includes a **low_stock_threshold** column, which is the trigger point for the notification engine and AI reorder suggestions.
- **Columns:**
 - **item_id:** Unique identifier for the SKU.
 - **sku_code:** A unique string (e.g., SG-OFF-001) used for physical tracking.
 - **item_name:** The name of the product.
 - **category:** A foreign key or string used to group items.
 - **current_stock:** An integer representing the real-time quantity on hand.
 - **unit_price:** The cost per unit, used for calculating inventory value.
 - **low_stock_threshold:** The minimum quantity allowed before an alert is triggered.
 - **last_updated:** Timestamp of the most recent stock change.

3.6.3 Table: Transactions

The transactions table is an immutable ledger. Once a record is created here, it is never deleted or modified, ensuring a perfect audit trail.

- **Primary Key:** **transaction_id** (UUID)

- **Foreign Keys:** item_id (links to InventoryItems), user_id (links to Users).
- **Design Rationale:** By linking every transaction to a user_id, the system provides accountability, showing exactly who moved what stock and when.
- **Columns:**
 - **transaction_id:** Unique identifier for the log entry.
 - **item_id:** The product being moved.
 - **user_id:** The staff member performing the action.
 - **type:** Enum ('IN' for restock, 'OUT' for sales/usage).
 - **quantity:** The amount of stock being moved.
 - **timestamp:** The exact date and time of the transaction.
 - **notes:** Optional field for context (e.g., "Damaged goods return").

3.6.4 Table: Notifications

This table manages the alerts generated by the system when stock levels hit critical points or when AI detects an anomaly.

- **Primary Key:** notification_id (UUID)
- **Foreign Key:** item_id (links to InventoryItems).

- **Design Rationale:** This allows the system to keep a history of alerts, which the AI module can later analyze to determine how often a product stays in a "Low Stock" state.
 - **Columns:**
 - **notification_id:** Unique identifier for the alert.
 - **item_id:** The product that triggered the alert.
 - **message:** A generated string (e.g., "SKU SG-OFF-001 is below threshold").
 - **status:** Enum ('Unread', 'Read', 'Resolved').
 - **alert_type:** Enum ('Low Stock', 'AI Prediction', 'System').
 - **created_at:** When the alert was triggered.
-

3.6.5 Relational Mapping and Constraints

To maintain the "Single Source of Truth," the following relational rules are enforced at the database level:

1. **One-to-Many (Products to Transactions):** One product can have thousands of transactions, but each transaction record belongs to exactly one product.
2. **One-to-Many (Users to Transactions):** One staff member can log many transactions, providing a clear history of employee activity.

3. **Cascading Rules:** If a product is deleted (though rarely recommended), the system is configured to either restrict the deletion or archive the related transactions to prevent "Orphaned Records."
4. **Check Constraints:** A database-level check is placed on `InventoryItems.current_stock` to ensure it never drops below zero. Any transaction that would result in negative stock is automatically rejected by the Logic Tier.



3.7 Code Design

The code design of StockGrid is centered around modularity and reusability. By separating the application into distinct layers, the system ensures that changes to the user interface do not impact the core business logic or database interactions.

3.7.1 BACKEND ARCHITECTURE: CONTROLLER-SERVICE-ROUTE PATTERN

The Node.js and Express backend follows a decoupled architecture divided into three primary layers:

1. ROUTES (The Entry Points):

This layer is responsible for defining the API endpoints (e.g., /api/products, /api/auth). Its sole responsibility is to receive the incoming HTTP request and hand it over to the appropriate controller.

2. CONTROLLERS (The Orchestrators):

Controllers handle the request and response lifecycle. They extract data from the request body or parameters, call the necessary service functions, and return the final JSON response or error status to the client.

3. SERVICES (The Business Logic):

The Service layer is where the actual "heavy lifting" occurs. This includes:

- Calculating stock movements.
- Interacting with the PostgreSQL database via Supabase.
- Triggering AI models for demand prediction.
- Logic for sending low-stock emails via SMTP.

By keeping this logic in services, it can be reused across different controllers without duplicating code.

3.7.2 FRONTEND ARCHITECTURE: ATOMIC DESIGN PATTERN

The React frontend is organized using the Atomic Design methodology, which breaks the UI down into five distinct levels of complexity:

1. ATOMS:

The smallest building blocks that cannot be broken down further, such as Buttons, Inputs, Icons, and Typography styles.

2. MOLECULES:

Groups of atoms bonded together to function as a unit. For example, a Search Bar molecule consists of an Input atom and a Button atom.

3. ORGANISMS:

Complex UI sections composed of groups of molecules and/or atoms. In StockGrid, the "Sidebar" or the "Analytics Dashboard Card" are organisms.

4. TEMPLATES:

Page-level objects that place components into a layout. They define the underlying content structure, such as the "Dashboard Layout" which contains the Sidebar, Header, and Main Content area.

5. PAGES:

The final instances of templates where actual data (from the API) is injected. Examples include the "Inventory List Page" or "AI Insights Page."

3.7.3 DESIGN ADVANTAGES

- Maintainability: Bugs can be isolated to specific layers (e.g., a calculation error is fixed in the Service layer without touching the UI).
- Scalability: New features, like barcode scanning, can be added as new components or services without restructuring the entire application.

- Collaboration: Developers can work on the frontend (Atoms/Molecules) independently of the backend (Services/Routes).



3.8 Menu Screens & Input Screens

The user interface of StockGrid is designed to minimize cognitive load while maximizing data entry efficiency. The following sections detail the primary navigation structures (Menu Screens) and the data capture points (Input Screens).

3.8.1 MENU SCREENS & NAVIGATION ARCHITECTURE

The menu system is built around a persistent sidebar navigation model that adapts based on the user's authenticated role.

1. GLOBAL NAVIGATION SIDEBAR:

The sidebar serves as the primary gateway to all system modules. It uses active-state highlighting to orient the user within the application.

- Dashboard Link: Direct access to real-time analytics.
- Inventory Link: Navigates to the comprehensive SKU management table.
- Transactions Link: Accesses the historical audit trail.
- AI Insights Link: (Admin/Manager only) Navigates to demand forecasting.
- Settings Link: System configuration and profile management.

2. ROLE-BASED MENU FILTERING:

The menu system dynamically filters available options based on the JWT claims. For example, the "User Management" and "Data Export" options are automatically hidden from 'Staff' level users to prevent unauthorized administrative actions.

3. MOBILE HAMBURGER MENU:

On smaller screens, the sidebar retracts into a hamburger menu icon to preserve screen real estate, ensuring that the interface remains functional for warehouse staff using mobile devices.

3.8.2 INPUT SCREENS: DATA CAPTURE & VALIDATION

Input screens are engineered with strict validation logic to ensure that only clean, accurate data enters the PostgreSQL database.

1. LOGIN & SIGNUP INTERFACES:

- Functionality: Secure portals for user authentication.
- Input Fields: Email (with regex validation) and Password (masked).

- Security Logic: Upon submission, the backend validates credentials and issues a JWT. If validation fails, the input screen provides non-specific error messages (e.g., "Invalid Credentials") to prevent account harvesting.

2. PRODUCT MANAGEMENT FORM:

This is the primary input screen for populating the inventory catalog.

- Key Fields: Product Name, SKU Code (Auto-generated or Manual), Category Dropdown, Unit Price, and Low-Stock Threshold.

- Validation: The "SKU Code" field performs a real-time check against the database to ensure uniqueness. The "Threshold" field must be a positive integer.

3. IN/OUT TRANSACTION MODAL:

To ensure rapid stock updates, the system uses a modal-based input screen that can be triggered from any page.

- Field 1 (Type): A toggle switch to select 'Stock In' or 'Stock Out'.

- Field 2 (Quantity): A numeric input that prevents negative values.

- Field 3 (Audit Comments): A text area for mandatory remarks (e.g., "Received from Supplier X" or "Order #10245").

- Logic: For 'Stock Out' transactions, the input screen displays a warning if the requested quantity exceeds the current 'In-Stock' amount.

4. SMART COLUMN MAPPING (IMPORT SCREEN):

When users upload a CSV or Excel file, the AI-assisted mapping screen appears.

- Logic: It lists the columns from the uploaded file and uses AI logic to suggest matches to StockGrid's database fields (e.g., matching "Product_ID" in the file to "SKU" in the system). Users can manually override these suggestions before the final import.

3.8.3 DESIGN PRINCIPLES FOR INPUTS

- Immediate Feedback: Input fields use color-coded borders (Green for valid, Red for invalid) to provide instant user feedback.

- Keyboard Optimization: All forms support 'Tab' navigation and 'Enter' to submit, allowing for high-speed data entry without a mouse.

- Error Handling: Inline error messages clearly explain why an input was rejected, reducing user frustration during bulk stock updates.

3.9 Test Procedures and Implementation

The reliability of an inventory management system is paramount, as errors in data processing can lead to financial discrepancies. Therefore, StockGrid underwent a rigorous testing lifecycle to ensure that the logic tier, data tier, and AI components function harmoniously.

3.9.1 TEST PROCEDURES

The testing strategy followed a bottom-up approach, starting from individual code units and progressing to the entire integrated system.

1. UNIT TESTING (BACKEND APIs):

Each API endpoint in the Node.js/Express backend was tested in isolation.

- Objective: To ensure that specific functions, such as 'calculateTax()' or 'validateSKU()', return the expected output for a given input.

- Tools: Jest and Supertest were utilized to simulate HTTP requests.

- Scenario Example: Testing the /api/transactions/out endpoint to ensure it correctly rejects a request if the 'quantity' field is a string instead of an integer.

2. INTEGRATION TESTING (STOCK CALCULATION LOGIC):

This phase focused on the interaction between the Logic Tier and the Data Tier (PostgreSQL).

- Objective: To verify that a 'Stock IN' transaction correctly increments the 'current_stock' value in the Products table and simultaneously creates an entry in the Audit Trail.

- Key Test: Ensuring that if a database connection fails mid-transaction, the system performs a 'Rollback' to maintain data integrity.

3. AI COMPONENT TESTING (SMART COLUMN MAPPING):

Since StockGrid uses AI to assist in bulk imports, specialized testing was performed on the mapping engine.

- Objective: To ensure the AI accurately identifies column headers from diverse CSV/Excel formats.

- Methodology: The system was fed 50+ different CSV variations (e.g., some using 'Prod_ID' and others using 'ItemCode'). The test passed if the AI correctly mapped these to the 'SKU' field with over 95% accuracy.

4. USER ACCEPTANCE TESTING (UAT):

The system was tested by representative users (Staff and Admins) to evaluate the 'Mobile-First' UI.

- Feedback Loop: Testing focused on the ease of logging transactions on mobile devices and the clarity of 'Low Stock' alert notifications.

3.9.2 IMPLEMENTATION STRATEGY

The implementation of StockGrid followed a phased rollout to minimize disruption and ensure a stable transition from manual spreadsheets.

PHASE 1: REQUIREMENT ANALYSIS & PROTOTYPING

- Detailed consultation with stakeholders to define SKU structures and threshold requirements.
- Development of a low-fidelity wireframe to finalize the 'Atomic' UI components.

PHASE 2: DEVELOPMENT & ENVIRONMENT SETUP

- Configuration of the Supabase (PostgreSQL) environment.
- Setting up the Gmail SMTP server for automated notification delivery.

- Coding of the core CRUD modules and JWT authentication logic.

PHASE 3: DATA MIGRATION & AI TRAINING

- Legacy data from existing spreadsheets was cleaned and imported using the AI-assisted mapping tool.
- The AI demand prediction model was initialized with historical transaction data to begin generating reorder suggestions.

PHASE 4: DEPLOYMENT & FINAL HANDOVER

- The application was deployed as a SaaS (Software as a Service) platform.
- Final documentation, including the User Manual and Operations Manual, was provided to the staff.
- A "Post-Implementation Review" was conducted two weeks after launch to address minor UI bugs and optimize query performance.

CHAPTER 4: USER MANUAL

4.1 User Manual

The User Manual for StockGrid is designed to guide users through the operational workflow of the system, ensuring that both administrative and warehouse staff can leverage the full power of the AI-driven features.

4.1.1 GETTING STARTED: AUTHENTICATION

The security of your inventory data is paramount. StockGrid utilizes a secure authentication gateway to prevent unauthorized access.

- Accessing the Portal: Open the application URL in any modern web browser.
- Login Process: Enter your registered organizational email and secure password.
- Role-Based Access: Once authenticated, the system generates a JWT (JSON Web Token) that dynamically configures the interface.
 - Admins see financial analytics and user management.
 - Staff see simplified transaction and stock-counting tools.

4.1.2 NAVIGATION AND LAYOUT OVERVIEW

After logging in, users are presented with a centralized navigation sidebar designed for rapid access to core modules:

- Dashboard: The primary landing page for high-level monitoring.
- Inventory: A detailed list of all SKUs and product specifications.
- Transactions: A read-only audit trail of every stock movement.
- AI Insights: Advanced tools for demand prediction and reorder suggestions.
- Alerts: A dedicated notification feed for items requiring immediate attention.

4.1.3 DASHBOARD OPERATIONS

The Dashboard serves as the "Nerve Center" of StockGrid. It provides at-a-glance visibility into business health.

- Smart Insights: Visual widgets display "Dead Stock Detection" (items not sold in 60+ days) and "Top Performing Products."
- Low Stock Alerts: A live ticker shows products that have breached their defined safety thresholds.
- Quick Adjust Buttons: For rapid updates, the dashboard features 'Quick Adjust' controls. These allow staff to log a simple IN/OUT transaction without navigating away from the main overview.

4.1.4 PRODUCT MANAGEMENT (CRUD)

Adding and maintaining the product catalog is a core functional area.

- Creating Products: Click the 'Add Product' button in the Inventory module.
- Defining Attributes: Users must provide a unique SKU, Product Name, Category, and Unit Price.
- Smart Threshold Handling: A critical step is setting the 'Low Stock Threshold.' This value determines when the system will trigger an automated alert and send an email notification via the SMTP service.
- Editing/Deleting: Existing records can be modified to update prices or archived if they are no longer being tracked.

4.1.5 MANAGING STOCK MOVEMENTS (IN / OUT)

StockGrid replaces manual spreadsheets with a structured transaction-based approach.

- Stock IN (Restocking): Used when new inventory arrives at the warehouse. Users select the product, enter the received quantity, and provide a comment (e.g., "Supplier Shipment #402").

- Stock OUT (Sales/Usage): Used when stock is removed for sales or internal use. The system performs a real-time calculation to ensure the remaining count is accurate.
- Audit Trail: Every movement is permanently logged with a timestamp and the ID of the user who performed the action, ensuring 100% accountability.

4.1.6 AI FEATURES AND DATA IMPORT

StockGrid uses AI to simplify complex data tasks.

- Smart Column Mapping: When uploading legacy data from Excel or CSV files, the AI automatically identifies columns (e.g., matching "Inventory_ID" to "SKU") to streamline the migration process.
- Demand Prediction: Located in the 'AI Insights' tab, this feature analyzes historical transaction trends to predict future stock requirements, helping managers avoid overstocking or stockouts.

4.2 Operations Manual / Menu Explanation

The Operations Manual provides a detailed functional breakdown of the StockGrid software modules. It explains the purpose of each menu item and the specific business intelligence provided by the AI-driven backend.

4.2.1 MODULE 1: DYNAMIC DASHBOARD

The Dashboard is the default landing page and acts as the "Nerve Center" for business administrators.

- High-Level Overview: Displays real-time KPIs (Key Performance Indicators) including Total Inventory Value, Total SKUs, and Current Stock-Outs.
- Sales Insights: Integrates with the transaction history to visualize sales velocity. It helps managers identify which products are moving fastest through graphical bar charts and line graphs.
- Dead Stock Detection: One of the most critical operational tools. The system highlights items that have had zero "OUT" transactions for a period of 60-90 days, allowing the business to clear capital tied up in stagnant inventory.

4.2.2 MODULE 2: SEARCHABLE INVENTORY LIST

The Inventory module is the primary workspace for warehouse managers and staff.

- Search & Filter Engine: A high-performance table that allows users to search by SKU code, Product Name, or Category. It supports multi-factor filtering to isolate specific brands or price ranges.

- Real-Time SKU Tracking: Displays the exact physical count of every item. The status is color-coded:

 - Green: Healthy Stock.

 - Yellow: Below Threshold (Warning).

 - Red: Out of Stock (Critical).

- Smart Mapping (Bulk Imports): This operational feature allows for the migration of large datasets. Users upload a CSV/Excel file, and the AI-driven mapping interface identifies the correct columns to ensure seamless data ingestion without manual re-entry.

4.2.3 MODULE 3: AI INSIGHTS ENGINE

This module separates StockGrid from traditional inventory tools by providing predictive rather than just reactive data.

- Demand Prediction Models: The AI analyzes the frequency, seasonality, and volume of past transactions. It provides a forecast of how much stock will likely be needed for the upcoming month.

- Reorder Suggestions: Based on the demand forecast and current lead times, the system generates a "Smart Shopping List." This suggests exactly which SKUs to reorder and in what quantity to maintain optimal stock levels without over-purchasing.
- Trend Analysis: Identifies emerging patterns in inventory usage, alerting management to sudden spikes in demand for specific categories.

4.2.4 MODULE 4: AUDIT & TRANSACTION LOGS

Ensures 100% transparency and accountability for all warehouse operations.

- Comprehensive Audit Trail: Every single "IN" and "OUT" movement is logged with the User ID of the person responsible.
- Verification: Operations managers can review "Remarks" on transactions to verify the reason for adjustments (e.g., "Supplier Return" or "Damaged on Floor").

4.3 Forms and Report Specifications

This section details the structured output and notification mechanisms of StockGrid. The system is designed to transform raw transaction data into actionable business intelligence through standardized reporting and automated alerting.

4.3.1 INPUT FORM SPECIFICATIONS

Every data entry point in StockGrid is governed by strict form specifications to maintain data integrity across the PostgreSQL database.

1. PRODUCT ENTRY FORM:

- Purpose: To register new SKUs into the system.
- Field Specifications:
 - Item Name: Text (Alpha-numeric), Required.
 - SKU Code: String (Unique), Required.
 - Initial Stock: Integer (Default: 0), Required.
 - Low-Stock Threshold: Integer (User-defined), Required.
 - Category: Dropdown Selection (Pre-defined), Required.

2. STOCK ADJUSTMENT FORM (IN/OUT):

- Purpose: To record physical inventory movements.

- Field Specifications:
 - Transaction Type: Radio Button (IN / OUT).
 - Quantity: Integer (Positive only), Required.
 - Reason/Remarks: Text Area, Optional (e.g., "Monthly Restock").

4.3.2 REPORT SPECIFICATIONS (OUTPUT)

StockGrid provides robust reporting capabilities to assist in financial auditing and inventory planning. All reports are generated dynamically from the latest database state.

1. INVENTORY VALUATION REPORT:

- Format: CSV / Excel.
- Content: A list of all products, their current stock levels, unit prices, and the total calculated value (Stock x Price).
- Usage: Used for end-of-month financial balancing.

2. DEAD STOCK & SLOW-MOVING REPORT:

- Format: System Dashboard View / CSV.

- Content: Items with zero "OUT" transactions over a 60-day period.
- Usage: Helps management identify capital tied up in non-moving goods.

3. AI REORDER SUGGESTION REPORT:

- Format: Excel.
- Content: Predicted demand vs. current stock, including suggested order quantities to meet future needs.

4.3.3 AUTOMATED NOTIFICATION SPECIFICATIONS

The system features a proactive alerting engine that ensures the Admin is notified immediately of critical inventory status changes.

1. LOW-STOCK ALERT ENGINE:

- Trigger: Occurs the moment a "Stock OUT" transaction causes the current_stock to fall below the defined low_stock_threshold.
- Protocol: SMTP (Simple Mail Transfer Protocol).
- Service Provider: Gmail SMTP Server.

- Content: A structured email containing the Product Name, SKU, Current Stock, and a direct link to the reorder page.

2. SYSTEM LOGS & ERROR NOTIFICATIONS:

- Trigger: Failed login attempts or database connectivity issues.

- Recipient: Designated System Administrator email.

3. SMART COLUMN MAPPING LOGS:

- Purpose: To provide a summary of bulk imports, showing how many rows were successfully mapped and if any entries were rejected due to format mismatches.

Drawbacks and Limitations

While StockGrid provides a robust, AI-enhanced framework for inventory management, certain technical and operational limitations exist in the current version. These limitations serve as the baseline for future versioning and system scaling.

1. DEPENDENCY ON ACTIVE INTERNET CONNECTIVITY (SaaS MODEL):

The primary drawback is the system's reliance on a continuous internet connection. Since StockGrid is built on a SaaS (Software as a Service) architecture with a cloud-hosted PostgreSQL database (Supabase), any disruption in network connectivity prevents real-time data synchronization.

- Impact: Warehouse staff in areas with poor connectivity may experience latency or inability to log transactions, which could lead to temporary data discrepancies between physical and digital stock.

2. ABSENCE OF MULTI-WAREHOUSE TRACKING:

In its initial version, the database schema is architected for a single centralized inventory location. It lacks the logic to distinguish between multiple physical warehouses or branch locations.

- Impact: Large-scale businesses with distributed storage cannot track stock transfers between locations or view localized stock levels, limiting the system's utility for enterprise-level logistics.

3. HARDWARE INTEGRATION LIMITATIONS:

The current system relies on manual data entry or bulk CSV imports. It does not yet support direct integration with hardware peripherals such as industrial barcode scanners or RFID (Radio Frequency Identification) receivers.

- Impact: This increases the time required for high-volume stock IN/OUT processing and maintains a minor risk of manual entry errors.

4. CONCURRENT EDIT CONFLICTS:

While the system supports multi-user roles, the initial version uses a standard RESTful polling approach. In extremely high-concurrency environments (e.g., hundreds of users updating the same SKU simultaneously), there is a theoretical risk of race conditions, although PostgreSQL's ACID compliance mitigates data corruption.

5. AI TRAINING DATA REQUIREMENTS:

The 'Smart Insights' and 'Demand Prediction' features require a significant baseline of historical transaction data (typically 3-6 months) to achieve high accuracy.

- Impact: For new businesses or those with erratic sales patterns, the AI-driven reorder suggestions may initially be less reliable until a sufficient data volume is established.

Proposed Enhancements

The current deployment of StockGrid serves as a robust foundation for inventory management. To evolve the system into a comprehensive Enterprise Resource Planning (ERP) solution, the following enhancements are proposed for future development cycles.

1. MULTI-CURRENCY AND INTERNATIONALIZATION (i18n) SUPPORT:

To support global supply chains, future versions will have a multi-currency engine.

- **Functionality:** The system will integrate with real-time exchange rate APIs to allow users to purchase stock in one currency (e.g., USD) and sell in another (e.g., INR), automatically calculating the conversion at the time of transaction.

- **Impact:** This will allow StockGrid to be used by businesses with international suppliers without manual conversion errors.

2. HARDWARE INTEGRATION: BARCODE & QR CODE SCANNING:

To move away from manual SKU entry, the next phase will implement direct hardware integration.

- Functionality: The web interface will utilize the 'Webcam API' or dedicated USB barcode scanners to instantly identify products.
- Logic: Scanning a code will automatically trigger the 'Stock Adjustment' modal and pre-fill the SKU, reducing the time required for warehouse audits by over 70%.

3. MOBILE APPLICATION (REACT NATIVE):

While the current UI is 'Mobile-First,' a dedicated native application will provide a superior performance profile.

- Functionality: Developed using React Native, the mobile app will offer offline-first capabilities, allowing staff to log transactions even without an internet connection, with data syncing once back online.

- Features: Native push notifications for low-stock alerts and the ability to use the smartphone's high-resolution camera as a professional-grade scanner.

4. ADVANCED AI PREDICTIVE ANALYTICS:

The next iteration of the AI module will move beyond simple demand prediction.

- Seasonal Forecasting: The AI will be trained to recognize seasonal trends (e.g., holiday spikes or monsoon-related demand shifts in Rajasthan) to adjust reorder suggestions automatically.

- Automated Procurement: Future versions will include the option to automatically generate Purchase Orders (POs) and email them to suppliers once stock hits the critical threshold.

5. MULTI-WAREHOUSE AND BIN LOCATION TRACKING:

The database schema will be upgraded to support a hierarchical location model.

- Functionality: Users will be able to track stock across multiple geographic locations (e.g., Jaipur Warehouse vs. Jodhpur Warehouse) and even drill down to specific "Bin Locations" or "Shelf Numbers" within a single facility.

6. THIRD-PARTY LOGISTICS (3PL) INTEGRATION:

StockGrid will offer API hooks to integrate with shipping and logistics providers.

- Functionality: Real-time tracking of incoming shipments and automated updates to the 'Inventory List' once the logistics provider confirms delivery at the warehouse gate.

Conclusions

The development and implementation of StockGrid mark a significant transition from conventional, reactive inventory tracking to a modern, proactive management ecosystem. By successfully integrating a high-performance full-stack architecture with AI-driven analytics, the project achieves its primary goal of digitizing and optimizing the supply chain for small-to-medium enterprises.

1. ARCHITECTURAL SYNERGY:

The project demonstrates that the combination of React (Frontend), Node.js (Logic), and PostgreSQL (Data) provides the stability and speed required for real-time business operations. The 'Three-Tier Architecture' ensured a clear separation of concerns, making the system both robust and easy to maintain during the development lifecycle.

2. IMPACT OF AI INTEGRATION:

One of the core achievements of StockGrid is proving that AI is not just for large-scale corporations. By implementing 'Smart Column Mapping' and 'Demand Prediction,' the system effectively reduced the time spent on administrative data entry and provided managers with actionable insights to prevent capital being locked in 'Dead Stock.'

3. ENHANCED OPERATIONAL EFFICIENCY:

The transition from manual spreadsheets to a centralized SaaS model has eliminated common errors such as duplicate SKU entries and mismatched stock counts. The automated notification engine, powered by Gmail SMTP, ensures that stockouts are minimized, thereby protecting the business from lost sales opportunities.

4. ACADEMIC AND PRACTICAL LEARNING:

From a developmental perspective, this project provided deep practical exposure to modern software engineering principles, including JWT-based security, RESTful API design, and 'Atomic' UI component structures. It successfully bridges the gap between academic theory and industrial application, as required by the MCA curriculum.

FINAL SUMMARY:

StockGrid stands as a comprehensive solution that transforms inventory management from a tedious clerical task into a data-driven business process.

While future enhancements like mobile scanning and multi-warehouse support will further scale the system, the current version successfully demonstrates how modern technology can drive operational excellence and transparency in the hardware and solar industry.

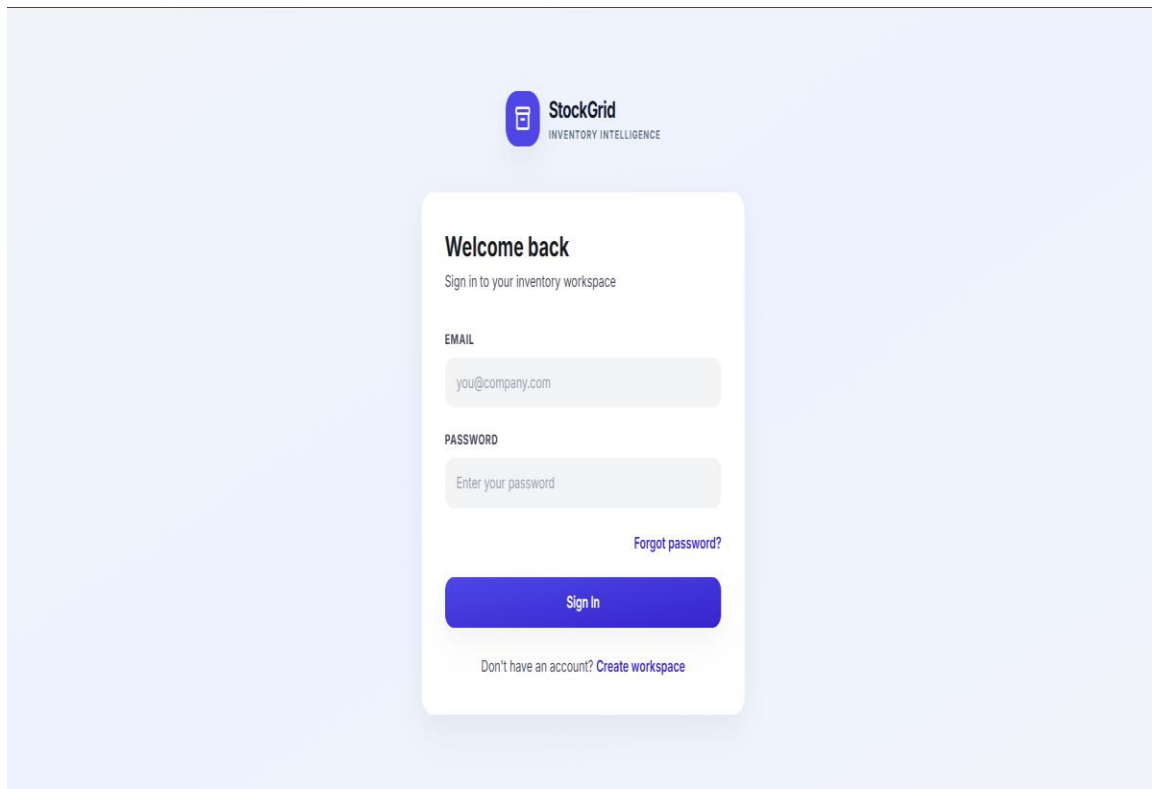
Bibliography

1. React Documentation (reactjs.org)
2. Node.js Design Patterns - Mario Casciaro
3. PostgreSQL: Up and Running - Regina Obe
4. Tailwind CSS Official Documentation.



Annexures

Annexure 1: Input Forms with data



The image shows a login form for StockGrid, a company specializing in inventory intelligence. The form is centered on a light blue background. At the top, the StockGrid logo is displayed, consisting of a purple square with a white grid icon and the text "StockGrid" and "INVENTORY INTELLIGENCE". Below the logo, the heading "Welcome back" is followed by the instruction "Sign in to your inventory workspace". The form contains two input fields: "EMAIL" with the placeholder "you@company.com" and "PASSWORD" with the placeholder "Enter your password". A link "Forgot password?" is located below the password field. A blue "Sign In" button is positioned below the input fields. At the bottom, a link "Don't have an account? Create workspace" is provided.

StockGrid
INVENTORY INTELLIGENCE

Welcome back
Sign in to your inventory workspace

EMAIL
you@company.com

PASSWORD
Enter your password

[Forgot password?](#)

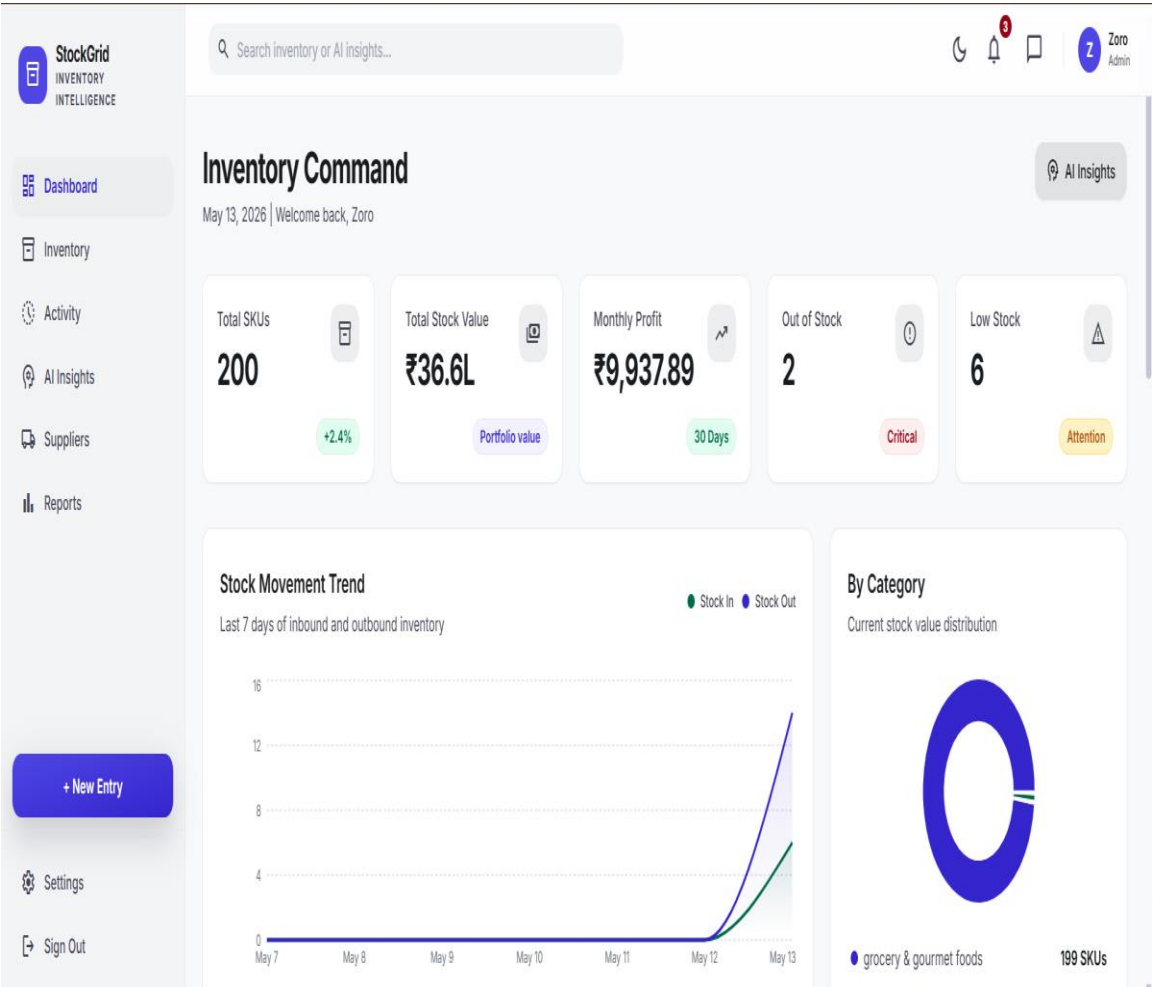
[Sign In](#)

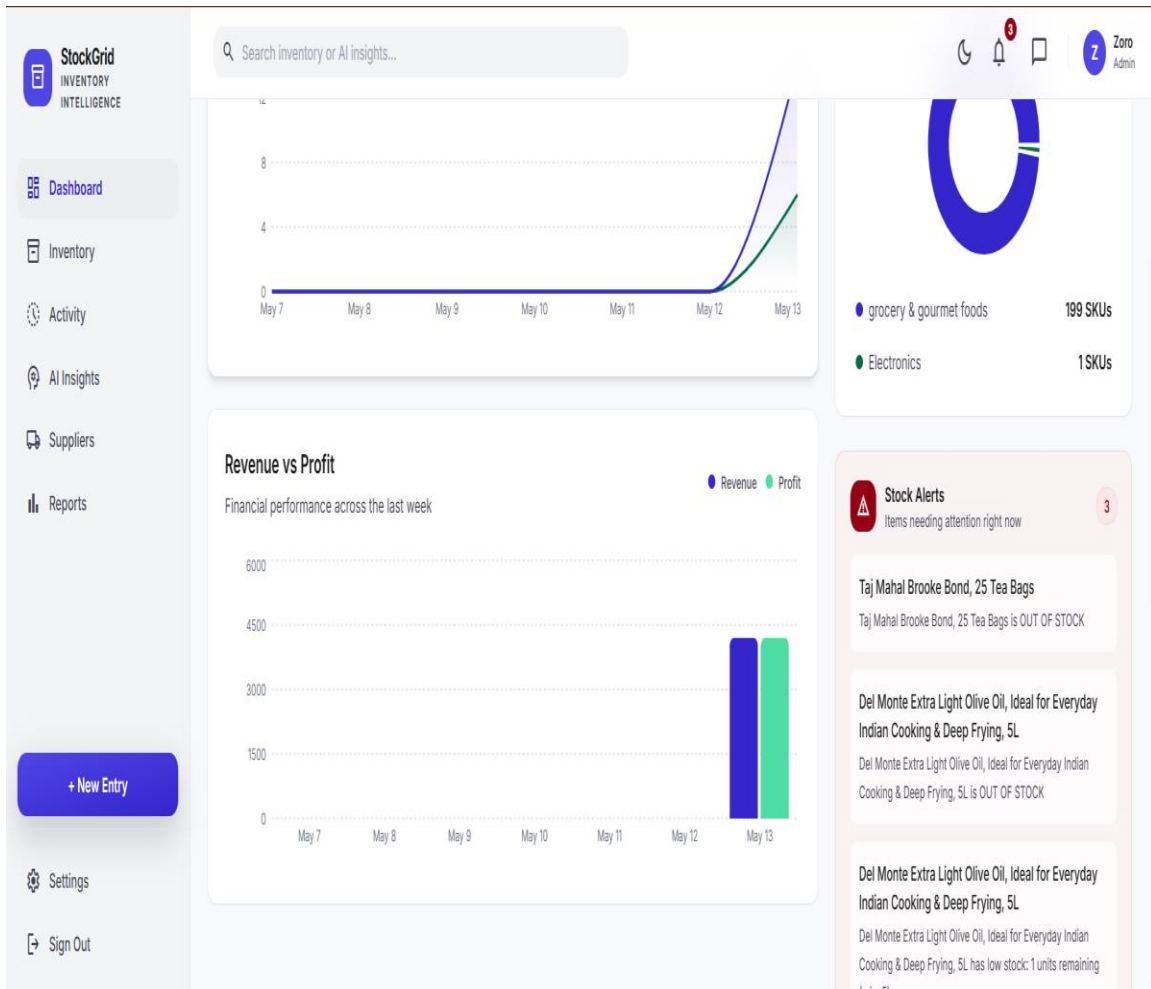
Don't have an account? [Create workspace](#)



68

Annexure 2: Output Reports with Data







StockGrid

INVENTORY INTELLIGENCE

Dashboard

Inventory

Activity

AI Insights

Suppliers

Reports

+ New Entry

Settings

Sign Out

Q Search inventory or AI insights...

May 7

May 8

May 9

May 10

May 11

May 12

May 13

Del Monte Extra Light Olive Oil, Ideal for Everyday Indian Cooking & Deep Frying, 5L

Del Monte Extra Light Olive Oil, Ideal for Everyday Indian Cooking & Deep Frying, 5L has low stock: 1 units remaining (min: 5)

Manage alerts

AI Analysis

Quick recommendations from current dashboard data

Generate a quick read on performance, risks, and opportunities from your latest inventory data.

Generate insights

Recent Transactions

View all

Latest inventory movements across your workspace

+	Manna Millets 5kg - Natural Grains Combo Pack of 5 Foxtail 1kg, Kodo 1kg, Little 1kg, Barnyard 1...	+6
	Stock adjustment	May 13
-	Shree Gulab Roasted Gram Flour Sattu 1Kg (500Gms*2)	-14
	Stock adjustment	May 13
-	Taj Mahal Brooke Bond, 25 Tea Bags	-16
	Stock adjustment	Apr 28
+	Typhoo Uplifting Lemon Grass Green Tea Bags (25 Tea Bags)	+25
	Stock adjustment	Apr 26



StockGrid

INVENTORY INTELLIGENCE

Dashboard

Inventory

Activity

AI Insights

Suppliers

Reports

+ New Entry

Settings

Sign Out

Search inventory or AI insights...







Zoro

Admin

INVENTORY > PRODUCTS

Products

Track, update, and review the items in your catalog.

Import

Export

+ Add Product

Search name or SKU...

All Categories

All Status

All Suppliers

PRODUCT	CATEGORY	SUPPLIER	STOCK LEVEL	UNIT PRICE	STATUS
wireless headphone boat SKU: WH08T	Electronics	Supplier I	20 units 100%	₹1,500.00	In Stock
Typhoo Uplifting Lemon Grass Gree... SKU: GRO-TYPUPL-998	grocery & gourmet foods	Supplier H	45 units 100%	₹210.00	In Stock
Shree Gulab Roasted Gram Flour Sa... SKU: GRO-SHRGUL-139	grocery & gourmet foods	Supplier B	50 units 100%	₹300.00	In Stock
Henfruit Protein Max Eggs, Pack of ... SKU: GRO-HENPRO-938	grocery & gourmet foods	Supplier A	90 units 100%	₹450.00	In Stock

72



StockGrid

INVENTORY INTELLIGENCE

Dashboard

Inventory

Activity

AI Insights

Suppliers

Reports

+ New Entry

Settings

Sign Out

Search inventory or AI insights...







Zoro

Admin

Stock Movements

6 transactions in last 30 days

Export Logs

+ Manual Adjustment

STOCK IN (300)

+31

STOCK OUT (300)

-53

ACTIVE ALERTS

3

Movements

Alerts (3)

All Products

All Types

dd-mm-yyyy

dd-mm-yyyy

DATE & TIME	TYPE	PRODUCT	QUANTITY	REASON	REFERENCE	BY
May 13, 2026 7:31 PM	+ Stock In	Manna Millets 5kg - Natural Grains Combo Pack of 5 Foxtail 1kg, Kodo 1kg, Little 1kg, Barnyard 1kg, Proso 1kg Native L... GRO-MANMIL-994	+6	Restock	-	Zoro
May 13, 2026 7:31 PM	- Stock Out	Shree Gulab Roasted Gram Flour Sattu 1kg (500Gms*2) GRO-SHRGUL-139	-14	Transfer Out	-	Zoro

73

StockGrid

INVENTORY INTELLIGENCE

Dashboard

Inventory

Activity

AI Insights

Suppliers

Reports

+ New Entry

Settings

Sign Out

Search inventory or AI insights...

AI Inventory Assistant

Powered by Gemini 2.0 Flash

Reorder

Dead Stock

Which products are at risk of stockout in the next 30 days?

What are my top-selling categories?

Show me reorder recommendations

Identify dead stock items

How can I reduce inventory carrying costs?

What should I reorder this week?

Good day! I'm your AI inventory assistant powered by Gemini. I can analyze your inventory data, predict demand, identify dead stock, suggest reorder quantities, and answer any inventory-related questions. What would you like to explore?

7:36 PM

Ask about inventory, demand, reorders, dead stock...

Press Enter to send - Shift+Enter for new line

74

**StockGrid**
INVENTORY
INTELLIGENCE

Dashboard

Inventory

Activity

AI Insights

Suppliers

Reports

+ New Entry

Settings

Sign Out

Search inventory or AI insights...

3

Zoro Admin

NETWORK > SUPPLIERS

Suppliers

9 suppliers in your network

Import

Export

Add Supplier



Supplier A
a@supplier.com
9876543210
Lead time: 3 daysProducts: 49



Supplier B
b@supplier.com
9876543211
Lead time: 5 daysProducts: 22



Supplier C
c@supplier.com
9876543212
Lead time: 4 daysProducts: 27



Supplier D
d@supplier.com
9876543213
Lead time: 6 daysProducts: 22

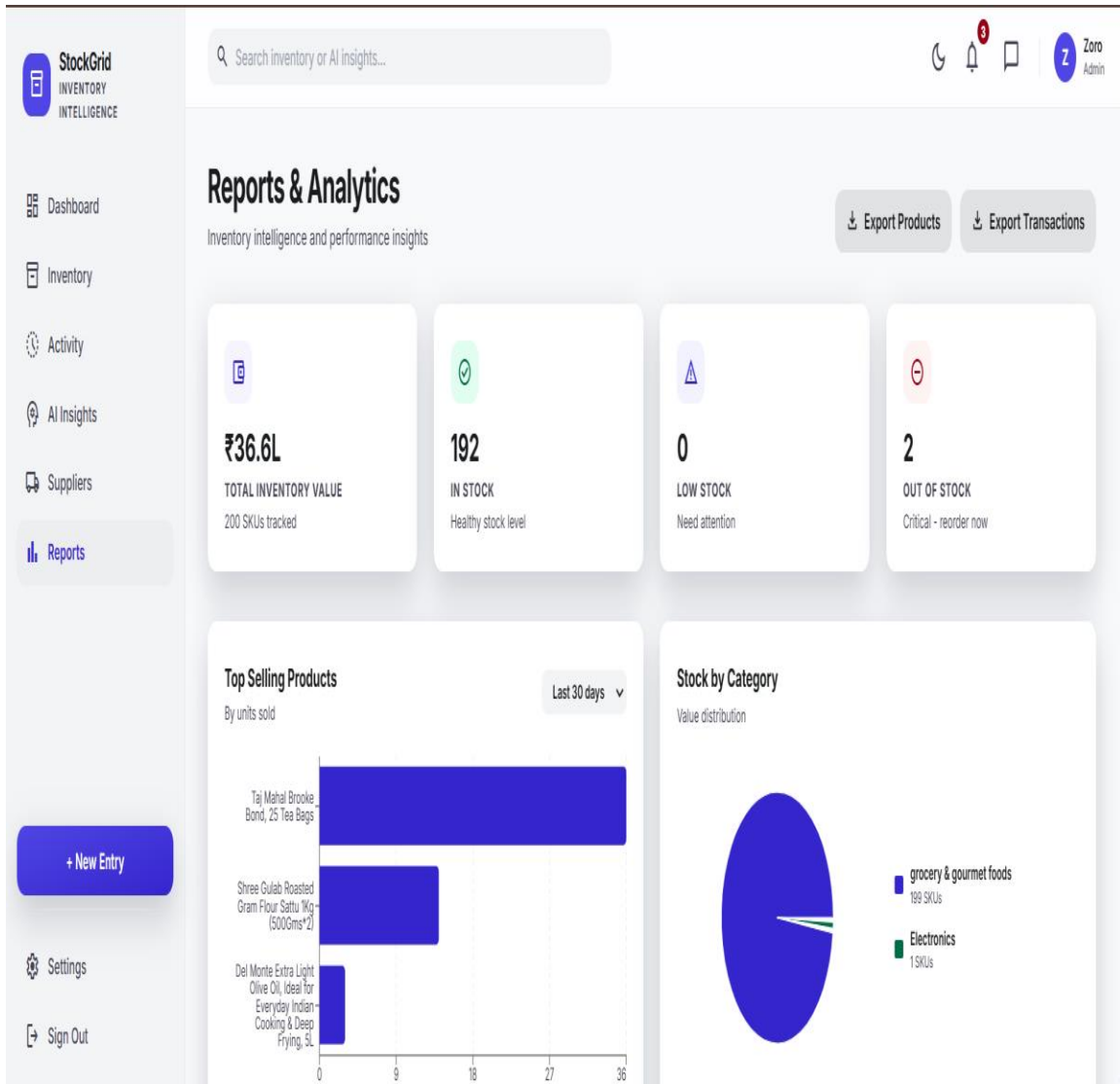


Supplier E
e@supplier.com
9876543214
Lead time: 2 daysProducts: 23



Supplier F
f@supplier.com
9876543215
Lead time: 7 daysProducts: 16

75



+ New Entry

⚙️ Settings

🚪 Sign Out

StockGrid

INVENTORY INTELLIGENCE

Dashboard

Inventory

Activity

AI Insights

Suppliers

Reports

+ New Entry

Settings

Sign Out

Q Search inventory or AI insights...

Z

Zoro Admin

Settings

Manage your profile and workspace

Profile

Team

About

Invite Team Member

Add managers and staff to your workspace

FULL NAME *

ROLE *

Jane Smith

Staff

EMAIL *

TEMP PASSWORD *

jane@company.com

They should change this

Role permissions:

Staff — Can view and record stock movements

Manager — Can add/edit products and suppliers

Admin — Full access including deletion and team management

Add Team Member

StockGrid

INVENTORY INTELLIGENCE

Dashboard

Inventory

Activity

AI Insights

Suppliers

Reports

+ New Entry

Settings

Sign Out

Search inventory or AI insights...

Zoro

Admin

Invite Team Member

Add managers and staff to your workspace

FULL NAME *

ROLE *

Jane Smith

Staff

EMAIL *

TEMP PASSWORD *

jane@company.com

They should change this

Role permissions:

Staff — Can view and record stock movements

Manager — Can add/edit products and suppliers

Admin — Full access including deletion and team management

Add Team Member

Team Members

Z

Zoro

panwarapptesting@gmail.com · admin

79

Annexure 3: Sample Code

Backend

authController.js

```
import { asyncHandler } from '../utils/asyncHandler.js';

import {
  deactivateTeamMember,
  deleteOrganization,
  inviteTeamMember,
  listTeamMembers,
  loginUser,
  reactivateTeamMember,
  registerUser,
} from '../services/authService.js';

import {
  PASSWORD_RESET_REQUEST_RESPONSE,
  requestPasswordReset,
  resetPasswordWithToken,
} from '../services/passwordResetService.js';

export const register = asyncHandler(async (req, res) => {
  const result = await registerUser(req.validated.body);
  res.status(201).json(result);
});
```

```
export const login = asyncHandler(async (req, res) => {  
  const result = await loginUser(req.validated.body);  
  res.json(result);  
});
```

```
export const forgotPassword = asyncHandler(async (req, res) => {  
  const { email } = req.validated.body;  
  const message = await requestPasswordReset(email);  
  res.json({ message: message || PASSWORD_RESET_REQUEST_RESPONSE });  
});
```

```
export const resetPassword = asyncHandler(async (req, res) => {  
  const { token, newPassword } = req.validated.body;  
  const result = await resetPasswordWithToken(token, newPassword);  
  res.json(result);  
});
```

```
export const me = asyncHandler(async (req, res) => {  
  res.json({ user: req.user });  
});
```

```
export const invite = asyncHandler(async (req, res) => {  
  const user = await inviteTeamMember({  
    organizationId: req.user.organization_id,
```

```
    ...req.validated.body,
  });

  res.status(201).json({ user });
});

export const team = asyncHandler(async (req, res) => {
  const teamMembers = await listTeamMembers(req.user.organization_id);
  res.json({ team: teamMembers });
});

export const reactivate = asyncHandler(async (req, res) => {
  const user = await reactivateTeamMember({
    organizationId: req.user.organization_id,
    targetUserId: req.validated.params.id,
  });

  res.json({ user });
});

export const deactivate = asyncHandler(async (req, res) => {
  const user = await deactivateTeamMember({
    organizationId: req.user.organization_id,
    actorUserId: req.user.id,
    targetUserId: req.validated.params.id,
```

```
});
```

```
res.json({ user });
```

```
});
```

```
export const removeOrganization = asyncHandler(async (req, res) => {
```

```
  await deleteOrganization(req.user.organization_id);
```

```
  res.json({ message: 'Workspace deleted successfully and all data purged' });
```

```
});
```



auth.js

```
import jwt from 'jsonwebtoken';

import { env } from '../config/env.js';

import { findUserById } from '../services/authService.js';

export async function authenticate(req, res, next) {
  try {
    const authHeader = req.headers.authorization;

    if (!authHeader || !authHeader.startsWith('Bearer ')) {
      return res.status(401).json({ error: 'No token provided' });
    }

    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, env.jwtSecret);
    const user = await findUserById(decoded.userId);

    if (!user) {
      return res.status(401).json({ error: 'User not found' });
    }

    if (!user.is_active) {
      return res.status(403).json({ error: 'Account deactivated' });
    }
  }
}
```

```
req.user = user;

next();

} catch (error) {

  if (error.name === 'TokenExpiredError') {

    return res.status(401).json({ error: 'Token expired' });

  }

  return res.status(401).json({ error: 'Invalid token' });

}

export function requireRole(...roles) {

  return (req, res, next) => {

    if (!roles.includes(req.user.role)) {

      return res.status(403).json({

        error: `Access denied. Required roles: ${roles.join(', ')}`,

      });

    }

    next();

  };

}
```


errorHandler.js

```
import { env } from '../config/env.js';

import { ApiError } from '../utils/apiError.js';

export function notFoundHandler(req, res, next) {
  next(new ApiError(`Route ${req.originalUrl} not found`, 404));
}

export function errorHandler(err, req, res, next) {
  const statusCode = err.statusCode || err.status || 500;
  const isOperational = err instanceof ApiError;

  if (statusCode >= 500 || !isOperational) {
    console.error(err);
  }

  const response = {
    error: statusCode >= 500 && !isOperational
      ? 'Internal server error'
      : err.message || 'Internal server error',
  };

  if (err.details) {
    response.errors = err.details;
  }
}
```

```
if (env.nodeEnv === 'development' && err.stack) {  
  response.stack = err.stack;  
}  
  
res.status(statusCode).json(response);  
}
```



rateLimiters.js

```
import rateLimit from 'express-rate-limit';

import { env } from '../config/env.js';

import { hashToken } from '../utils/crypto.js';

function buildJsonRateLimiter(options) {

  return rateLimit({

    standardHeaders: true,

    legacyHeaders: false,

    handler: (req, res) => {

      res.status(429).json({

        error: options.message?.error || 'Too many requests. Please try again later.',

      });

    },

    ...options,

  });

}

function buildEmailKey(req) {

  const email = typeof req.body?.email === 'string'

    ? req.body.email.trim().toLowerCase()

    : req.ip;

  return `forgot-password:${hashToken(email)}`;

}
```

```
export const apiLimiter = buildJsonRateLimiter({  
  windowMs: 15 * 60 * 1000,  
  max: 200,  
  message: { error: 'Too many requests. Please slow down.' },  
});
```

```
export const aiLimiter = buildJsonRateLimiter({  
  windowMs: 60 * 1000,  
  max: 20,  
  message: { error: 'Too many AI requests, please slow down.' },  
});
```

```
export const forgotPasswordIpLimiter = buildJsonRateLimiter({  
  windowMs: 60 * 60 * 1000,  
  max: env.passwordResetRequestMaxPerHour,  
  message: { error: 'Too many password reset requests. Please try again later.' },  
});
```

```
export const forgotPasswordEmailLimiter = buildJsonRateLimiter({  
  windowMs: 60 * 60 * 1000,  
  max: env.passwordResetRequestMaxPerHour,  
  keyGenerator: buildEmailKey,  
  message: { error: 'Too many password reset requests. Please try again later.' },  
});
```

```
export const resetPasswordIpLimiter = buildJsonRateLimiter({  
  windowMs: 60 * 60 * 1000,  
  max: 10,  
  message: { error: 'Too many password reset attempts. Please try again later.' },  
});
```



validateRequest.js

```
import { matchedData, validationResult } from 'express-validator';

import { ApiError } from '../utils/apiError.js';

export function validateRequest(req, res, next) {

  const errors = validationResult(req);

  if (!errors.isEmpty()) {
    return next(new ApiError('Validation failed', 400, errors.array()));
  }

  req.validated = {
    body: matchedData(req, { locations: ['body'], includeOptionals: true }),
    params: matchedData(req, { locations: ['params'], includeOptionals: true }),
    query: matchedData(req, { locations: ['query'], includeOptionals: true }),
  };

  next();
}
```

ai.js

```
import express from 'express';

import { supabase } from '../config/supabase.js';

import { authenticate } from '../middleware/auth.js';

import aiService from '../services/aiService.js';

const router = express.Router();

// POST /api/ai/chat
router.post('/chat', authenticate, async (req, res, next) => {
  try {
    const { messages } = req.body;

    if (!messages || !Array.isArray(messages)) {
      return res.status(400).json({ error: 'Messages array required' });
    }

    // Build context from recent dashboard data
    const orgId = req.user.organization_id;

    const { data: products } = await supabase
      .from('products')
      .select('name, category, transactions(type, quantity)')
      .eq('organization_id', orgId)
      .eq('is_deleted', false)
      .limit(50);
```



```

const context = products ? `Organization has ${products.length} products.` : "";

const response = await aiService.chat(messages, context);
res.json({ response });
} catch (err) {
  if (err.message.includes('not configured')) {
    return res.status(503).json({ error: err.message });
  }
  next(err);
}
});

// POST /api/ai/predict
router.post('/predict', authenticate, async (req, res, next) => {
  try {
    const { product_id } = req.body;
    const orgId = req.user.organization_id;

    const { data: product } = await supabase
      .from('products')
      .select('name')
      .eq('id', product_id)
      .eq('organization_id', orgId)
      .single();
  }
});

```

```
if (!product) return res.status(404).json({ error: `Product with ID ${product_id}
not found or doesn't belong to your organization.` });
```

```
// Get last 90 days of sales
```

```
const since = new Date();
```

```
since.setDate(since.getDate() - 90);
```

```
const { data: transactions } = await supabase
```

```
.from('transactions')
```

```
.select('quantity, created_at')
```

```
.eq('product_id', product_id)
```

```
.eq('organization_id', orgId)
```

```
.eq('type', 'OUT')
```

```
.gte('created_at', since.toISOString())
```

```
.order('created_at');
```

```
// Group by day
```

```
const dailyMap = {};
```

```
transactions?.forEach(tx => {
```

```
  const day = tx.created_at.split('T')[0];
```

```
  dailyMap[day] = (dailyMap[day] || 0) + tx.quantity;
```

```
});
```

```
const prediction = await aiService.predictDemand(
```

```
  Object.entries(dailyMap).map(([date, qty]) => ({ date, qty })),
```

```
    product.name
  );

  res.json(prediction);
} catch (err) {
  if (err.message.includes('not configured')) return res.status(503).json({ error:
err.message });
  next(err);
}
});

// POST /api/ai/reorder-suggestions
router.post('/reorder-suggestions', authenticate, async (req, res, next) => {
  try {
    const orgId = req.user.organization_id;

    const { data: products } = await supabase
      .from('products')
      .select(`
        id, name, sku, min_stock, price, supplier_name,
        supplier:suppliers(name, lead_time_days),
        transactions(type, quantity, created_at)
      `)
      .eq('organization_id', orgId)
      .eq('is_deleted', false);
```

```
if (!products) return res.status(404).json({ error: 'No products found' });
```

```
const since = new Date();
```

```
since.setDate(since.getDate() - 30);
```

```
const productsWithData = products.map(p => {
```

```
  const stock = (p.transactions || []).reduce((acc, tx) =>
```

```
    tx.type === 'IN' ? acc + tx.quantity : acc - tx.quantity, 0);
```

```
  const recentSales = (p.transactions || []).
```

```
    .filter(tx => tx.type === 'OUT' && new Date(tx.created_at) > since)
```

```
    .reduce((s, tx) => s + tx.quantity, 0);
```

```
  const avgDailySales = recentSales / 30;
```

```
  return {
```

```
    id: p.id, name: p.name, sku: p.sku,
```

```
    current_stock: stock, min_stock: p.min_stock,
```

```
    avg_daily_sales: Math.round(avgDailySales * 100) / 100,
```

```
    lead_time_days: p.supplier?.lead_time_days || 7,
```

```
    supplier_name: p.supplier?.name || p.supplier_name || 'Unknown'
```

```
  };
```

```
}).filter(p => p.current_stock <= p.min_stock * 1.5);
```

```

    if (productsWithData.length === 0) {
      return res.json({ recommendations: [], summary: 'All products are well-stocked.'
    });
  }

  const suggestions = await aiService.suggestReorder(productsWithData);
  res.json(suggestions);
} catch (err) {
  if (err.message.includes('not configured')) return res.status(503).json({ error:
err.message });
  next(err);
}
});

// POST /api/ai/dead-stock
router.post('/dead-stock', authenticate, async (req, res, next) => {
  try {
    const orgId = req.user.organization_id;
    const { days = 60 } = req.body;
    const since = new Date();
    since.setDate(since.getDate() - parseInt(days));

    const { data: products } = await supabase
      .from('products')
      .select('id, name, sku, category, price, transactions(type, quantity, created_at)')

```

```

    .eq('organization_id', orgId)
    .eq('is_deleted', false);

const deadStock = (products || []).filter(p => {
  const stock = (p.transactions || []).reduce((acc, tx) =>
    tx.type === 'IN' ? acc + tx.quantity : acc - tx.quantity, 0);
  if (stock <= 0) return false;
  return !(p.transactions || []).some(tx =>
    tx.type === 'OUT' && new Date(tx.created_at) > since);
}).map(p => {
  const stock = (p.transactions || []).reduce((acc, tx) =>
    tx.type === 'IN' ? acc + tx.quantity : acc - tx.quantity, 0);
  return { id: p.id, name: p.name, sku: p.sku, category: p.category, current_stock:
stock, value: Math.round(stock * p.price * 100) / 100 };
});

if (deadStock.length === 0) {
  return res.json({ recommendations: [], summary: 'No dead stock detected. All
products have recent sales activity.', total_dead_stock_value: 0 });
}

const analysis = await aiService.analyzeDeadStock(deadStock);
res.json(analysis);
} catch (err) {
  if (err.message.includes('not configured')) return res.status(503).json({ error:
err.message });
}

```

```
    next(err);
  }
});

// POST /api/ai/column-map
router.post('/column-map', authenticate, async (req, res, next) => {
  try {
    const { headers, sample_data } = req.body;
    if (!headers || !Array.isArray(headers)) {
      return res.status(400).json({ error: 'Headers array required' });
    }

    const mapping = await aiService.mapCSVColumns(headers, sample_data || []);
    res.json({ mapping });
  } catch (err) {
    if (err.message.includes('not configured')) return res.status(503).json({ error: err.message });
    next(err);
  }
});

// POST /api/ai/generate-sku
router.post('/generate-sku', authenticate, async (req, res, next) => {
  try {
    const { name, category } = req.body;
```



```
if (!name) return res.status(400).json({ error: 'Product name required' });

const sku = await aiService.generateSKU(name, category);

res.json({ sku });

} catch (err) {

  if (err.message.includes('not configured')) return res.status(503).json({ error:
err.message });

  next(err);

}

});

// POST /api/ai/dashboard-insights

router.post('/dashboard-insights', authenticate, async (req, res, next) => {

  try {

    const { dashboard_data } = req.body;

    if (!dashboard_data) return res.status(400).json({ error: 'Dashboard data
required' });

    const insights = await aiService.generateDashboardInsights(dashboard_data);

    res.json(insights);

  } catch (err) {

    if (err.message.includes('not configured')) return res.status(503).json({ error:
err.message });

    next(err);

  }

});

export default router;
```

alerts.js

```
import express from 'express';  
import { supabase } from '../config/supabase.js';  
import { authenticate } from '../middleware/auth.js';
```

```
const router = express.Router();
```

```
// GET /api/alerts
```

```
router.get('/', authenticate, async (req, res, next) => {  
  try {  
    const { resolved = 'false', limit = 50 } = req.query;  
    const { data, error } = await supabase  
      .from('alerts')  
      .select('*', product:products(id, name, sku))  
      .eq('organization_id', req.user.organization_id)  
      .eq('resolved', resolved === 'true')  
      .order('created_at', { ascending: false })  
      .limit(parseInt(limit));  
  
    if (error) throw error;  
    res.json({ alerts: data });  
  } catch (err) { next(err); }  
});
```

```
// PATCH /api/alerts/:id/resolve
```

```
router.patch('/:id/resolve', authenticate, async (req, res, next) => {  
  try {  
    const { error } = await supabase  
      .from('alerts')  
      .update({ resolved: true, resolved_at: new Date().toISOString() })  
      .eq('id', req.params.id)  
      .eq('organization_id', req.user.organization_id);  
  
    if (error) throw error;  
    res.json({ message: 'Alert resolved' });  
  } catch (err) { next(err); }  
});  
  
export default router;
```

auth.js

```
import express from 'express';

import { body, param } from 'express-validator';

import * as authController from '../controllers/authController.js';

import { authenticate, requireRole } from '../middleware/auth.js';

import {
  forgotPasswordEmailLimiter,
  forgotPasswordIpLimiter,
  resetPasswordIpLimiter,
} from '../middleware/rateLimiters.js';

import { validateRequest } from '../middleware/validateRequest.js';

const router = express.Router();

const normalizeEmail = (value) => String(value).trim().toLowerCase();

const passwordRules = (fieldName, label = 'Password') =>
  body(fieldName)
    .isString().withMessage(`${label} is required`)
    .isLength({ min: 8 }).withMessage(`${label} must be at least 8 characters long`)
    .matches(/^(?=.*[a-z])(?=.*[A-Z])(?=.*\d).+$/)
    .withMessage(`${label} must include uppercase, lowercase, and a number`);

router.post(
  '/register',
```

```

[
    body('name').trim().notEmpty().withMessage('Name is required'),
    body('email').isEmail().withMessage('Valid email
required').customSanitizer(normalizeEmail),
    passwordRules('password'),
    body('organizationName').trim().notEmpty().withMessage('Organization name is
required'),
],
validateRequest,
authController.register
);

router.post(
    '/login',
    [
        body('email').isEmail().withMessage('Valid email
required').customSanitizer(normalizeEmail),
        body('password').isString().notEmpty().withMessage('Password is required'),
    ],
    validateRequest,
    authController.login
);

router.post(
    '/forgot-password',
    forgotPasswordIpLimiter,

```

```
forgotPasswordEmailLimiter,  
[  
  body('email').isEmail().withMessage('Valid email  
required').customSanitizer(normalizeEmail),  
],  
validateRequest,  
authController.forgotPassword  
);
```

```
router.post(  
  '/reset-password',  
  resetPasswordIpLimiter,  
  [  
    body('token')  
      .trim()  
      .isLength({ min: 64, max: 64 }).withMessage('Reset token is invalid')  
      .matches(/^[a-f0-9]+$/i).withMessage('Reset token is invalid'),  
    passwordRules('newPassword', 'New password'),  
    body('confirmPassword')  
      .custom((value, { req }) => value === req.body.newPassword)  
      .withMessage('Passwords do not match'),  
  ],  
  validateRequest,  
  authController.resetPassword  
);
```

```
router.get('/me', authenticate, authController.me);
```

```
router.post(
  '/invite',
  authenticate,
  requireRole('admin'),
  [
    body('name').trim().notEmpty().withMessage('Name is required'),
    body('email').isEmail().withMessage('Valid email
required').customSanitizer(normalizeEmail),
    body('role').isIn(['manager', 'staff']).withMessage('Role must be manager or
staff'),
    passwordRules('password', 'Temporary password'),
  ],
  validateRequest,
  authController.invite
);
```

```
router.get('/team', authenticate, requireRole('admin', 'manager'),
authController.team);
```

```
router.patch(
  '/reactivate/:id',
  authenticate,
  requireRole('admin'),
```



```
[param('id').isUUID().withMessage('Valid user ID required')],  
validateRequest,  
authController.reactivate  
];
```

```
router.patch(  
  '/deactivate/:id',  
  authenticate,  
  requireRole('admin'),  
  [param('id').isUUID().withMessage('Valid user ID required')],  
  validateRequest,  
  authController.deactivate  
);
```

```
router.delete(  
  '/organization',  
  authenticate,  
  requireRole('admin'),  
  authController.removeOrganization  
);
```

```
export default router;
```

importExport.js

```
import express from 'express';

import multer from 'multer';

import csv from 'csv-parser';

import { Readable } from 'stream';

import * as XLSX from 'xlsx';

import { v4 as uuidv4 } from 'uuid';

import { supabase } from '../config/supabase.js';

import { authenticate, requireRole } from '../middleware/auth.js';

import { generateSKU } from '../utils/skuGenerator.js';

import aiService from '../services/aiService.js';

const router = express.Router();

const upload = multer({ storage: multer.memoryStorage(), limits: { fileSize: 10 * 1024 * 1024 } });

function normalizeHeader(value = '') {

  return String(value)

    .replace(/^\\uFEFF/, '')

    .trim()

    .toLowerCase()

    .replace(/^[^a-z0-9]+/g, '_')

    .replace(/^[^_+|_+$/g, '');

}
```

```
function getTrimmedValue(row, columnName) {  
  if (!columnName) return null;  
  const value = row[columnName];  
  if (value === undefined || value === null) return null;  
  return String(value).trim();  
}
```

```
function parseDecimalValue(value, fallback = 0) {  
  if (value === null || value === undefined || value === '') return fallback;  
  const parsed = parseFloat(String(value).replace(/,/g, "").trim());  
  return Number.isNaN(parsed) ? null : parsed;  
}
```

```
function parseIntegerValue(value, fallback = 0) {  
  if (value === null || value === undefined || value === '') return fallback;  
  const parsed = parseInt(String(value).replace(/,/g, "").trim(), 10);  
  return Number.isNaN(parsed) ? null : parsed;  
}
```

```
function normalizeSupplierKey(value = '') {  
  return String(value).trim().toLowerCase();  
}
```

```
function normalizeSku(value = '') {  
  return String(value)
```

```
.trim()
.toUpperCase()
.replace(/^[A-Z0-9-]/g, '-')
.replace(/-+/g, '-')
.replace(/^-|-$/g, '')
.substring(0, 100);
}
```

```
function buildUniqueSku(baseSku, existingSkus, seenSkus) {
  const normalizedBase = normalizeSku(baseSku) || 'GEN-PROD';
  let candidate = normalizedBase;
  let suffix = 2;

  while (existingSkus.has(candidate) || seenSkus.has(candidate)) {
    const suffixText = `-${suffix}`;
    candidate = `${normalizedBase.substring(0, Math.max(1, 100 -
suffixText.length))}${suffixText}`;
    suffix += 1;
  }
}
```

```
seenSkus.add(candidate);
```

```
return {
  sku: candidate,
  adjusted: candidate !== normalizedBase,
}
```

```
};  
}
```

```
function chunkArray(items = [], size = 50) {  
  if (!Array.isArray(items) || items.length === 0) return [];
```

```
  const chunks = [];  
  for (let index = 0; index < items.length; index += size) {  
    chunks.push(items.slice(index, index + size));  
  }
```

```
  return chunks;  
}
```

```
function sanitizeImportError(error, fallback = 'Import failed for this row') {
```

```
  const rawMessage =  
    error?.message ||  
    error?.details ||  
    error?.hint ||  
    error?.error_description ||  
    fallback;
```

```
  return String(rawMessage)  
    .replace(/\s+/g, ' ')
```

```
.replace(/duplicate key value violates unique constraint.*$/i, 'SKU already exists  
in your inventory')
```

```
.trim();
```

```
}
```

```
function inferProductMapping(headers = []) {
```

```
  const headerLookup = {};
```

```
  headers.forEach((header) => {
```

```
    headerLookup[normalizeHeader(header)] = header;
```

```
  });
```

```
  const candidates = {
```

```
    name: ['name', 'product_name', 'product', 'item_name', 'item', 'title'],
```

```
    sku: ['sku', 'product_code', 'item_code', 'code', 'product_sku'],
```

```
    price: ['price', 'unit_price', 'selling_price', 'sale_price', 'mrp', 'cost', 'cost_price'],
```

```
    category: ['category', 'product_category', 'group', 'type'],
```

```
    supplier: ['supplier', 'supplier_name', 'vendor', 'vendor_name', 'brand'],
```

```
    min_stock: ['min_stock', 'minimum_stock', 'reorder_level', 'reorder_point',  
'low_stock_threshold'],
```

```
    initial_stock: ['initial_stock', 'opening_stock', 'stock', 'current_stock', 'quantity',  
'qty', 'opening_qty'],
```

```
    description: ['description', 'product_description', 'details', 'notes'],
```

```
    unit: ['unit', 'uom', 'unit_of_measure', 'measure'],
```

```
  };
```

```
  return Object.entries(candidates).reduce((acc, [field, possibleHeaders]) => {
```

```

    const matchedHeader = possibleHeaders.map((key) =>
headerLookup[key]).find(Boolean);

    if (matchedHeader) acc[field] = matchedHeader;

    return acc;

  }, {});
}

// POST /api/import

router.post('/import', authenticate, requireRole('admin', 'manager'),
upload.single('file'), async (req, res, next) => {
  try {
    if (!req.file) return res.status(400).json({ error: 'No file uploaded' });

    const { mapping: mappingStr, dry_run } = req.body;
    let mapping = {};
    try { mapping = JSON.parse(mappingStr || '{}'); } catch {}

    const ext = req.file.originalname.split('.').pop().toLowerCase();
    let rows = [];

    if (ext === 'csv') {
      const results = [];
      await new Promise((resolve, reject) => {
        const stream = Readable.from(req.file.buffer.toString());
        stream

```



```

.pipe(csv())
.on('data', (data) => results.push(data))
.on('end', resolve)
.on('error', reject);
});

rows = results;
} else if (['xlsx', 'xls'].includes(ext)) {
  const workbook = XLSX.read(req.file.buffer, { type: 'buffer' });
  const worksheet = workbook.Sheets[workbook.SheetNames[0]];
  rows = XLSX.utils.sheet_to_json(worksheet);
} else {
  return res.status(400).json({ error: 'Only CSV and Excel files supported' });
}

if (rows.length === 0) return res.status(400).json({ error: 'File is empty' });

const headers = Object.keys(rows[0]);
if (Object.keys(mapping).length === 0) {
  const delay = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
  for (let attempt = 1; attempt <= 4; attempt += 1) {
    try {
      mapping = await aiService.mapCSVColumns(headers, rows.slice(0,
Math.min(50, rows.length)));
      break;
    } catch (err) {

```

```

    if (attempt === 4) {
      mapping = inferProductMapping(headers);
      if (!mapping.name && headers.length > 0) {
        mapping.name = headers[0];
      }
    } else {
      await delay(Math.pow(2, attempt) * 500);
    }
  }
}

const orgId = req.user.organization_id;
const userId = req.user.id;
const { data: existingProducts, error: existingProductsError } = await supabase
  .from('products')
  .select('sku')
  .eq('organization_id', orgId)
  .eq('is_deleted', false);

if (existingProductsError) throw existingProductsError;

const existingSkuSet = new Set(
  (existingProducts || [])
    .map((product) => normalizeSku(product.sku))

```

```

        .filter(Boolean)
    );
    const seenSkuSet = new Set();
    const validRows = [];
    const errorRows = [];

    rows.forEach((row, index) => {
        const rowErrors = [];
        const originalSku = getTrimmedValue(row, mapping.sku);
        const name = getTrimmedValue(row, mapping.name);
        const rawPrice = getTrimmedValue(row, mapping.price);
        const rawCategory = getTrimmedValue(row, mapping.category);
        const rawSupplierName = getTrimmedValue(row, mapping.supplier);
        const rawMinStock = getTrimmedValue(row, mapping.min_stock);
        const rawInitialStock = getTrimmedValue(row, mapping.initial_stock);
        const price = parseDecimalValue(rawPrice, 0);
        const minStock = mapping.min_stock ? parseIntegerValue(rawMinStock, 5) : 5;
        const initialStock = mapping.initial_stock ? parseIntegerValue(rawInitialStock, 0)
: 0;

        if (!name) rowErrors.push('Name is required');
        if (price === null) rowErrors.push('Price must be a number');
        if (minStock === null) rowErrors.push('Min stock must be a whole number');
        if (initialStock === null) rowErrors.push('Initial stock must be a whole number');
    });

```

```
if (minStock !== null && minStock < 0) rowErrors.push('Min stock cannot be negative');
```

```
if (initialStock !== null && initialStock < 0) rowErrors.push('Initial stock cannot be negative');
```

```
const category = rawCategory || 'Uncategorized';
```

```
const skuSeed = originalSku || generateSKU(name || 'PRODUCT', category);
```

```
const { sku: finalSku, adjusted: skuAdjusted } = buildUniqueSku(skuSeed, existingSkuSet, seenSkuSet);
```

```
const normalizedRow = {  
  row_number: index + 2,  
  name,  
  original_sku: originalSku || null,  
  sku: finalSku,  
  sku_adjusted: skuAdjusted,  
  price: price ?? 0,  
  category,  
  supplier_name: rawSupplierName || null,  
  min_stock: minStock ?? 5,  
  description: getTrimmedValue(row, mapping.description),  
  unit: getTrimmedValue(row, mapping.unit) || 'units',  
  initial_stock: initialStock ?? 0,  
};
```

```
if (rowErrors.length > 0) {
```

```

errorRows.push({
  row: index + 2,
  product_name: name || null,
  original_sku: originalSku || null,
  final_sku: finalSku,
  error_type: 'validation',
  errors: rowErrors,
  message: rowErrors.join(', '),
  data: row,
});
} else {
  validRows.push(normalizedRow);
}
});

if (dry_run === 'true') {
  return res.json({
    preview: validRows.slice(0, 10),
    valid_count: validRows.length,
    error_count: errorRows.length,
    errors: errorRows,
    headers,
    mapping,
    import_summary: `${validRows.length} valid row(s), ${errorRows.length}
row(s) flagged during preview.`
  });
}

```

```

});
}

const supplierMap = {};
const unmappedSuppliers = new Set();
const supplierWarningSet = new Set();
const uniqueSupplierNames = [...new Set(
  validRows
    .map((row) => row.supplier_name)
    .filter(Boolean)
    .map((name) => name.trim())
    .filter(Boolean)
)];

if (uniqueSupplierNames.length > 0) {
  const { data: existingSuppliers, error: suppliersError } = await supabase
    .from('suppliers')
    .select('id, name')
    .eq('organization_id', orgId)
    .eq('is_deleted', false);

  if (suppliersError) throw suppliersError;

  (existingSuppliers || []).forEach((supplier) => {
    supplierMap[normalizeSupplierKey(supplier.name)] = supplier.id;
  });
}

```

```

});

uniqueSupplierNames.forEach((name) => {
  if (!supplierMap[normalizeSupplierKey(name)]) {
    unmappedSuppliers.add(name);
  }
});
}

const preparedRows = validRows.map((row) => {
  const supplierKey = row.supplier_name ?
normalizeSupplierKey(row.supplier_name) : null;

  const supplierId = supplierKey ? supplierMap[supplierKey] || null : null;

  if (row.supplier_name && !supplierId) {
    supplierWarningSet.add(`Row ${row.row_number}: supplier
"${row.supplier_name}" not found, imported without supplier link`);
  }

  return {
    ...row,
    product_id: uuidv4(),
    supplier_id: supplierId,
  };
});

```



```
const imported = [];  
  
const importErrors = [];  
  
const productRowById = new Map(preparedRows.map((row) => [row.product_id,  
row]));
```

```
const importSingleRow = async (row) => {  
  const productPayload = {  
    id: row.product_id,  
    name: row.name,  
    sku: row.sku,  
    price: row.price,  
    category: row.category,  
    supplier_id: row.supplier_id,  
    min_stock: row.min_stock,  
    description: row.description,  
    unit: row.unit,  
    organization_id: orgId,  
    created_by: userId,  
    is_deleted: false,  
  };
```

```
const { data: product, error: productError } = await supabase  
  .from('products')  
  .insert(productPayload)
```

```
.select()
```

```
.single();
```

```
if (productError) throw { stage: 'product', cause: productError };
```

```
if (row.initial_stock > 0) {
```

```
  const { error: transactionError } = await supabase
```

```
    .from('transactions')
```

```
    .insert({
```

```
      id: uuidv4(),
```

```
      product_id: row.product_id,
```

```
      organization_id: orgId,
```

```
      type: 'IN',
```

```
      quantity: row.initial_stock,
```

```
      reason: 'CSV Import',
```

```
      created_by: userId,
```

```
    });
```

```
if (transactionError) {
```

```
  await supabase
```

```
    .from('products')
```

```
    .delete()
```

```
    .eq('id', row.product_id)
```

```
    .eq('organization_id', orgId);
```

```
    throw { stage: 'transaction', cause: transactionError };  
  }  
}
```

```
imported.push({  
  ...product,  
  current_stock: row.initial_stock,  
  imported_row: row.row_number,  
});  
};
```

```
for (const batch of chunkArray(preparedRows, 50)) {  
  const productPayloads = batch.map((row) => ({  
    id: row.product_id,  
    name: row.name,  
    sku: row.sku,  
    price: row.price,  
    category: row.category,  
    supplier_id: row.supplier_id,  
    min_stock: row.min_stock,  
    description: row.description,  
    unit: row.unit,  
    organization_id: orgId,  
    created_by: userId,  
    is_deleted: false,
```

```
}});
```

```
const { data: insertedProducts, error: batchProductError } = await supabase  
  .from('products')  
  .insert(productPayloads)  
  .select();
```

```
if (batchProductError) {  
  for (const row of batch) {  
    try {  
      await importSingleRow(row);  
    } catch (err) {  
      importErrors.push({  
        row: row.row_number,  
        product_name: row.name,  
        original_sku: row.original_sku || null,  
        final_sku: row.sku,  
        error_type: err?.stage === 'transaction' ? 'transaction' : 'database',  
        message: sanitizeImportError(err?.cause || err),  
      });  
    }  
  }  
  continue;  
}
```

```

const rowsNeedingTransactions = batch.filter((row) => row.initial_stock > 0);
if (rowsNeedingTransactions.length > 0) {
  const transactionPayloads = rowsNeedingTransactions.map((row) => ({
    id: uuidv4(),
    product_id: row.product_id,
    organization_id: orgId,
    type: 'IN',
    quantity: row.initial_stock,
    reason: 'CSV Import',
    created_by: userId,
  }));

  const { error: transactionBatchError } = await supabase
    .from('transactions')
    .insert(transactionPayloads);

  if (transactionBatchError) {
    await supabase
      .from('products')
      .delete()
      .in('id', batch.map((row) => row.product_id))
      .eq('organization_id', orgId);

    batch.forEach((row) => {
      importErrors.push({

```

```

    row: row.row_number,
    product_name: row.name,
    original_sku: row.original_sku || null,
    final_sku: row.sku,
    error_type: 'transaction',
    message: `Initial stock transaction failed:
    ${sanitizeImportError(transactionBatchError)}`,
  });
});
continue;
}
}

(insertedProducts || []).forEach((product) => {
  const sourceRow = productRowById.get(product.id);
  imported.push({
    ...product,
    current_stock: sourceRow?.initial_stock ?? 0,
    imported_row: sourceRow?.row_number,
  });
});
}

const allErrors = [...errorRows, ...importErrors];
const importSummary =

```

```

imported.length === 0 && allErrors.length > 0
? `Import failed. ${allErrors.length} row(s) could not be imported.`
: allErrors.length > 0
? `${imported.length} row(s) imported, ${allErrors.length} row(s) skipped.`
: `Imported ${imported.length} row(s) successfully.`;

res.json({
  imported_count: imported.length,
  error_count: allErrors.length,
  errors: allErrors,
  products: imported.slice(0, 20),
  supplier_warnings: [...supplierWarningSet],
  unmapped_suppliers: [...unmappedSuppliers],
  import_summary: importSummary,
});
} catch (err) {
  next(err);
}
});

// GET /api/export/products
router.get('/export/products', authenticate, async (req, res, next) => {
  try {
    const { format = 'csv', category, stock_status } = req.query;
    const orgId = req.user.organization_id;

```



```

const { data: products, error } = await supabase
  .from('products')
  .select('*', supplier:suppliers(name), transactions(type, quantity)')
  .eq('organization_id', orgId)
  .eq('is_deleted', false)
  .order('name');

if (error) throw error;

const rows = products.map(p => {
  const stock = (p.transactions || []).reduce((acc, tx) =>
    tx.type === 'IN' ? acc + tx.quantity : acc - tx.quantity, 0);
  return {
    Name: p.name,
    SKU: p.sku,
    Category: p.category,
    'Unit Price': p.price,
    'Current Stock': stock,
    'Min Stock': p.min_stock,
    'Stock Value': Math.round(stock * p.price * 100) / 100,
    Supplier: p.supplier?.name || p.supplier_name || '',
    Unit: p.unit || 'units',
    Description: p.description || '',
    Status: stock <= 0 ? 'Out of Stock' : stock <= p.min_stock ? 'Low Stock' : 'In Stock',
    'Created At': new Date(p.created_at).toLocaleDateString()
  }
});

```

```

    };
  });

  if (format === 'xlsx') {
    const wb = XLSX.utils.book_new();
    const ws = XLSX.utils.json_to_sheet(rows);
    XLSX.utils.book_append_sheet(wb, ws, 'Products');
    const buffer = XLSX.write(wb, { type: 'buffer', bookType: 'xlsx' });
    res.setHeader('Content-Disposition', 'attachment; filename=products.xlsx');
    res.setHeader('Content-Type', 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet');
    return res.send(buffer);
  }

  // CSV
  const headers = Object.keys(rows[0] || {});
  const csvContent = [
    headers.join(','),
    ...rows.map(row => headers.map(h => `${String(row[h] || '')}.replace(/"/g,
    '""')}`).join(','))
  ].join('\n');

  res.setHeader('Content-Disposition', 'attachment; filename=products.csv');
  res.setHeader('Content-Type', 'text/csv');
  res.send(csvContent);

```

```

    } catch (err) {
      next(err);
    }
  });

// GET /api/export/transactions
router.get('/export/transactions', authenticate, async (req, res, next) => {
  try {
    const { format = 'csv', start_date, end_date } = req.query;
    const orgId = req.user.organization_id;

    let query = supabase
      .from('transactions')
      .select('*', product:products(name, sku), user:users(name))
      .eq('organization_id', orgId)
      .order('created_at', { ascending: false });

    if (start_date) query = query.gte('created_at', start_date);
    if (end_date) query = query.lte('created_at', end_date + 'T23:59:59');

    const { data, error } = await query;
    if (error) throw error;

    const rows = data.map(tx => ({
      Date: new Date(tx.created_at).toLocaleDateString(),

```

```

    Type: tx.type,
    Product: tx.product?.name || "",
    SKU: tx.product?.sku || "",
    Quantity: tx.quantity,
    Reason: tx.reason || "",
    Reference: tx.reference_no || "",
    'Performed By': tx.user?.name || "",
    Notes: tx.notes || ""
  }));

  if (format === 'xlsx') {
    const wb = XLSX.utils.book_new();
    XLSX.utils.book_append_sheet(wb, XLSX.utils.json_to_sheet(rows),
    'Transactions');

    const buffer = XLSX.write(wb, { type: 'buffer', bookType: 'xlsx' });
    res.setHeader('Content-Disposition', 'attachment; filename=transactions.xlsx');
    res.setHeader('Content-Type', 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet');

    return res.send(buffer);
  }

  const headers = Object.keys(rows[0] || {});
  const csvContent = [
    headers.join(','),
    ...rows.map(row => headers.map(h => `${String(row[h] || '')}.replace(/"/g,
    '""')}`)).join(',')
  ]

```

```

    ].join('\n');

    res.setHeader('Content-Disposition', 'attachment; filename=transactions.csv');
    res.setHeader('Content-Type', 'text/csv');
    res.send(csvContent);
  } catch (err) {
    next(err);
  }
});

// POST /api/import/suppliers
router.post('/import/suppliers', authenticate, requireRole('admin', 'manager'),
upload.single('file'), async (req, res, next) => {
  try {
    if (!req.file) return res.status(400).json({ error: 'No file uploaded' });

    const { mapping: mappingStr, dry_run } = req.body;
    let mapping = {};
    try { mapping = JSON.parse(mappingStr || '{}'); } catch {}

    const ext = req.file.originalname.split('.').pop().toLowerCase();
    let rows = [];

    if (ext === 'csv') {
      const results = [];

```

```

    await new Promise((resolve, reject) => {
      const stream = Readable.from(req.file.buffer.toString());
      stream.pipe(csv())
        .on('data', data => results.push(data))
        .on('end', resolve)
        .on('error', reject);
    });
    rows = results;
  } else if (['xlsx', 'xls'].includes(ext)) {
    const wb = XLSX.read(req.file.buffer, { type: 'buffer' });
    const ws = wb.Sheets[wb.SheetNames[0]];
    rows = XLSX.utils.sheet_to_json(ws);
  } else {
    return res.status(400).json({ error: 'Only CSV and Excel files supported' });
  }

  if (rows.length === 0) return res.status(400).json({ error: 'File is empty' });

  const headers = Object.keys(rows[0]);

  // If no mapping provided, use AI or heuristics to detect columns
  if (Object.keys(mapping).length === 0) {
    const delay = ms => new Promise(res => setTimeout(res, ms));
    for (let attempt = 1; attempt <= 3; attempt++) {
      try {

```

```

        mapping = await aiService.mapSupplierColumns(headers, rows.slice(0,
Math.min(20, rows.length)));

        break;
    } catch (err) {
        if (attempt === 3) {
            headers.forEach(h => {
                const norm = h.toLowerCase().trim().replace(/^[a-z0-9]/g, '
').replace(/\\s+/g, '_');

                if (['name', 'supplier', 'supplier_name', 'company',
'company_name'].includes(norm)) mapping.name = h;

                if (['email', 'contact_email'].includes(norm)) mapping.contact_email = h;

                if (['phone', 'mobile', 'phone_number', 'contact_number'].includes(norm))
mapping.phone = h;

                if (['address', 'full_address', 'location'].includes(norm)) mapping.address = h;

                if (['lead_time', 'lead_time_days', 'delivery_days'].includes(norm))
mapping.lead_time_days = h;

                if (['notes', 'remark', 'remarks', 'description'].includes(norm)) mapping.notes
= h;
            });

            if (!mapping.name && headers.length > 0) mapping.name = headers[0];
        } else {
            await delay(1000 * attempt);
        }
    }
}
}
}
}

```



```

const validRows = [];
const errorRows = [];

rows.forEach((row, idx) => {
  const errors = [];
  const name = mapping.name ? row[mapping.name] : null;

  if (!name || name.toString().trim() === '') errors.push('Name is required');

  const transformed = {
    name: name?.toString().trim(),
    contact_email: mapping.contact_email ?
row[mapping.contact_email]?.toString().trim() : null,
    phone: mapping.phone ? row[mapping.phone]?.toString().trim() : null,
    address: mapping.address ? row[mapping.address]?.toString().trim() : null,
    lead_time_days: mapping.lead_time_days ?
parseInt(row[mapping.lead_time_days]) || null : null,
    notes: mapping.notes ? row[mapping.notes]?.toString().trim() : null,
  };

  if (errors.length > 0) {
    errorRows.push({ row: idx + 2, data: row, errors });
  } else {
    validRows.push(transformed);
  }
}

```



```
});
```

```
if (dry_run === 'true') {  
  return res.json({  
    preview: validRows.slice(0, 10),  
    valid_count: validRows.length,  
    error_count: errorRows.length,  
    errors: errorRows,  
    headers,  
    mapping  
  });  
}
```

```
const orgId = req.user.organization_id;  
const imported = [];  
const importErrors = [];  
const chunkSize = 300;
```

```
for (let i = 0; i < validRows.length; i += chunkSize) {  
  const chunk = validRows.slice(i, i + chunkSize);  
  
  const promises = chunk.map(async (row) => {  
    try {  
      const supplierId = uuidv4();  
      const { data: supplier, error: sErr } = await supabase
```

```
.from('suppliers')
.insert({
  id: supplierId,
  ...row,
  organization_id: orgId,
  is_deleted: false
})
.select()
.single();

if (sErr) throw sErr;
return { success: true, supplier };
} catch (err) {
return { success: false, row: row.name, error: err.message };
}
});

const results = await Promise.all(promises);
results.forEach(res => {
  if (res.success) imported.push(res.supplier);
  else importErrors.push({ row: res.row, error: res.error });
});
}

res.json({
```

```

imported_count: imported.length,
error_count: importErrors.length + errorRows.length,
errors: [...errorRows, ...importErrors],
suppliers: imported.slice(0, 20)
});
} catch (err) {
  next(err);
}
});

// GET /api/export/suppliers
router.get('/export/suppliers', authenticate, async (req, res, next) => {
  try {
    const { format = 'csv', ids } = req.query;
    const orgId = req.user.organization_id;

    let query = supabase
      .from('suppliers')
      .select('*', products(id))
      .eq('organization_id', orgId)
      .order('name');

    if (ids) {
      const idArray = ids.split(',').filter(id => id.trim());
      if (idArray.length > 0) {

```

```
    query = query.in('id', idArray);  
  }  
}
```

```
const { data: suppliers, error } = await query;  
if (error) throw error;
```

```
const rows = suppliers.map(s => ({  
  Name: s.name,  
  'Contact Email': s.contact_email || "",  
  Phone: s.phone || "",  
  Address: s.address || "",  
  'Lead Time (Days)': s.lead_time_days || "",  
  'Products Supplied': s.products ? s.products.length : 0,  
  Notes: s.notes || "",  
  'Created At': new Date(s.created_at).toLocaleDateString()  
}));
```

```
if (rows.length === 0) {  
  rows.push({  
    Name: "", 'Contact Email': "", Phone: "", Address: "", 'Lead Time (Days)': "", 'Products  
Supplied': "", Notes: "", 'Created At': "  
  });  
}
```

```

if (format === 'xlsx') {
  const wb = XLSX.utils.book_new();
  const ws = XLSX.utils.json_to_sheet(rows);
  XLSX.utils.book_append_sheet(wb, ws, 'Suppliers');
  const buffer = XLSX.write(wb, { type: 'buffer', bookType: 'xlsx' });
  res.setHeader('Content-Disposition', 'attachment; filename=suppliers.xlsx');
  res.setHeader('Content-Type', 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet');
  return res.send(buffer);
}

const headers = Object.keys(rows[0] || {});
const csvContent = [
  headers.join(','),
  ...rows.map(row => headers.map(h => `${String(row[h] || '')}.replace(/"/g,
  '""')}`).join(','))
].join('\n');
res.setHeader('Content-Disposition', 'attachment; filename=suppliers.csv');
res.setHeader('Content-Type', 'text/csv');
res.send(csvContent);
} catch (err) {
  next(err);
}
});
export default router;

```

products.js

```
import express from 'express';

import { body, query, validationResult } from 'express-validator';

import { v4 as uuidv4 } from 'uuid';

import { supabase } from '../config/supabase.js';

import { authenticate, requireRole } from '../middleware/auth.js';

import { generateSKU } from '../utils/skuGenerator.js';

const router = express.Router();

// Helper: calculate current stock for a product
export async function calculateStock(productId, organizationId) {
  const { data, error } = await supabase
    .from('transactions')
    .select('type, quantity')
    .eq('product_id', productId)
    .eq('organization_id', organizationId);

  if (error) throw error;

  return data.reduce((acc, tx) => {
    return tx.type === 'IN' ? acc + tx.quantity : acc - tx.quantity;
  }, 0);
}
```

```
// GET /api/products
router.get('/', authenticate, async (req, res, next) => {
  try {
    const {
      page = 1,
      limit = 20,
      search = "",
      category = "",
      supplier_id = "",
      stock_status = "",
      sort_by = 'created_at',
      sort_order = 'desc'
    } = req.query;

    const offset = (parseInt(page) - 1) * parseInt(limit);
    const orgId = req.user.organization_id;

    let queryBuilder = supabase
      .from('products')
      .select(`
        *,
        supplier:suppliers(id, name),
        transactions(type, quantity, created_at)
      `, { count: 'exact' })
      .eq('organization_id', orgId)
```



```
.eq('is_deleted', false);

if (search) {
  queryBuilder =
queryBuilder.or(`name.ilike.%${search}%,sku.ilike.%${search}%`);
}
if (category) {
  queryBuilder = queryBuilder.eq('category', category);
}
if (supplier_id) {
  queryBuilder = queryBuilder.eq('supplier_id', supplier_id);
}

const validSortFields = ['name', 'created_at', 'price', 'category', 'sku'];
const sortField = validSortFields.includes(sort_by) ? sort_by : 'created_at';
queryBuilder = queryBuilder
  .order(sortField, { ascending: sort_order === 'asc' });

const { data: products, error } = await queryBuilder;
if (error) throw error;

// Calculate current stock and 30d sales for each product
const thirtyDaysAgo = new Date();
thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);
```

```

const enriched = products.map(product => {
  let stock = 0;
  let sold_last_30_days = 0;
  (product.transactions || []).forEach(tx => {
    if (tx.type === 'IN') {
      stock += tx.quantity;
    } else {
      stock -= tx.quantity;
      if (new Date(tx.created_at) >= thirtyDaysAgo) {
        sold_last_30_days += tx.quantity;
      }
    }
  });

  const { transactions, ...rest } = product;

  const stockStatus = stock <= 0 ? 'out_of_stock' : stock <= product.min_stock ? 'low'
: 'in_stock';

  return { ...rest, current_stock: stock, stock_status: stockStatus, sold_last_30_days
};

});

// Filter by stock_status after calculation and sort in-stock first if no specific status
let filtered = stock_status
? enriched.filter(p => p.stock_status === stock_status)

```

: enriched;

```
if (!stock_status && sort_by === 'created_at') {  
  filtered = filtered.sort((a, b) => {  
    const aInStock = a.current_stock > 0;  
    const bInStock = b.current_stock > 0;  
    if (aInStock && !bInStock) return -1;  
    if (!aInStock && bInStock) return 1;  
    // fallback to standard created_at sort if tie  
    return sort_order === 'asc'  
      ? new Date(a.created_at) - new Date(b.created_at)  
      : new Date(b.created_at) - new Date(a.created_at);  
  });  
}  
  
const totalCount = filtered.length;  
const paginatedProducts = filtered.slice(offset, offset + parseInt(limit));  
  
res.json({  
  products: paginatedProducts,  
  pagination: {  
    page: parseInt(page),  
    limit: parseInt(limit),  
    total: totalCount,  
    pages: Math.ceil(totalCount / parseInt(limit))  
  }  
})
```

```

    }
  });
} catch (err) {
  next(err);
}
});

```

```

// GET /api/products/:id
router.get('/:id', authenticate, async (req, res, next) => {
  try {
    const { data: product, error } = await supabase
      .from('products')
      .select('*', supplier:suppliers(id, name, contact_email, phone), transactions('*'))
      .eq('id', req.params.id)
      .eq('organization_id', req.user.organization_id)
      .eq('is_deleted', false)
      .single();

    if (error || !product) {
      return res.status(404).json({ error: 'Product not found' });
    }

    const stock = (product.transactions || []).reduce((acc, tx) => {
      return tx.type === 'IN' ? acc + tx.quantity : acc - tx.quantity;
    }, 0);
  }
}

```

```
res.json({
  ...product,
  current_stock: stock,
  stock_status: stock <= 0 ? 'out_of_stock' : stock <= product.min_stock ? 'low' :
'in_stock'
});
} catch (err) {
  next(err);
}
});

// POST /api/products
router.post('/', authenticate, requireRole('admin', 'manager'), [
  body('name').trim().notEmpty().withMessage('Product name is required'),
  body('price').isFloat({ min: 0 }).withMessage('Price must be a positive number'),
  body('category').trim().notEmpty().withMessage('Category is required'),
  body('min_stock').isInt({ min: 0 }).withMessage('Min stock must be a non-negative
integer'),
], async (req, res, next) => {
  try {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }
  }
```

```
const { name, sku, price, category, supplier_id, min_stock, description, unit,
image_url, initial_stock } = req.body;
```

```
// Auto-generate SKU if not provided
```

```
const finalSku = sku || generateSKU(name, category);
```

```
// Check SKU uniqueness within org
```

```
const { data: existing } = await supabase
```

```
.from('products')
```

```
.select('id')
```

```
.eq('sku', finalSku)
```

```
.eq('organization_id', req.user.organization_id)
```

```
.eq('is_deleted', false)
```

```
.single();
```

```
if (existing) {
```

```
  return res.status(409).json({ error: 'SKU already exists in your organization' });
```

```
}
```

```
const productId = uuidv4();
```

```
const { data: product, error } = await supabase
```

```
.from('products')
```

```
.insert({
```

```
  id: productId,
```

```

    name,
    sku: finalSku,
    price: parseFloat(price),
    category,
    supplier_id: supplier_id || null,
    min_stock: parseInt(min_stock),
    description: description || null,
    unit: unit || 'units',
    image_url: image_url || null,
    organization_id: req.user.organization_id,
    created_by: req.user.id,
    is_deleted: false
  })
  .select()
  .single();

if (error) throw error;

// Create initial stock transaction if provided
if (initial_stock && parseInt(initial_stock) > 0) {
  await supabase.from('transactions').insert({
    id: uuidv4(),
    product_id: productId,
    organization_id: req.user.organization_id,
    type: 'IN',

```

```

    quantity: parseInt(initial_stock),
    reason: 'Initial stock',
    created_by: req.user.id
  });
}

// Log audit
await supabase.from('audit_logs').insert({
  id: uuidv4(),
  organization_id: req.user.organization_id,
  user_id: req.user.id,
  action: 'CREATE_PRODUCT',
  resource_type: 'product',
  resource_id: productId,
  metadata: { name, sku: finalSku }
});

res.status(201).json({ ...product, current_stock: parseInt(initial_stock) || 0 });
} catch (err) {
  next(err);
}
});

// PUT /api/products/:id

```



```

router.put('/:id', authenticate, requireRole('admin', 'manager'), async (req, res, next)
=> {
  try {
    const { name, price, category, supplier_id, min_stock, description, unit, image_url }
    = req.body;

    const { data: product, error } = await supabase
      .from('products')
      .update({
        name, price: parseFloat(price), category,
        supplier_id: supplier_id || null,
        min_stock: parseInt(min_stock), description, unit, image_url,
        updated_at: new Date().toISOString()
      })
      .eq('id', req.params.id)
      .eq('organization_id', req.user.organization_id)
      .eq('is_deleted', false)
      .select()
      .single();

    if (error) throw error;
    if (!product) return res.status(404).json({ error: 'Product not found' });

    // Audit log
    await supabase.from('audit_logs').insert({

```

```

    id: uuidv4(),
    organization_id: req.user.organization_id,
    user_id: req.user.id,
    action: 'UPDATE_PRODUCT',
    resource_type: 'product',
    resource_id: req.params.id,
    metadata: { updated_fields: Object.keys(req.body) }
  });

  const stock = await calculateStock(req.params.id, req.user.organization_id);
  res.json({ ...product, current_stock: stock });
} catch (err) {
  next(err);
}
});

// DELETE /api/products/:id (soft delete)
router.delete('/:id', authenticate, requireRole('admin'), async (req, res, next) => {
  try {
    const { error } = await supabase
      .from('products')
      .update({ is_deleted: true, updated_at: new Date().toISOString() })
      .eq('id', req.params.id)
      .eq('organization_id', req.user.organization_id);
  }
});

```

```
if (error) throw error;

await supabase.from('audit_logs').insert({
  id: uuidv4(),
  organization_id: req.user.organization_id,
  user_id: req.user.id,
  action: 'DELETE_PRODUCT',
  resource_type: 'product',
  resource_id: req.params.id,
  metadata: {}
});

res.json({ message: 'Product deleted successfully' });
} catch (err) {
  next(err);
}
});

// POST /api/products/bulk-delete
router.post('/bulk-delete', authenticate, requireRole('admin'), async (req, res, next)
=> {
  try {
    const { ids } = req.body;
    if (!Array.isArray(ids) || ids.length === 0) {
      return res.status(400).json({ error: 'List of product IDs required' });
    }
  }
});
```

```
}

const { error } = await supabase
  .from('products')
  .update({ is_deleted: true, updated_at: new Date().toISOString() })
  .in('id', ids)
  .eq('organization_id', req.user.organization_id);

if (error) throw error;

await supabase.from('audit_logs').insert({
  id: uuidv4(),
  organization_id: req.user.organization_id,
  user_id: req.user.id,
  action: 'BULK_DELETE_PRODUCTS',
  resource_type: 'product',
  metadata: { deleted_count: ids.length, ids }
});

res.json({ message: `${ids.length} products deleted successfully` });
} catch (err) {
  next(err);
}
});
```

```
// GET /api/products/meta/categories

router.get('/meta/categories', authenticate, async (req, res, next) => {
  try {
    const { data, error } = await supabase
      .from('products')
      .select('category')
      .eq('organization_id', req.user.organization_id)
      .eq('is_deleted', false);

    if (error) throw error;

    const categories = [...new Set(data.map(p => p.category))].filter(Boolean);
    res.json({ categories });
  } catch (err) {
    next(err);
  }
});

export default router;
```

reports.js

```
import express from 'express';

import { supabase } from '../config/supabase.js';
import { authenticate } from '../middleware/auth.js';

const router = express.Router();

// GET /api/reports/dashboard - KPIs for dashboard
router.get('/dashboard', authenticate, async (req, res, next) => {
  try {
    const orgId = req.user.organization_id;

    const { data: products, error: pErr } = await supabase
      .from('products')
      .select('*')
      .eq('organization_id', orgId)
      .eq('is_deleted', false);

    if (pErr) throw pErr;

    // Get all transactions for these products to compute current stock
    const { data: allTx, error: allTxErr } = await supabase
      .from('transactions')
      .select('product_id, type, quantity')
      .eq('organization_id', orgId);
```

```

if (allTxErr) throw allTxErr;

// Build a stock map
const stockMap = {};

(allTx || []).forEach(tx => {
  if (!stockMap[tx.product_id]) stockMap[tx.product_id] = 0;
  if (tx.type === 'IN') stockMap[tx.product_id] += tx.quantity;
  else stockMap[tx.product_id] -= tx.quantity;
});

let totalValue = 0;
let outOfStock = 0;
let lowStock = 0;

const enriched = (products || []).map(p => {
  const stock = stockMap[p.id] || 0;
  const val = stock * (p.price || 0);
  totalValue += val;
  if (stock <= 0) outOfStock++;
  else if (stock <= (p.min_stock || 0)) lowStock++;
  return { ...p, current_stock: stock };
});

// Get transactions in last 30 days for trends

```

```

const thirtyDaysAgo = new Date();
thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);

const { data: recentTx, error: txErr } = await supabase
  .from('transactions')
  .select('id, type, quantity, created_at, unit_cost, product_id,
product:products(name, category, price)')
  .eq('organization_id', orgId)
  .gte('created_at', thirtyDaysAgo.toISOString())
  .order('created_at', { ascending: false });

if (txErr) {
  console.error('Error fetching recent transactions:', txErr);
  throw txErr;
}

// Map product data from our local products array for extra safety
const productMap = {};
(products || []).forEach(p => productMap[p.id] = p);

const recentTxEnriched = (recentTx || []).map(tx => {
  const p = productMap[tx.product_id] || tx.product || {};
  return {
    ...tx,
    product: {

```



```
    ...tx.product,  
    ...p  
  }  
};  
});
```

```
if (!recentTx) {  
  console.error('No recent transactions data returned from Supabase');  
  throw new Error('Database returned empty transaction set');  
}
```

```
const totalSalesQty = recentTx.filter(t => t.type === 'OUT').reduce((s, t) => s +  
t.quantity, 0);
```

```
const totalRestockQty = recentTx.filter(t => t.type === 'IN').reduce((s, t) => s +  
t.quantity, 0);
```

```
const salesByDay = {};
```

```
const last7Days = [];
```

```
for (let i = 6; i >= 0; i--) {
```

```
  const d = new Date();
```

```
  d.setDate(d.getDate() - i);
```

```
  const key = d.toISOString().split('T')[0];
```

```
  last7Days.push(key);
```

```
  salesByDay[key] = { date: key, in: 0, out: 0, revenue: 0, profit: 0 };
```

```
}
```

```
let thisMonthRevenue = 0;

let thisMonthCost = 0;

recentTxEnriched.forEach(tx => {

  const day = tx.created_at.split('T')[0];

  if (tx.type === 'OUT') {

    const qty = tx.quantity;

    const price = tx.product?.price || 0;

    // cost_price column is missing in DB; fallback to unit_cost from transaction or 0

    const costPrice = tx.unit_cost || 0;

    thisMonthRevenue += (qty * price);

    thisMonthCost += (qty * costPrice);

    if (salesByDay[day]) {

      salesByDay[day].out += qty;

      salesByDay[day].revenue = (salesByDay[day].revenue || 0) + (qty * price);

      salesByDay[day].profit = (salesByDay[day].profit || 0) + (qty * (price -

costPrice));

    }

  } else if (tx.type === 'IN') {

    if (salesByDay[day]) {

      salesByDay[day].in += tx.quantity;

    }

  }

}
```

```

    }
  });

  const thisMonthProfit = thisMonthRevenue - thisMonthCost;

  // Category distribution
  const categoryMap = {};
  enriched.forEach(p => {
    const cat = p.category || 'Uncategorized';
    if (!categoryMap[cat]) categoryMap[cat] = { count: 0, value: 0 };
    categoryMap[cat].count++;
    categoryMap[cat].value += p.current_stock * p.price;
  });

  res.json({
    kpis: {
      total_skus: products.length,
      total_stock_value: Math.round(totalValue * 100) / 100,
      out_of_stock: outOfStock,
      low_stock: lowStock,
      sales_last_30d: totalSalesQty,
      restock_last_30d: totalRestockQty,
      profit_last_30d: Math.round(thisMonthProfit * 100) / 100
    },
    sales_trend: Object.values(salesByDay),
  });

```

```
category_distribution: Object.entries(categoryMap).map(([name, data]) => ({
  name, ...data
})),
recent_transactions: recentTxEnriched.slice(0, 10)
});
} catch (err) {
  next(err);
}
});
```

```
// GET /api/reports/inventory-valuation
router.get('/inventory-valuation', authenticate, async (req, res, next) => {
  try {
    const orgId = req.user.organization_id;

    const { data: products, error: pErr } = await supabase
      .from('products')
      .select('*')
      .eq('organization_id', orgId)
      .eq('is_deleted', false);

    if (pErr) throw pErr;

    const { data: allTx, error: allTxErr } = await supabase
      .from('transactions')
```

```

.select('product_id, type, quantity')
.eq('organization_id', orgId);

if (allTxErr) throw allTxErr;

const stockMap = {};
const soldMap = {};

(allTx || []).forEach(tx => {
  if (!stockMap[tx.product_id]) { stockMap[tx.product_id] = 0;
soldMap[tx.product_id] = 0; }

  if (tx.type === 'IN') stockMap[tx.product_id] += tx.quantity;
  else {
    stockMap[tx.product_id] -= tx.quantity;
    soldMap[tx.product_id] += tx.quantity;
  }
});

const valuation = (products || []).map(p => {
  const stock = stockMap[p.id] || 0;
  const total_sold = soldMap[p.id] || 0;

  // cost_price missing in DB, default to 0
  const cost_price = p.cost_price || 0;
  const profit_per_unit = (p.price || 0) - cost_price;
  const margin_pct = (p.price || 0) > 0 ? (profit_per_unit / p.price) * 100 : 0;
  const total_profit = profit_per_unit * total_sold;

```

```

    return {
      id: p.id, name: p.name, sku: p.sku, category: p.category,
      unit_price: p.price, cost_price, current_stock: stock,
      total_value: Math.round(stock * p.price * 100) / 100,
      profit_per_unit, margin_pct: Math.round(margin_pct * 10) / 10, total_profit,
      status: stock <= 0 ? 'out_of_stock' : stock <= p.min_stock ? 'low' : 'in_stock'
    };
  });

  const totalValue = valuation.reduce((s, p) => s + p.total_value, 0);

  res.json({ valuation, total_value: Math.round(totalValue * 100) / 100 });
} catch (err) {
  next(err);
}
});

// GET /api/reports/top-products
router.get('/top-products', authenticate, async (req, res, next) => {
  try {
    const orgId = req.user.organization_id;
    const { days = 30 } = req.query;
    const since = new Date();
    since.setDate(since.getDate() - parseInt(days));

```

```
const { data, error } = await supabase
  .from('transactions')
  .select('product_id, quantity, product:products(name, sku, category)')
  .eq('organization_id', orgId)
  .eq('type', 'OUT')
  .gte('created_at', since.toISOString());

if (error) throw error;

const productMap = {};
data.forEach(tx => {
  if (!productMap[tx.product_id]) {
    productMap[tx.product_id] = {
      product_id: tx.product_id,
      name: tx.product?.name,
      sku: tx.product?.sku,
      category: tx.product?.category,
      total_sold: 0
    };
  }
  productMap[tx.product_id].total_sold += tx.quantity;
});

const top = Object.values(productMap)
```

```
.sort((a, b) => b.total_sold - a.total_sold)

.slice(0, 10);

res.json({ top_products: top });
} catch (err) {
  next(err);
}
});

// GET /api/reports/dead-stock
router.get('/dead-stock', authenticate, async (req, res, next) => {
  try {
    const orgId = req.user.organization_id;
    const { days = 60 } = req.query;
    const since = new Date();
    since.setDate(since.getDate() - parseInt(days));

    // Get all products
    const { data: products, error: pErr } = await supabase
      .from('products')
      .select('id, name, sku, category, price')
      .eq('organization_id', orgId)
      .eq('is_deleted', false);

    if (pErr) throw pErr;
```



```

const { data: allTx, error: txErr } = await supabase
  .from('transactions')
  .select('product_id, type, quantity, created_at')
  .eq('organization_id', orgId);

if (txErr) throw txErr;

const txByProduct = {};
(allTx || []).forEach(tx => {
  if (!txByProduct[tx.product_id]) txByProduct[tx.product_id] = [];
  txByProduct[tx.product_id].push(tx);
});

const deadStock = (products || []).filter(p => {
  const txs = txByProduct[p.id] || [];
  const stock = txs.reduce((acc, tx) =>
    tx.type === 'IN' ? acc + tx.quantity : acc - tx.quantity, 0);
  if (stock <= 0) return false; // out of stock, not dead stock

  const recentSale = txs.some(tx =>
    tx.type === 'OUT' && new Date(tx.created_at) > since);
  return !recentSale;
}).map(p => {
  const txs = txByProduct[p.id] || [];

```

```
const stock = txs.reduce((acc, tx) =>
  tx.type === 'IN' ? acc + tx.quantity : acc - tx.quantity, 0);

  return { id: p.id, name: p.name, sku: p.sku, category: p.category, current_stock:
stock, value: stock * p.price };
});

res.json({ dead_stock: deadStock, total_items: deadStock.length });
} catch (err) {
  next(err);
}
});

export default router;
```



suppliers.js

```
import express from 'express';  
import { body, validationResult } from 'express-validator';  
import { v4 as uuidv4 } from 'uuid';  
import { supabase } from '../config/supabase.js';  
import { authenticate, requireRole } from '../middleware/auth.js';
```

```
const router = express.Router();
```

```
router.get('/', authenticate, async (req, res, next) => {
```

```
  try {
```

```
    const { data, error } = await supabase
```

```
      .from('suppliers')
```

```
      .select('*', products:products(id, name, sku)')
```

```
      .eq('organization_id', req.user.organization_id)
```

```
      .eq('is_deleted', false)
```

```
      .order('name');
```

```
    if (error) throw error;
```

```
    res.json({ suppliers: data });
```

```
  } catch (err) { next(err); }
```

```
});
```

```
router.get('/:id', authenticate, async (req, res, next) => {
```

```
  try {
```

```

const { data, error } = await supabase
  .from('suppliers')
  .select('*', products:products(id, name, sku, category)')
  .eq('id', req.params.id)
  .eq('organization_id', req.user.organization_id)
  .eq('is_deleted', false)
  .single();

if (error || !data) return res.status(404).json({ error: 'Supplier not found' });
res.json(data);
} catch (err) { next(err); }
});

router.post('/', authenticate, requireRole('admin', 'manager'), [
  body('name').trim().notEmpty().withMessage('Supplier name required'),
  body('contact_email').optional().isEmail().withMessage('Valid email required'),
], async (req, res, next) => {
  try {
    const errors = validationResult(req);
    if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array() });

    const { name, contact_email, phone, address, notes, lead_time_days } = req.body;

    const { data, error } = await supabase
      .from('suppliers')

```

```

.insert({
  id: uuidv4(),
  name, contact_email: contact_email || null,
  phone: phone || null,
  address: address || null,
  notes: notes || null,
  lead_time_days: lead_time_days ? parseInt(lead_time_days) : 7,
  organization_id: req.user.organization_id,
  is_deleted: false
})
.select()
.single();

if (error) throw error;
res.status(201).json(data);
} catch (err) { next(err); }
});

router.put('/:id', authenticate, requireRole('admin', 'manager'), async (req, res, next)
=> {
  try {
    const { name, contact_email, phone, address, notes, lead_time_days } = req.body;
    const { data, error } = await supabase
      .from('suppliers')

```

```
.update({ name, contact_email, phone, address, notes, lead_time_days,  
updated_at: new Date().toISOString() })
```

```
.eq('id', req.params.id)
```

```
.eq('organization_id', req.user.organization_id)
```

```
.select()
```

```
.single();
```

```
if (error) throw error;
```

```
res.json(data);
```

```
} catch (err) { next(err); }
```

```
});
```

```
router.delete('/:id', authenticate, requireRole('admin'), async (req, res, next) => {
```

```
  try {
```

```
    const { error } = await supabase
```

```
      .from('suppliers')
```

```
      .update({ is_deleted: true })
```

```
      .eq('id', req.params.id)
```

```
      .eq('organization_id', req.user.organization_id);
```

```
    if (error) throw error;
```

```
    res.json({ message: 'Supplier deleted' });
```

```
  } catch (err) { next(err); }
```

```
});
```

```
router.post('/bulk-delete', authenticate, requireRole('admin'), async (req, res, next)
=> {
  try {
    const { ids } = req.body;

    if (!Array.isArray(ids) || ids.length === 0) return res.status(400).json({ error: 'IDs
required' });

    const { error } = await supabase
      .from('suppliers')
      .update({ is_deleted: true })
      .in('id', ids)
      .eq('organization_id', req.user.organization_id);

    if (error) throw error;
    res.json({ message: `${ids.length} suppliers deleted` });
  } catch (err) { next(err); }
});

export default router;
```

transactions.js

```
import express from 'express';

import { body, validationResult } from 'express-validator';

import { v4 as uuidv4 } from 'uuid';

import { supabase } from '../config/supabase.js';

import { authenticate, requireRole } from '../middleware/auth.js';

import { calculateStock } from './products.js';

import { checkLowStockAlert } from '../services/alertService.js';

const router = express.Router();

// POST /api/transactions - Create a stock transaction (IN or OUT)
router.post('/', authenticate, requireRole('admin', 'manager', 'staff'), [
  body('product_id').isUUID().withMessage('Valid product ID required'),
  body('type').isIn(['IN', 'OUT']).withMessage('Type must be IN or OUT'),
  body('quantity').isInt({ min: 1 }).withMessage('Quantity must be a positive integer'),
  body('reason').trim().notEmpty().withMessage('Reason is required'),
], async (req, res, next) => {
  try {
    const errors = validationResult(req);

    if (!errors.isEmpty()) {
      return res.status(400).json({ errors: errors.array() });
    }
  }
```



```

    const { product_id, type, quantity, reason, reference_no, notes, unit_cost } =
req.body;

    const orgId = req.user.organization_id;

    // Verify product belongs to org

    console.log(`[DEBUG] Finding product ${product_id} for org ${orgId}`);

    const { data: product, error: productError } = await supabase

    .from('products')

    .select('id, name, min_stock, organization_id, is_deleted')

    .eq('id', product_id)

    .eq('organization_id', orgId)

    .eq('is_deleted', false)

    .single();

    if (productError || !product) {

        console.error(`[Transaction Error] Product lookup failed for ID: ${product_id}.
Error:`, productError);

        return res.status(404).json({ error: `Product with ID ${product_id} not found or
doesn't belong to your organization.` });

    }

    console.log(`[Transaction] Product found: ${product.name} (Min Stock:
${product.min_stock})`);

    // For OUT transactions, verify sufficient stock

    if (type === 'OUT') {

```

```

const currentStock = await calculateStock(product_id, orgId);
if (currentStock < parseInt(quantity)) {
  return res.status(400).json({
    error: `Insufficient stock. Current: ${currentStock}, Requested: ${quantity}`
  });
}
}

const txId = uuidv4();
const { data: transaction, error } = await supabase
  .from('transactions')
  .insert({
    id: txId,
    product_id,
    organization_id: orgId,
    type,
    quantity: parseInt(quantity),
    reason,
    reference_no: reference_no || null,
    notes: notes || null,
    unit_cost: unit_cost ? parseFloat(unit_cost) : 0,
    created_by: req.user.id
  })
  .select('*', product:products(id, name, sku, min_stock))
  .single();

```

```
if (error) throw error;

// Calculate new stock
const newStock = await calculateStock(product_id, orgId);

// Check if low stock alert should trigger
await checkLowStockAlert(product_id, product.name, newStock,
product.min_stock, orgId, req.user.id);

res.status(201).json({ transaction, new_stock: newStock });
} catch (err) {
  next(err);
}
});

// GET /api/transactions
router.get('/', authenticate, async (req, res, next) => {
  try {
    const {
      page = 1,
      limit = 25,
      product_id,
      type,
      start_date,
```

```

    end_date,
    search = ''
  } = req.query;

  const offset = (parseInt(page) - 1) * parseInt(limit);
  const orgId = req.user.organization_id;

  let queryBuilder = supabase
    .from('transactions')
    .select(`
      *,
      product:products(id, name, sku, category),
      user:users(id, name, email)
    `, { count: 'exact' })
    .eq('organization_id', orgId);

  if (product_id) queryBuilder = queryBuilder.eq('product_id', product_id);
  if (type && ['IN', 'OUT'].includes(type)) queryBuilder = queryBuilder.eq('type',
type);
  if (start_date) queryBuilder = queryBuilder.gte('created_at', start_date);
  if (end_date) queryBuilder = queryBuilder.lte('created_at', end_date +
'T23:59:59');

  const { data: transactions, error, count } = await queryBuilder
    .order('created_at', { ascending: false })

```

```

    .range(offset, offset + parseInt(limit) - 1);

    if (error) throw error;

    res.json({
      transactions,
      pagination: {
        page: parseInt(page),
        limit: parseInt(limit),
        total: count,
        pages: Math.ceil(count / parseInt(limit))
      }
    });
  } catch (err) {
    next(err);
  }
});

// GET /api/transactions/summary
router.get('/summary', authenticate, async (req, res, next) => {
  try {
    const orgId = req.user.organization_id;
    const { days = 30 } = req.query;
    const since = new Date();
    since.setDate(since.getDate() - parseInt(days));

```

```
const { data, error } = await supabase
  .from('transactions')
  .select('type, quantity, created_at, product:products(name, category)')
  .eq('organization_id', orgId)
  .gte('created_at', since.toISOString());

if (error) throw error;

const totalIn = data.filter(t => t.type === 'IN').reduce((s, t) => s + t.quantity, 0);
const totalOut = data.filter(t => t.type === 'OUT').reduce((s, t) => s + t.quantity, 0);

res.json({ total_in: totalIn, total_out: totalOut, total_transactions: data.length });
} catch (err) {
  next(err);
}
});

export default router;
```

aiService.js

```
import { GoogleGenerativeAI } from '@google/generative-ai';

import dotenv from 'dotenv';

dotenv.config();

const MODEL = process.env.GEMINI_MODEL || 'gemini-1.5-flash';

class AIService {
  constructor() {
    if (!process.env.GEMINI_API_KEY) {
      console.warn(' ⚠️ GEMINI_API_KEY not set. AI features will be disabled. ');
      this.enabled = false;
      return;
    }
    this.genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
    this.model = this.genAI.getGenerativeModel({ model: MODEL });
    this.enabled = true;
    console.log(' 🚀 AI Service initialized with model: ${MODEL} ');
  }

  async _call(prompt, systemPrompt = "") {
    if (!this.enabled) throw new Error('AI service not configured. Please set GEMINI_API_KEY. ');

    const fullPrompt = systemPrompt
```

```

? `${systemPrompt}\n\n${prompt}`

: prompt;

const result = await this.model.generateContent(fullPrompt);
const response = await result.response;
return response.text();
}

async _callJSON(prompt, systemPrompt = "") {
  const text = await this._call(prompt + '\n\nRespond ONLY with valid JSON. No
markdown, no backticks, no explanation.', systemPrompt);
  try {
    return JSON.parse(text.replace(/``json|``/g, "").trim());
  } catch {
    // Try to extract JSON
    const match = text.match(/\{[^\s\S]*\}|\[ [^\s\S]* \]/);
    if (match) return JSON.parse(match[0]);
    throw new Error('AI returned invalid JSON: ' + text.substring(0, 200));
  }
}

// Chat assistant
async chat(messages, context = "") {
  const systemPrompt = `You are an expert inventory management assistant for a
business called "The Curator".

```


You help managers and staff make smart inventory decisions based on data they provide.

Be concise, actionable, and data-driven in your responses.

```
${context ? `Business context: ${context}` : ''};
```

```
const conversationHistory = messages.map(m =>
  `${m.role === 'user' ? 'User' : 'Assistant'}: ${m.content}`
).join('\n');
```

```
const lastMessage = messages[messages.length - 1];
```

```
const prompt = `${conversationHistory}`;
```

```
return await this._call(prompt, systemPrompt);
```

```
}
```

```
// Demand prediction
```

```
async predictDemand(salesData, productName, days = 30) {
```

```
  const systemPrompt = `You are an inventory analytics AI. Analyze sales patterns
  and predict future demand.`;
```

```
  const prompt = `Analyze demand for "${productName}" and predict next ${days}
  days.`;
```

Historical sales data (last 90 days, daily quantities sold):

```
${JSON.stringify(salesData)}
```

Return JSON with this structure:

```
{  
  "predicted_demand": number,  
  "confidence": "high|medium|low",  
  "trend": "increasing|stable|decreasing",  
  "insight": "brief explanation",  
  "recommendation": "action to take",  
  "peak_periods": ["list of predicted high-demand periods"]  
};
```

```
  return await this._callJSON(prompt, systemPrompt);
```

```
}
```

```
// Reorder suggestions
```

```
async suggestReorder(products) {
```

```
  const systemPrompt = `You are an inventory optimization AI that helps prevent  
stockouts.`;
```

```
  const prompt = `Analyze these low-stock products and provide reorder  
recommendations:
```

```
  ${JSON.stringify(products)}
```

For each product, calculate if reorder is needed based on:

- avg_daily_sales: average daily sales
- lead_time_days: supplier lead time
- current_stock: current inventory

- min_stock: minimum threshold

Formula: $\text{reorder_qty} = (\text{avg_daily_sales} \times \text{lead_time_days} \times 1.5) - \text{current_stock}$

Return JSON:

```
{  
  "recommendations": [  
    {  
      "product_id": "id",  
      "product_name": "name",  
      "current_stock": number,  
      "suggested_order_qty": number,  
      "urgency": "critical|high|medium|low",  
      "reason": "explanation",  
      "estimated_stockout_days": number  
    }  
  ],  
  "summary": "overall summary"  
};
```

```
    return await this._callJSON(prompt, systemPrompt);  
  }
```

// Dead stock analysis

```
async analyzeDeadStock(deadStockItems) {
```

```
const systemPrompt = `You are a retail inventory optimization specialist.`;  
  
const prompt = `These products haven't had any sales in 60+ days. Analyze and  
suggest actions:
```

```
${JSON.stringify(deadStockItems)}
```

Return JSON:

```
{  
  "total_dead_stock_value": number,  
  "recommendations": [  
    {  
      "product_id": "id",  
      "product_name": "name",  
      "action": "discount|bundle|return_to_supplier|write_off|seasonal",  
      "suggested_discount_pct": number or null,  
      "reason": "why this action"  
    }  
  ],  
  "insight": "overall strategic insight"  
};
```

```
return await this._callJSON(prompt, systemPrompt);  
}
```

```
// CSV column mapping for products
```

```
async mapCSVColumns(headers, sampleData = []) {  
  const systemPrompt = `You map CSV column headers to inventory system fields.`;  
  const prompt = `Map these CSV column headers to the correct inventory fields:
```

Headers: \${JSON.stringify(headers)}

Sample data (first 3 rows): \${JSON.stringify(sampleData)}

Target fields: name, sku, price, category, supplier, min_stock, description, unit,
initial_stock

Return JSON where key = target field, value = matching CSV header (or null if not
found):

```
{  
  "name": "column_header_or_null",  
  "sku": "column_header_or_null",  
  "price": "column_header_or_null",  
  "category": "column_header_or_null",  
  "supplier": "column_header_or_null",  
  "min_stock": "column_header_or_null",  
  "description": "column_header_or_null",  
  "unit": "column_header_or_null",  
  "initial_stock": "column_header_or_null"  
};
```

```
return await this._callJSON(prompt, systemPrompt);
```

```
}
```

```
// CSV column mapping for suppliers
```

```
async mapSupplierColumns(headers, sampleData = []) {
```

```
  const systemPrompt = `You map CSV column headers to supplier database fields.`;
```

```
  const prompt = `Map these CSV column headers to the correct supplier fields:
```

```
  Headers: ${JSON.stringify(headers)}
```

```
  Sample data (first 3 rows): ${JSON.stringify(sampleData)}
```

```
  Target fields: name, contact_email, phone, address, lead_time_days, notes
```

```
  Return JSON where key = target field, value = matching CSV header (or null if not found):
```

```
{
```

```
  "name": "column_header_or_null",
```

```
  "contact_email": "column_header_or_null",
```

```
  "phone": "column_header_or_null",
```

```
  "address": "column_header_or_null",
```

```
  "lead_time_days": "column_header_or_null",
```

```
  "notes": "column_header_or_null"
```

```
};
```

```
  return await this._callJSON(prompt, systemPrompt);
```

```
}
```

// SKU generator

```
async generateSKU(productName, category = '') {
```

```
  const prompt = `Generate a deterministic, structured SKU code for a product.
```

```
  Product name: "${productName}"
```

```
  Category: "${category}"
```

Rules:

- 6-12 characters max
- Uppercase letters and numbers only
- No spaces or special chars
- Must be strictly derived from the product name and category.
- First 3 chars represent the category, followed by chars from the product name.
- ALWAYS return the exact same SKU for the exact same input. Wait, no random numbers.

```
Return JSON: { "sku": "GENERATEDSKU" };
```

```
  const result = await this._callJSON(prompt);
```

```
  return result.sku;
```

```
}
```

// General insights from dashboard data

```
async generateDashboardInsights(dashboardData) {
```

```
const systemPrompt = `You are a business intelligence AI for inventory management.`;
```

```
const prompt = `Analyze this inventory dashboard data and provide 3 key actionable insights:
```

```
${JSON.stringify(dashboardData)}
```

Return JSON:

```
{  
  "insights": [  
    {  
      "type": "warning|opportunity|info",  
      "title": "short title",  
      "description": "detailed insight",  
      "action": "recommended action"  
    }  
  ]  
};
```

```
return await this._callJSON(prompt, systemPrompt);  
}  
}
```

```
export default new AIService();
```


alertService.js

```
import { v4 as uuidv4 } from 'uuid';

import { supabase } from '../config/supabase.js';

export async function checkLowStockAlert(productId, productName, currentStock,
minStock, organizationId, userId) {

  try {

    if (currentStock <= minStock) {

      const alertType = currentStock <= 0 ? 'out_of_stock' : 'low_stock';

      // Check for existing unresolved alert

      const { data: existing } = await supabase

        .from('alerts')

        .select('id')

        .eq('product_id', productId)

        .eq('organization_id', organizationId)

        .eq('type', alertType)

        .eq('resolved', false)

        .single();

      if (!existing) {

        await supabase.from('alerts').insert({

          id: uuidv4(),

          product_id: productId,

          organization_id: organizationId,
```

```
type: alertType,
message: currentStock <= 0
  ? `${productName} is OUT OF STOCK`
  : `${productName} has low stock: ${currentStock} units remaining (min:
${minStock})`,
current_stock: currentStock,
min_stock: minStock,
resolved: false,
created_by: userId
});
}
} else {
  // Resolve any existing alerts if stock is now above min
  await supabase.from('alerts')
    .update({ resolved: true, resolved_at: new Date().toISOString() })
    .eq('product_id', productId)
    .eq('organization_id', organizationId)
    .eq('resolved', false);
}
} catch (err) {
  console.error('Alert check failed:', err);
}
}
```

authService.js

```
import jwt from 'jsonwebtoken';

import { v4 as uuidv4 } from 'uuid';

import { supabase } from '../config/supabase.js';

import { env } from '../config/env.js';

import { ApiError } from '../utils/apiError.js';

import { hashPassword, verifyPassword } from '../utils/passwords.js';

const SAFE_USER_FIELDS = 'id, name, email, role, organization_id, is_active,
created_at, updated_at';

const USER_WITH_PASSWORD_FIELDS = `${SAFE_USER_FIELDS}, password_hash`;

function signAuthToken(user) {
  return jwt.sign(
    {
      userId: user.id,
      organizationId: user.organization_id,
    },
    env.jwtSecret,
    { expiresIn: env.jwtExpiresIn }
  );
}

function sanitizeUser(user) {
  if (!user) {
```

```
    return null;
  }
```

```
    const { password_hash, ...safeUser } = user;
    return safeUser;
  }
```

```
export async function findUserByEmail(email, { includePasswordHash = false } = {})
{
  const { data, error } = await supabase
    .from('users')
    .select(includePasswordHash ? USER_WITH_PASSWORD_FIELDS :
SAFE_USER_FIELDS)
    .eq('email', email)
    .maybeSingle();

  if (error) {
    throw error;
  }

  return data;
}
```

```
export async function findUserById(userId, { includePasswordHash = false } = {}) {
  const { data, error } = await supabase
```

```

    .from('users')

    .select(includePasswordHash ? USER_WITH_PASSWORD_FIELDS :
SAFE_USER_FIELDS)

    .eq('id', userId)

    .maybeSingle();

if (error) {
    throw error;
}

return data;
}

export async function registerUser({ name, email, password, organizationName }) {
    const existingUser = await findUserByEmail(email);
    if (existingUser) {
        throw new ApiError('Email already registered', 409);
    }

    const organizationId = uuidv4();
    const userId = uuidv4();
    const now = new Date().toISOString();
    const passwordHash = await hashPassword(password);

    const { error: organizationError } = await supabase

```

```
.from('organizations')

.insert({
  id: organizationId,
  name: organizationName,
  plan: 'starter',
  created_at: now,
  updated_at: now,
});

if (organizationError) {
  throw organizationError;
}

try {
  const { data: user, error: userError } = await supabase
    .from('users')
    .insert({
      id: userId,
      name,
      email,
      password_hash: passwordHash,
      role: 'admin',
      organization_id: organizationId,
      is_active: true,
      created_at: now,
```

```
        updated_at: now,
    })
    .select(SAFE_USER_FIELDS)
    .single();

    if (userError) {
        throw userError;
    }

    return {
        token: signAuthToken(user),
        user,
    };
} catch (error) {
    await supabase.from('organizations').delete().eq('id', organizationId);
    throw error;
}
}

export async function loginUser({ email, password }) {
    const user = await findUserByEmail(email, { includePasswordHash: true });

    if (!user) {
        throw new ApiError('Invalid credentials', 401);
    }
}
```

```
if (!user.is_active) {  
  throw new ApiError('Account deactivated', 403);  
}  
  
const isPasswordValid = await verifyPassword(password, user.password_hash);  
if (!isPasswordValid) {  
  throw new ApiError('Invalid credentials', 401);  
}  
  
return {  
  token: signAuthToken(user),  
  user: sanitizeUser(user),  
};  
}  
  
export async function inviteTeamMember({  
  organizationId,  
  name,  
  email,  
  role,  
  password,  
}) {  
  const existingUser = await findUserByEmail(email);  
  if (existingUser) {
```



```
throw new ApiError('Email already registered', 409);
}

const now = new Date().toISOString();
const passwordHash = await hashPassword(password);
const userId = uuidv4();

const { data: user, error } = await supabase
  .from('users')
  .insert({
    id: userId,
    name,
    email,
    password_hash: passwordHash,
    role,
    organization_id: organizationId,
    is_active: true,
    created_at: now,
    updated_at: now,
  })
  .select(SAFE_USER_FIELDS)
  .single();

if (error) {
  throw error;
}
```

```
}
```

```
  return user;
```

```
}
```

```
export async function listTeamMembers(organizationId) {
```

```
  const { data, error } = await supabase
```

```
    .from('users')
```

```
    .select('id, name, email, role, is_active, updated_at')
```

```
    .eq('organization_id', organizationId)
```

```
    .order('name');
```

```
  if (error) {
```

```
    throw error;
```

```
}
```

```
  return (data || []).map((member) => ({
```

```
    ...member,
```

```
    deactivated_at: member.is_active ? null : member.updated_at || null,
```

```
  }));
```

```
}
```

```
export async function reactivateTeamMember({ organizationId, targetUserId }) {
```

```
  const { data: user, error: fetchError } = await supabase
```

```
    .from('users')
```

```
.select('id, is_active, updated_at')
.eq('id', targetUserId)
.eq('organization_id', organizationId)
.maybeSingle();

if (fetchError) {
  throw fetchError;
}

if (!user) {
  throw new ApiError('User not found', 404);
}

if (user.is_active) {
  throw new ApiError('User is already active', 400);
}

if (user.updated_at) {
  const daysSinceDeactivation = (Date.now() - new
Date(user.updated_at).getTime()) / (1000 * 60 * 60 * 24);

  if (daysSinceDeactivation > 45) {
    throw new ApiError('Reactivation window has expired for this member', 403);
  }
}
```

```

const { data, error } = await supabase
  .from('users')
  .update({
    is_active: true,
    updated_at: new Date().toISOString(),
  })
  .eq('id', targetUserId)
  .eq('organization_id', organizationId)
  .select(SAFE_USER_FIELDS)
  .single();

if (error) {
  throw error;
}

return data;
}

export async function deactivateTeamMember({
  organizationId,
  actorUserId,
  targetUserId,
}) {
  if (actorUserId === targetUserId) {
    throw new ApiError('You cannot deactivate your own account', 400);
  }
}

```

```
}
```

```
const { data: user, error: fetchError } = await supabase  
  .from('users')  
  .select('id, is_active')  
  .eq('id', targetUserId)  
  .eq('organization_id', organizationId)  
  .maybeSingle();
```

```
if (fetchError) {  
  throw fetchError;  
}
```

```
if (!user) {  
  throw new ApiError('User not found', 404);  
}
```

```
if (!user.is_active) {  
  throw new ApiError('User is already deactivated', 400);  
}
```

```
const now = new Date().toISOString();  
const { data, error } = await supabase  
  .from('users')  
  .update({
```

```
    is_active: false,
    updated_at: now,
  })
  .eq('id', targetUserId)
  .eq('organization_id', organizationId)
  .select(SAFE_USER_FIELDS)
  .single();

if (error) {
  throw error;
}

return data;
}

export async function deleteOrganization(organizationId) {
  const { data: organization, error: fetchError } = await supabase
    .from('organizations')
    .select('id')
    .eq('id', organizationId)
    .maybeSingle();

  if (fetchError) {
    throw fetchError;
  }
}
```

```
if (!organization) {  
  throw new ApiError('Organization not found', 404);  
}
```

```
const { error } = await supabase  
  .from('organizations')  
  .delete()  
  .eq('id', organizationId);
```

```
if (error) {  
  throw error;  
}  
}
```



emailService.js

```
import nodemailer from 'nodemailer';

import { env, isSmtplibConfigured } from '../config/env.js';

import { ApiError } from '../utils/apiError.js';

import { buildPasswordResetEmail } from '../utils/emailTemplates.js';

let transporter;

function getTransporter() {
  if (!isSmtplibConfigured()) {
    if (env.nodeEnv === 'development') {
      return {
        sendMail: async (options) => {
          console.log('\n--- SIMULATED EMAIL ---');
          console.log('To: ${options.to}');
          console.log('Subject: ${options.subject}');
          console.log('Text:\n${options.text}');
          console.log('-----\n');
          return true;
        }
      };
    }
    throw new ApiError('Password reset email service is not configured.', 503);
  }
}
```



```
if (!transporter) {  
  transporter = nodemailer.createTransport({  
    host: env.smtpHost,  
    port: env.smtpPort,  
    secure: env.smtpSecure,  
    auth: {  
      user: env.smtpUser,  
      pass: env.smtpPass,  
    },  
  });  
}  
  
return transporter;  
}  
  
export function assertEmailServiceConfigured() {  
  if (!isSmtplibConfigured() && env.nodeEnv !== 'development') {  
    throw new ApiError('Password reset email service is not configured.', 503);  
  }  
}  
  
export async function sendPasswordResetEmail({  
  to,  
  name,  
  resetUrl,
```

```
    expiresInMinutes,  
  }) {  
    const mailer = getTransporter();  
    const { subject, text, html } = buildPasswordResetEmail({  
      appName: env.appName,  
      recipientName: name,  
      resetUrl,  
      expiresInMinutes,  
    });  
  
    await mailer.sendMail({  
      from: `${env.mailFromName} <${env.mailFromEmail}>`,  
      to,  
      subject,  
      text,  
      html,  
    });  
  }  
}
```

passwordResetService.js

```
import { supabase } from '../config/supabase.js';
import { env } from '../config/env.js';
import { ApiError } from '../utils/apiError.js';
import { generateSecureToken, hashToken } from '../utils/crypto.js';
import { hashPassword, verifyPassword } from '../utils/passwords.js';
import { assertEmailServiceConfigured, sendPasswordResetEmail } from
'./emailService.js';
import { findUserByEmail } from './authService.js';

export const PASSWORD_RESET_REQUEST_RESPONSE =
  'If an account exists for this email, a reset link has been sent.';

const INVALID_RESET_TOKEN_MESSAGE = 'Reset token is invalid or has expired.';

function buildResetUrl(rawToken) {
  return
  `${env.frontendUrl}${env.resetPasswordPath}?token=${encodeURIComponent(raw
  Token)}`;
}

async function purgeExpiredPasswordResetTokens() {
  const { error } = await supabase
    .from('password_reset_tokens')
    .delete()
    .lte('expires_at', new Date().toISOString());
```

```
if (error) {  
  throw error;  
}  
}
```

```
async function replaceUserPasswordResetToken(userId, tokenHash, expiresAt) {
```

```
  const { error: deleteError } = await supabase
```

```
    .from('password_reset_tokens')
```

```
    .delete()
```

```
    .eq('user_id', userId);
```

```
if (deleteError) {  
  throw deleteError;  
}
```

```
const { data, error } = await supabase
```

```
  .from('password_reset_tokens')
```

```
  .insert({
```

```
    user_id: userId,
```

```
    token_hash: tokenHash,
```

```
    expires_at: expiresAt,
```

```
  })
```

```
  .select('id')
```

```
  .single();
```

```
if (error) {  
  throw error;  
}  
  
return data;  
}  
  
export async function requestPasswordReset(email) {  
  assertEmailServiceConfigured();  
  await purgeExpiredPasswordResetTokens();  
  
  const user = await findUserByEmail(email);  
  if (!user || !user.is_active) {  
    return PASSWORD_RESET_REQUEST_RESPONSE;  
  }  
  
  const rawToken = generateSecureToken();  
  const tokenHash = hashToken(rawToken);  
  const expiresAt = new Date(  
    Date.now() + env.passwordResetTokenTtlMinutes * 60 * 1000  
  ).toISOString();  
  
  const tokenRecord = await replaceUserPasswordResetToken(user.id, tokenHash,  
    expiresAt);
```

```
const resetUrl = buildResetUrl(rawToken);

try {
  await sendPasswordResetEmail({
    to: user.email,
    name: user.name,
    resetUrl,
    expiresInMinutes: env.passwordResetTokenTtlMinutes,
  });
} catch (error) {
  console.error('Password reset email delivery failed:', error);
  await supabase.from('password_reset_tokens').delete().eq('id', tokenRecord.id);
}

return PASSWORD_RESET_REQUEST_RESPONSE;
}

export async function resetPasswordWithToken(rawToken, newPassword) {
  await purgeExpiredPasswordResetTokens();

  const tokenHash = hashToken(rawToken);
  const now = new Date().toISOString();

  const { data: resetToken, error: tokenLookupError } = await supabase
    .from('password_reset_tokens')
```

```
.select('id, user_id, expires_at')
.eq('token_hash', tokenHash)
.gt('expires_at', now)
.maybeSingle();

if (tokenLookupError) {
  throw tokenLookupError;
}

if (!resetToken) {
  throw new ApiError(INVALID_RESET_TOKEN_MESSAGE, 400);
}

const { data: user, error: userLookupError } = await supabase
  .from('users')
  .select('id, is_active, password_hash')
  .eq('id', resetToken.user_id)
  .maybeSingle();

if (userLookupError) {
  throw userLookupError;
}

if (!user) {
  throw new ApiError(INVALID_RESET_TOKEN_MESSAGE, 400);
}
```

```
}

if (!user.is_active) {
  throw new ApiError('Account deactivated. Contact your administrator.', 403);
}

const isSamePassword = await verifyPassword(newPassword,
user.password_hash);
if (isSamePassword) {
  throw new ApiError('Choose a password you have not used recently.', 400);
}

// Delete first so the token cannot be replayed if the user retries the request.
const { data: deletedToken, error: deleteError } = await supabase
  .from('password_reset_tokens')
  .delete()
  .eq('id', resetToken.id)
  .eq('token_hash', tokenHash)
  .select('id')
  .maybeSingle();

if (deleteError) {
  throw deleteError;
}
```



```
if (!deletedToken) {  
  throw new ApiError(INVALID_RESET_TOKEN_MESSAGE, 400);  
}  
  
const passwordHash = await hashPassword(newPassword);  
const { error: updateError } = await supabase  
  .from('users')  
  .update({  
    password_hash: passwordHash,  
    updated_at: new Date().toISOString(),  
  })  
  .eq('id', user.id);  
  
if (updateError) {  
  throw new ApiError('Unable to update password. Please request a new reset  
link.', 500);  
}  
  
const { error: cleanupError } = await supabase  
  .from('password_reset_tokens')  
  .delete()  
  .eq('user_id', user.id);  
  
if (cleanupError) {  
  throw cleanupError;  
}
```

```
}
```

```
return {
```

```
    message: 'Password updated successfully. You can sign in now.',
```

```
};
```

```
}
```



index.js

```
import express from 'express';

import cors from 'cors';

import helmet from 'helmet';

import { env } from './config/env.js';

import { errorHandler, notFoundHandler } from './middleware/errorHandler.js';

import { aiLimiter, apiLimiter } from './middleware/rateLimiters.js';

import authRoutes from './routes/auth.js';

import productRoutes from './routes/products.js';

import transactionRoutes from './routes/transactions.js';

import supplierRoutes from './routes/suppliers.js';

import reportRoutes from './routes/reports.js';

import aiRoutes from './routes/ai.js';

import importExportRoutes from './routes/importExport.js';

import alertRoutes from './routes/alerts.js';

const app = express();

if (env.nodeEnv === 'production') {
  app.set('trust proxy', 1);
}

app.use(helmet());

app.use(cors({
```

```
origin: env.frontendUrl,  
credentials: true,  
});  
  
app.use(apiLimiter);  
app.use(express.json({ limit: '10mb' }));  
app.use(express.urlencoded({ extended: true, limit: '10mb' }));  
  
app.use((req, res, next) => {  
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`);  
  next();  
});  
  
app.get('/health', (req, res) => {  
  res.json({ status: 'ok', timestamp: new Date().toISOString() });  
});  
  
app.use('/api/auth', authRoutes);  
app.use('/api/products', productRoutes);  
app.use('/api/transactions', transactionRoutes);  
app.use('/api/suppliers', supplierRoutes);  
app.use('/api/reports', reportRoutes);  
app.use('/api/ai', aiLimiter, aiRoutes);  
app.use('/api', importExportRoutes);  
app.use('/api/alerts', alertRoutes);
```

```
app.use(notFoundHandler);  
app.use(errorHandler);  
  
if (env.nodeEnv !== 'test') {  
  app.listen(env.port, () => {  
    console.log(`StockGrid API running on port ${env.port}`);  
  });  
}  
  
export default app;
```



Database

schema.sql

```
-- =====  
  
-- Stitch AI Inventory SaaS - Complete Database Schema  
  
-- Run this in your Supabase SQL Editor  
  
-- =====  
  
-- Enable UUID extension  
CREATE EXTENSION IF NOT EXISTS "uuid-oss";  
  
-- =====  
  
-- ORGANIZATIONS (Multi-tenant root)  
-- =====  
  
CREATE TABLE IF NOT EXISTS organizations (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  name VARCHAR(255) NOT NULL,  
  plan VARCHAR(50) DEFAULT 'starter', -- starter, pro, enterprise  
  settings JSONB DEFAULT '{}',  
  created_at TIMESTAMPTZ DEFAULT NOW(),  
  updated_at TIMESTAMPTZ DEFAULT NOW()  
);  
  
-- =====  
  
-- USERS
```

```

-- =====

CREATE TABLE IF NOT EXISTS users (

  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE
  CASCADE,

  name VARCHAR(255) NOT NULL,

  email VARCHAR(255) UNIQUE NOT NULL,

  password_hash VARCHAR(255) NOT NULL,

  role VARCHAR(50) NOT NULL DEFAULT 'staff', -- admin, manager, staff

  is_active BOOLEAN DEFAULT TRUE,

  deactivated_at TIMESTAMPTZ,

  avatar_url TEXT,

  created_at TIMESTAMPTZ DEFAULT NOW(),

  updated_at TIMESTAMPTZ DEFAULT NOW()

);

CREATE INDEX IF NOT EXISTS idx_users_org ON users(organization_id);
CREATE INDEX IF NOT EXISTS idx_users_email ON users(email);

-- =====

-- PASSWORD RESET TOKENS

-- =====

CREATE TABLE IF NOT EXISTS password_reset_tokens (

  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),

  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,

```

```
token_hash TEXT NOT NULL,  
expires_at TIMESTAMPTZ NOT NULL,  
created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

```
CREATE UNIQUE INDEX IF NOT EXISTS idx_password_reset_tokens_hash  
ON password_reset_tokens(token_hash);  
CREATE INDEX IF NOT EXISTS idx_password_reset_tokens_user_id  
ON password_reset_tokens(user_id);  
CREATE INDEX IF NOT EXISTS idx_password_reset_tokens_expires_at  
ON password_reset_tokens(expires_at);
```

```
-- =====  
-- SUPPLIERS  
-- =====
```

```
CREATE TABLE IF NOT EXISTS suppliers (  
id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE  
CASCADE,  
name VARCHAR(255) NOT NULL,  
contact_email VARCHAR(255),  
phone VARCHAR(100),  
address TEXT,  
notes TEXT,  
lead_time_days INTEGER DEFAULT 7,
```



```
is_deleted BOOLEAN DEFAULT FALSE,  
created_at TIMESTAMPTZ DEFAULT NOW(),  
updated_at TIMESTAMPTZ DEFAULT NOW()  
);
```

```
CREATE INDEX IF NOT EXISTS idx_suppliers_org ON suppliers(organization_id);
```

```
-- =====
```

```
-- PRODUCTS
```

```
-- =====
```

```
CREATE TABLE IF NOT EXISTS products (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE  
  CASCADE,  
  name VARCHAR(255) NOT NULL,  
  sku VARCHAR(100) NOT NULL,  
  price DECIMAL(12, 2) NOT NULL DEFAULT 0,  
  cost_price DECIMAL(12, 2) DEFAULT 0,  
  category VARCHAR(100),  
  supplier_id UUID REFERENCES suppliers(id) ON DELETE SET NULL,  
  supplier_name VARCHAR(255),  
  min_stock INTEGER NOT NULL DEFAULT 5,  
  description TEXT,  
  unit VARCHAR(50) DEFAULT 'units',  
  image_url TEXT,
```

```

    created_by UUID REFERENCES users(id) ON DELETE SET NULL,
    is_deleted BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT NOW(),
    updated_at TIMESTAMPTZ DEFAULT NOW(),
    UNIQUE(organization_id, sku)
);

```

```

CREATE INDEX IF NOT EXISTS idx_products_org ON products(organization_id);
CREATE INDEX IF NOT EXISTS idx_products_sku ON products(sku);
CREATE INDEX IF NOT EXISTS idx_products_category ON products(category);
CREATE INDEX IF NOT EXISTS idx_products_supplier ON products(supplier_id);

```

```

-- =====
-- TRANSACTIONS (The source of truth for stock levels)
-- =====

```

```

CREATE TABLE IF NOT EXISTS transactions (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE CASCADE,
    product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
    type VARCHAR(10) NOT NULL CHECK (type IN ('IN', 'OUT')),
    quantity INTEGER NOT NULL CHECK (quantity > 0),
    reason VARCHAR(255) NOT NULL,
    reference_no VARCHAR(100),
    notes TEXT,

```

```
unit_cost DECIMAL(12, 2),
created_by UUID REFERENCES users(id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);
```

```
CREATE INDEX IF NOT EXISTS idx_transactions_org ON
transactions(organization_id);
```

```
CREATE INDEX IF NOT EXISTS idx_transactions_product ON
transactions(product_id);
```

```
CREATE INDEX IF NOT EXISTS idx_transactions_type ON transactions(type);
```

```
CREATE INDEX IF NOT EXISTS idx_transactions_created ON transactions(created_at
DESC);
```

```
-- =====
-- ALERTS (Low stock and out-of-stock notifications)
-- =====
```

```
CREATE TABLE IF NOT EXISTS alerts (
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
  organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE
  CASCADE,
  product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,
  type VARCHAR(50) NOT NULL, -- low_stock, out_of_stock
  message TEXT NOT NULL,
  current_stock INTEGER,
  min_stock INTEGER,
  resolved BOOLEAN DEFAULT FALSE,
```

```

resolved_at TIMESTAMPTZ,
created_by UUID REFERENCES users(id) ON DELETE SET NULL,
created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_alerts_org ON alerts(organization_id);
CREATE INDEX IF NOT EXISTS idx_alerts_resolved ON alerts(resolved);

-- =====
-- AUDIT LOGS (Full audit trail)
-- =====

CREATE TABLE IF NOT EXISTS audit_logs (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    organization_id UUID NOT NULL REFERENCES organizations(id) ON DELETE
    CASCADE,
    user_id UUID REFERENCES users(id) ON DELETE SET NULL,
    action VARCHAR(100) NOT NULL,
    resource_type VARCHAR(50),
    resource_id UUID,
    metadata JSONB DEFAULT '{}',
    ip_address VARCHAR(50),
    created_at TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX IF NOT EXISTS idx_audit_org ON audit_logs(organization_id);

```

```
CREATE INDEX IF NOT EXISTS idx_audit_user ON audit_logs(user_id);  
CREATE INDEX IF NOT EXISTS idx_audit_created ON audit_logs(created_at DESC);
```

```
-- =====
```

```
-- VIEWS
```

```
-- =====
```

```
-- Current stock view (computed from transactions)
```

```
CREATE OR REPLACE VIEW current_stock AS
```

```
SELECT
```

```
  p.id AS product_id,
```

```
  p.organization_id,
```

```
  p.name,
```

```
  p.sku,
```

```
  p.category,
```

```
  p.price,
```

```
  p.cost_price,
```

```
  p.min_stock,
```

```
  p.supplier_id,
```

```
  COALESCE(SUM(CASE WHEN t.type = 'IN' THEN t.quantity ELSE -t.quantity END),  
0)::INTEGER AS current_stock,
```

```
  CASE
```

```
    WHEN COALESCE(SUM(CASE WHEN t.type = 'IN' THEN t.quantity ELSE -t.quantity  
END), 0) <= 0 THEN 'out_of_stock'
```

```
    WHEN COALESCE(SUM(CASE WHEN t.type = 'IN' THEN t.quantity ELSE -t.quantity  
END), 0) <= p.min_stock THEN 'low_stock'
```

```
ELSE 'in_stock'

END AS stock_status

FROM products p

LEFT JOIN transactions t ON t.product_id = p.id

WHERE p.is_deleted = FALSE

GROUP BY p.id, p.organization_id, p.name, p.sku, p.category, p.price, p.cost_price,
p.min_stock, p.supplier_id;
```

```
-- =====
```

```
-- ROW LEVEL SECURITY (RLS) - Enable for multi-tenancy
```

```
-- =====
```

```
ALTER TABLE organizations ENABLE ROW LEVEL SECURITY;
ALTER TABLE users ENABLE ROW LEVEL SECURITY;
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE transactions ENABLE ROW LEVEL SECURITY;
ALTER TABLE suppliers ENABLE ROW LEVEL SECURITY;
ALTER TABLE alerts ENABLE ROW LEVEL SECURITY;
ALTER TABLE audit_logs ENABLE ROW LEVEL SECURITY;
```

```
-- Note: Since we're using service role key in backend,
```

```
-- RLS won't block our API. But it protects direct DB access.
```

```
-- For the service role, all operations are allowed.
```

```
-- =====
```

-- SAMPLE DATA (Optional - uncomment to use)

-- =====

-- INSERT INTO organizations (id, name, plan) VALUES

-- ('00000000-0000-0000-0000-000000000001', 'Demo Company', 'pro');

-- INSERT INTO users (id, organization_id, name, email, password_hash, role) VALUES

-- ('00000000-0000-0000-0000-000000000002',

-- '00000000-0000-0000-0000-000000000001',

-- 'Admin User', 'admin@demo.com',

-- '\$2a\$12\$LQv3c1yqBWVHxkd0LHAKCOYz6TtxMQJqhN8/LewdBPj3bp/lqJ7Z6', --
password: Admin123!

-- 'admin');



Frontend

AppLayout.jsx

```
import { useState, useEffect } from 'react'

import { Outlet, NavLink, useNavigate } from 'react-router-dom'

import { useQuery } from '@tanstack/react-query'

import { useAuthStore } from '../store/authStore'

import { alertsApi } from '../services/api'

const NAV_ITEMS = [

  { to: '/', icon: 'dashboard', label: 'Dashboard', exact: true },

  { to: '/products', icon: 'inventory_2', label: 'Inventory' },

  { to: '/transactions', icon: 'history_toggle_off', label: 'Activity' },

  { to: '/ai-insights', icon: 'psychology', label: 'AI Insights' },

  { to: '/suppliers', icon: 'local_shipping', label: 'Suppliers' },

  { to: '/reports', icon: 'bar_chart', label: 'Reports' },

]

export default function AppLayout() {

  const { user, logout } = useAuthStore()

  const navigate = useNavigate()

  const [isMobileMenuOpen, setIsMobileMenuOpen] = useState(false)

  const closeMenu = () => setIsMobileMenuOpen(false)

  const [isDarkMode, setIsDarkMode] = useState(() => {
```



```
    return document.documentElement.classList.contains('dark') ||
    localStorage.getItem('theme_preference') !== 'light'
  })

  // Hook to persist theme changes
  useEffect(() => {
    if (isDarkMode) {
      document.documentElement.classList.add('dark')
      localStorage.setItem('theme_preference', 'dark')
    } else {
      document.documentElement.classList.remove('dark')
      localStorage.setItem('theme_preference', 'light')
    }
  }, [isDarkMode])

  const toggleTheme = () => setIsDarkMode(prev => !prev)

  // Prevent sidebar lock on resize
  useEffect(() => {
    const handleResize = () => {
      if (window.innerWidth >= 1024) setIsMobileMenuOpen(false)
    }
    window.addEventListener('resize', handleResize)
    return () => window.removeEventListener('resize', handleResize)
  }, [])
```

```

const { data: alertsData } = useQuery({
  queryKey: ['alerts-count'],
  queryFn: () => alertsApi.list({ resolved: 'false', limit: 99 }),
  refetchInterval: 60000,
  select: d => d.data.alerts.length,
  enabled: !!user,
})

const alertCount = alertsData || 0

return (
  <div className="flex h-screen bg-surface overflow-hidden">
    /* Mobile backdrop */
    {isMobileMenuOpen && (
      <div className="fixed inset-0 bg-black/40 z-40 lg:hidden backdrop-blur-sm
transition-opacity" onClick={closeMenu} />
    )}

    /* Sidebar */

    <aside className={`fixed inset-y-0 left-0 w-64 bg-surface-container-low flex
flex-col border-r border-outline-variant/30 z-50 transform transition-transform
duration-300 ease-in-out lg:translate-x-0 ${isMobileMenuOpen ? 'translate-x-0' : '-
translate-x-full'}`>

      /* Logo */

      <div className="p-6 flex items-center gap-3">

```

```

<div className="w-10 h-10 rounded-xl bg-primary-container flex items-
center justify-center text-white flex-shrink-0">

  <span className="material-symbols-outlined text-xl">inventory_2</span>

</div>

<div>

  <h1 className="text-base font-bold text-on-surface tracking-tight leading-
tight">StockGrid</h1>

  <p className="text-[10px] uppercase tracking-widest text-on-surface-
variant font-bold">Inventory Intelligence</p>

</div>

</div>

{/* Nav */}

<nav className="flex-1 px-4 mt-2 space-y-1">

  {NAV_ITEMS.map(item => (

    <NavLink

      key={item.to}

      to={item.to}

      end={item.exact}

      onClick={closeMenu}

      className={{ isActive }} =>

        `flex items-center gap-3 px-3 py-2.5 rounded-xl text-sm font-medium
transition-all ${

          isActive

            ? 'text-primary font-semibold bg-surface-container shadow-sm'

            : 'text-on-surface-variant hover:text-primary hover:bg-surface-
container/60'

```

```

    }`
  }
  >
    <span className="material-symbols-outlined text-[20px]">{item.icon}</span>
    <span>{item.label}</span>
  </NavLink>
)}}
</nav>

{/* New Entry CTA */}
<div className="px-4 pb-2">
  <button
    onClick={() => navigate('/products')}
    className="w-full bg-gradient-to-br from-primary-container to-primary text-white py-3 rounded-xl text-sm font-semibold editorial-shadow transition-transform active:scale-95 hover:opacity-90"
  >
    + New Entry
  </button>
</div>

{/* Bottom nav */}
<div className="px-4 py-4 border-t border-slate-200/50 space-y-1">
  <NavLink
    to="/settings"

```

```

onClick={closeMenu}

className={({ isActive }) =>
  `flex items-center gap-3 px-3 py-2.5 rounded-xl text-sm font-medium
  transition-colors ${
    isActive ? 'text-primary bg-surface-container' : 'text-on-surface-variant
    hover:text-primary hover:bg-surface-container/60'
  }`
}
>
<span className="material-symbols-outlined text-[20px]">settings</span>
<span>Settings</span>
</NavLink>
<button
  onClick={logout}
  className="w-full flex items-center gap-3 px-3 py-2.5 rounded-xl text-sm
  font-medium text-on-surface-variant hover:text-tertiary hover:bg-error-
  container/50 transition-colors"
>
  <span className="material-symbols-outlined text-[20px]">logout</span>
  <span>Sign Out</span>
</button>
</div>
</aside>

{ /* Main area */ }

<div className="lg:ml-64 flex-1 flex flex-col min-h-screen overflow-hidden">

```

```
{/* Top bar */}
```

```
<header className="fixed top-0 right-0 w-full lg:w-[calc(100%-16rem)] h-16 z-30 bg-surface-container-lowest/90 backdrop-blur-xl border-b border-outline-variant/30 flex items-center justify-between px-4 lg:px-8">
```

```
<div className="flex items-center flex-1 max-w-xl gap-2">
```

```
<button onClick={() => setIsMobileMenuOpen(true)} className="lg:hidden p-2 text-on-surface-variant hover:text-primary rounded-lg">
```

```
<span className="material-symbols-outlined text-[24px]">menu</span>
```

```
</button>
```

```
<div className="relative w-full hidden sm:block">
```

```
<span className="material-symbols-outlined absolute left-3 top-1/2 -translate-y-1/2 text-on-surface-variant text-[18px]">search</span>
```

```
<input
```

```
type="text"
```

```
placeholder="Search inventory or AI insights..."
```

```
className="w-full bg-surface-container-low border-none rounded-xl pl-10 pr-4 py-2 text-sm focus:ring-1 focus:ring-indigo-500 transition-all outline-none"
```

```
/>
```

```
</div>
```

```
</div>
```

```
<div className="flex items-center gap-4">
```

```
<button
```

```
onClick={toggleTheme}
```

```
className="p-2 -mr-2 text-on-surface-variant hover:text-primary hover:bg-surface-container rounded-full transition-colors flex items-center justify-center pointer-events-auto"
```

```
aria-label="Toggle Theme"
```

```

>

<span className="material-symbols-outlined">

  {isDarkMode ? 'light_mode' : 'dark_mode'}

</span>

</button>


<NavLink

  to="/transactions"

  className="relative p-2 flex items-center justify-center text-on-surface-
variant hover:text-primary hover:bg-surface-container rounded-full transition-
colors"

  >

    <span className="material-symbols-outlined">notifications</span>

    {alertCount > 0 && (

      <span className="absolute -top-1 -right-1 w-4 h-4 bg-tertiary rounded-full
text-white text-[10px] flex items-center justify-center font-bold">

        {alertCount > 9 ? '9+' : alertCount}

      </span>

    )}

  </NavLink>

  <NavLink to="/ai-insights" className="p-2 flex items-center justify-center
text-on-surface-variant hover:text-primary hover:bg-surface-container rounded-full
transition-colors">

    <span className="material-symbols-outlined">chat_bubble</span>

  </NavLink>

  <div className="h-6 w-px bg-outline-variant/50 mx-1" />

  <div className="flex items-center gap-2.5">

```



```
<div className="w-8 h-8 rounded-full bg-primary-container flex items-center justify-center text-white text-sm font-bold flex-shrink-0">
```

```
  {user?.name?.charAt(0)?.toUpperCase() || 'U'}
```

```
</div>
```

```
<div className="hidden sm:block">
```

```
  <p className="text-xs font-semibold text-on-surface leading-tight">{user?.name}</p>
```

```
  <p className="text-[10px] text-on-surface-variant capitalize">{user?.role}</p>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
</header>
```

```
{/* Page content */}
```

```
<main className="flex-1 pt-16 overflow-y-auto scrollbar-thin">
```

```
  <div className="page-enter">
```

```
    <Outlet />
```

```
  </div>
```

```
</main>
```

```
</div>
```

```
</div>
```

```
)
```

```
}
```


BulkTransactionModal.jsx

```
import { useState } from 'react'

import toast from 'react-hot-toast'

import { transactionsApi } from '../services/api'

export default function BulkTransactionModal({ products, onClose, onSave }) {

  const [form, setForm] = useState({
    type: 'IN', quantity: '', reason: '', reference_no: '', notes: ''
  })

  const [isProcessing, setIsProcessing] = useState(false)

  const handleBulkSubmit = async (e) => {
    e.preventDefault()

    if (!form.quantity || parseInt(form.quantity) < 1) return toast.error('Enter a valid quantity')

    setIsProcessing(true)

    let successCount = 0

    let failCount = 0

    const promises = products.map(async (p) => {

      try {

        await transactionsApi.create({ ...form, product_id: p.id, quantity:
parseInt(form.quantity) })

        successCount++
```

```
    } catch (err) {  
      failCount++  
    }  
  })
```

```
  await Promise.all(promises)  
  setIsProcessing(false)
```

```
  if (failCount === 0) {  
    toast.success(`Successfully updated stock for ${successCount} products!`)  
  } else {  
    toast.error(`${successCount} updated, ${failCount} failed.`)  
  }
```

```
  onSave()  
}
```

```
const IN_REASONS = ['Purchase Order', 'Restock', 'Return from Customer',  
'Transfer In', 'Found in Count', 'Other']
```

```
const OUT_REASONS = ['Sale', 'Transfer Out', 'Damaged/Write-off', 'Sample',  
'Theft/Loss', 'Other']
```

```
return (  
  <div className="fixed inset-0 z-50 flex items-center justify-center">  
    <div className="absolute inset-0 bg-black/30 backdrop-blur-sm"  
      onClick={onClose} />
```

```
<div className="relative bg-white rounded-2xl shadow-editorial-lg w-full max-w-md mx-4">
```

```
<div className="px-6 pt-6 pb-4 border-b border-surface-container-low">
```

```
<div className="flex items-center justify-between mb-1">
```

```
<h2 className="text-xl font-bold text-on-surface">Bulk Stock Adjustment</h2>
```

```
<button onClick={onClose} className="p-2 rounded-lg hover:bg-surface-container-low transition-colors">
```

```
<span className="material-symbols-outlined text-on-surface-variant">close</span>
```

```
</button>
```

```
</div>
```

```
<p className="text-sm text-on-surface-variant">
```

```
Applying the same stock movement to <span className="font-bold text-on-surface">{products.length} selected products</span>.
```

```
</p>
```

```
</div>
```

```
<form onSubmit={handleBulkSubmit} className="p-6 space-y-4">
```

```
<div>
```

```
<label className="block text-xs font-bold text-on-surface-variant uppercase tracking-wider mb-2">Movement Type *</label>
```

```
<div className="grid grid-cols-2 gap-2">
```

```
{['IN', 'OUT'].map(type => {
```

```
<button
```

```
key={type} type="button"
```

```
onClick={() => setForm({ ...form, type, reason: '' })}
```

```

        className={py-3 rounded-xl text-sm font-bold transition-all flex items-
center justify-center gap-2 ${
    form.type === type
    ? type === 'IN'
    ? 'bg-secondary-container/30 text-secondary ring-2 ring-secondary'
    : 'bg-primary-fixed/40 text-primary ring-2 ring-primary'
    : 'bg-surface-container-low text-on-surface-variant hover:bg-surface-
container'
}}
>
    <span className="material-symbols-outlined text-[18px]">{type === 'IN'
? 'add_circle' : 'remove_circle'}</span>
    Stock {type === 'IN' ? 'In' : 'Out'}
</button>
)}}
</div>
</div>

<div>
    <label className="block text-xs font-bold text-on-surface-variant uppercase
tracking-wider mb-1.5">Quantity (Per Product) *</label>
    <input
        required type="number" min="1" value={form.quantity}
        onChange={e => setForm({ ...form, quantity: e.target.value })}
        className="w-full bg-surface-container-low border-none rounded-xl px-4
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none text-center text-2xl
font-bold"

```

```
placeholder="0"
```

```
/>
```

```
</div>
```

```
<div>
```

```
<label className="block text-xs font-bold text-on-surface-variant uppercase  
tracking-wider mb-1.5">Reason *</label>
```

```
<select required value={form.reason} onChange={e => setForm({ ...form,  
reason: e.target.value })}
```

```
className="w-full bg-surface-container-low border-none rounded-xl px-4  
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none">
```

```
<option value="">Select reason...</option>
```

```
{{(form.type === 'IN' ? IN_REASONS : OUT_REASONS).map(r =>
```

```
<option key={r} value={r}>{r}</option>}}
```

```
</select>
```

```
</div>
```

```
<div>
```

```
<label className="block text-xs font-bold text-on-surface-variant uppercase  
tracking-wider mb-1.5">Reference No. (optional)</label>
```

```
<input value={form.reference_no} onChange={e => setForm({ ...form,  
reference_no: e.target.value })}
```

```
className="w-full bg-surface-container-low border-none rounded-xl px-4  
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none"
```

```
placeholder="PO#, Invoice#, etc." />
```

```
</div>
```

```

<div>

  <label className="block text-xs font-bold text-on-surface-variant uppercase
tracking-wider mb-1.5">Notes (optional)</label>

  <textarea value={form.notes} onChange={e => setForm({ ...form, notes:
e.target.value })}

    rows={2} className="w-full bg-surface-container-low border-none
rounded-xl px-4 py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none
resize-none" />

</div>

<div className="flex gap-3 pt-1">

  <button type="button" onClick={onClose} disabled={isProcessing}

    className="flex-1 py-3 bg-surface-container-highest rounded-xl text-sm
font-semibold text-on-surface hover:bg-surface-container-high transition-colors
disabled:opacity-50">

    Cancel

  </button>

  <button type="submit" disabled={isProcessing}

    className={`flex-1 py-3 text-white rounded-xl text-sm font-semibold
editorial-shadow active:scale-95 transition-all disabled:opacity-60 ${

      form.type === 'IN'

        ? 'bg-secondary hover:opacity-90'

        : 'bg-gradient-to-br from-primary-container to-primary hover:opacity-90'

      }`}>

    {isProcessing ? 'Processing Batch...' : `Apply Stock ${form.type}`}

  </button>

</div>

```

```
</form>  
</div>  
</div>  
)  
}
```



ImportModal.jsx

```
import { useState, useRef } from 'react'
import { useMutation } from '@tanstack/react-query'
import toast from 'react-hot-toast'
import { importExportApi, aiApi } from '../services/api'
import { formatCurrencyINR } from '../utils/currency'
```

```
const STEPS = ['upload', 'mapping', 'preview', 'done']
```

```
const TARGET_FIELDS = [
  { key: 'name', label: 'Product Name', required: true },
  { key: 'sku', label: 'SKU' },
  { key: 'price', label: 'Price', required: true },
  { key: 'category', label: 'Category' },
  { key: 'supplier', label: 'Supplier' },
  { key: 'min_stock', label: 'Min Stock' },
  { key: 'initial_stock', label: 'Initial Stock' },
  { key: 'description', label: 'Description' },
  { key: 'unit', label: 'Unit' },
]
```

```
function getErrorMessage(error) {
  if (!error) return 'Import failed for this row'
  if (error.message) return error.message
}
```



```
    if (Array.isArray(error.errors) && error.errors.length > 0) return  
    error.errors.join(', ')  
  
    return 'Import failed for this row'  
}
```

```
export default function ImportModal({ onClose, onImported }) {  
  const [step, setStep] = useState('upload')  
  const [file, setFile] = useState(null)  
  const [mapping, setMapping] = useState({})  
  const [headers, setHeaders] = useState([])  
  const [preview, setPreview] = useState(null)  
  const [errors, setErrors] = useState([])  
  const [result, setResult] = useState(null)  
  const [previewStats, setPreviewStats] = useState({ valid_count: 0, error_count: 0 })  
  const [dragging, setDragging] = useState(false)  
  const fileRef = useRef()  
  
  const previewMutation = useMutation({  
    mutationFn: async () => {  
      const fd = new FormData()  
      fd.append('file', file)  
      fd.append('mapping', JSON.stringify(mapping))  
      fd.append('dry_run', 'true')  
      return importExportApi.importPreview(fd)  
    },
```

```

onSuccess: (res) => {
  setPreview(res.data.preview)
  setErrors(res.data.errors || [])
  setPreviewStats({
    valid_count: res.data.valid_count || 0,
    error_count: res.data.error_count || 0,
  })
  setStep('preview')
},
onError: (error) => {
  const message = error?.response?.data?.error || error?.message || 'Preview failed'
  toast.error(message)
},
})

const importMutation = useMutation({
  mutationFn: async () => {
    const fd = new FormData()
    fd.append('file', file)
    fd.append('mapping', JSON.stringify(mapping))
    return importExportApi.import(fd)
  },
  onSuccess: (res) => {
    setResult(res.data)
    setStep('done')
  }
})

```

```

    },
    onError: (error) => {
      const message = error?.response?.data?.error || error?.message || 'Import failed'
      toast.error(message)
    },
  })
}

```

```

const aiMapMutation = useMutation({
  mutationFn: async (hdrs) => aiApi.columnMap(hdrs, []),
  onSuccess: (res) => {
    setMapping(res.data.mapping || {})
    toast.success('AI mapped your columns!')
  },
  onError: () => toast('Could not auto-map. Please map manually.', { icon: 'i' }),
})

```

```

const handleFile = async (f) => {
  if (!f) return
  const ext = f.name.split('.').pop().toLowerCase()
  if (!['csv', 'xlsx', 'xls'].includes(ext)) {
    return toast.error('Only CSV and Excel files supported')
  }
}

```

```

setFile(f)
setResult(null)

```

```
setPreview(null)

setErrors([])

setPreviewStats({ valid_count: 0, error_count: 0 })

if (ext === 'csv') {
  const text = await f.text()
  const firstLine = text.split('\n')[0]
  const hdrs = firstLine.split(',').map((h) => h.replace(/"/g,
    '').trim()).filter(Boolean)
  setHeaders(hdrs)
  aiMapMutation.mutate(hdrs)
} else {
  setHeaders([])
}
setStep('mapping')
}

const handleDrop = (e) => {
  e.preventDefault()
  setDragging(false)
  const f = e.dataTransfer.files[0]
  if (f) handleFile(f)
}

const previewValidCount = previewStats.valid_count || 0
```

```

const finalErrors = result?.errors || []

const supplierWarnings = result?.supplier_warnings || []

const importFailedCompletely = result && result.imported_count === 0 &&
result.error_count > 0

const hasPartialFailure = result && result.imported_count > 0 &&
result.error_count > 0

return (
  <div className="fixed inset-0 z-50 flex items-center justify-center">
    <div className="absolute inset-0 bg-black/30 backdrop-blur-sm"
onClick={onClose} />
    <div className="relative bg-white rounded-2xl shadow-editorial-lg w-full max-
w-2xl mx-4 max-h-[90vh] overflow-hidden flex flex-col">
      <div className="px-6 pt-6 pb-4 border-b border-surface-container-low flex
items-center justify-between flex-shrink-0">
        <div>
          <h2 className="text-xl font-bold text-on-surface">Import Products</h2>
          <p className="text-xs text-on-surface-variant mt-0.5">CSV or Excel - AI-
powered column detection</p>
        </div>
        <button onClick={onClose} className="p-2 rounded-lg hover:bg-surface-
container-low transition-colors">
          <span className="material-symbols-outlined text-on-surface-
variant">close</span>
        </button>
      </div>

```

```

<div className="flex px-6 py-3 gap-2 flex-shrink-0 border-b border-surface-
container-low">

  {STEPS.map((s, i) => (

    <div key={s} className="flex items-center gap-2">

      <div

        className={`w-6 h-6 rounded-full flex items-center justify-center text-xs
font-bold transition-colors ${

          STEPS.indexOf(step) >= i ? 'bg-primary text-white' : 'bg-surface-container
text-on-surface-variant'

        }}

      >

        {i + 1}

      </div>

      <span

        className={`text-xs font-medium capitalize hidden sm:block ${

          step === s ? 'text-primary font-bold' : 'text-on-surface-variant'

        }}

      >

        {s}

      </span>

      {i < STEPS.length - 1 && <div className="w-8 h-px bg-surface-container-
high" />}

    </div>

  )})

</div>

```

```

<div className="overflow-y-auto flex-1 p-6 scrollbar-thin">
  {step === 'upload' && (
    <div
      onDragOver={(e) => { e.preventDefault(); setDragging(true) }}
      onDragLeave={() => setDragging(false)}
      onDrop={handleDrop}
      className={`border-2 border-dashed rounded-2xl p-12 text-center
        transition-colors cursor-pointer ${
          dragging ? 'border-primary bg-primary-fixed/10' : 'border-outline-variant
        hover:border-primary/50 hover:bg-surface-container-low'
        }}
      onClick={() => fileRef.current?.click()}
    >
      <span className="material-symbols-outlined text-5xl text-on-surface-
        variant mb-4 block">upload_file</span>
      <p className="text-base font-bold text-on-surface mb-1">Drop your file
        here</p>
      <p className="text-sm text-on-surface-variant mb-4">or click to
        browse</p>
      <p className="text-xs text-outline">Supports: .csv, .xlsx, .xls - Max
        10MB</p>
      <input ref={fileRef} type="file" accept=".csv,.xlsx,.xls" className="hidden"
        onChange={(e) => handleFile(e.target.files[0])} />
    </div>
  )}

  {step === 'mapping' && (

```



```

<div>
  <div className="flex items-center justify-between mb-4">
    <div>
      <p className="text-sm font-bold text-on-surface">Column Mapping</p>
      <p className="text-xs text-on-surface-variant mt-0.5">
        File: <span className="font-mono text-primary">{file?.name}</span>
        {aiMapMutation.isPending && <span className="ml-2 text-primary">AI
detecting columns...</span>}
      </p>
    </div>
    <button
      onClick={() => { setStep('upload'); setFile(null); setHeaders([]) }}
      className="text-xs text-on-surface-variant hover:text-on-surface"
    >
      Change file
    </button>
  </div>

  {headers.length === 0 ? (
    <div className="bg-surface-container-low rounded-xl p-4 text-sm text-on-
surface-variant text-center">
      Excel file detected. Mapping will be applied on import.
    </div>
  ) : (
    <div className="space-y-3">

```



```

{TARGET_FIELDS.map((field) => (
  <div key={field.key} className="flex items-center gap-4">
    <div className="w-36 flex-shrink-0">
      <span className="text-xs font-bold text-on-surface">
        {field.label}
        {field.required && <span className="text-tertiary ml-0.5">*</span>}
      </span>
    </div>
    <div className="flex-1">
      <select
        value={mapping[field.key] || ""}
        onChange={(e) => setMapping({ ...mapping, [field.key]: e.target.value ||
undefined })}
        className="w-full bg-surface-container-low border-none rounded-xl
px-3 py-2 text-sm focus:ring-2 focus:ring-primary/30 outline-none"
      >
        <option value="">- Skip field -</option>
        {headers.map((h) => <option key={h} value={h}>{h}</option>)}
      </select>
    </div>
    {mapping[field.key] && (
      <span className="material-symbols-outlined text-secondary text-
[18px] flex-shrink-0">check_circle</span>
    )}
  </div>
)}}

```

```

    </div>

  }}

  <div className="flex gap-3 mt-6">

    <button

      onClick={() => previewMutation.mutate()}

      disabled={previewMutation.isPending || (!mapping.name &&
headers.length > 0)}

      className="flex-1 py-3 bg-gradient-to-br from-primary-container to-
primary text-white rounded-xl text-sm font-semibold editorial-shadow
hover:opacity-90 active:scale-95 transition-all disabled:opacity-50"

    >

      {previewMutation.isPending ? 'Analyzing...' : 'Preview Import'}

    </button>
  </div>
</div>
  }}

  {step === 'preview' && preview && (
    <div>

      <div className="flex items-center gap-4 mb-4">

        <div className="flex-1 bg-secondary-container/20 rounded-xl p-3 text-
center">

          <p className="text-2xl font-bold text-
secondary">{previewValidCount}</p>

          <p className="text-xs text-on-surface-variant font-medium">Valid
rows</p>

```

```

</div>

<div className="flex-1 bg-tertiary-fixed/30 rounded-xl p-3 text-center">

  <p className="text-2xl font-bold text-tertiary">{previewStats.error_count
|| 0}</p>

  <p className="text-xs text-on-surface-variant font-medium">Error
rows</p>

</div>

</div>

<div className="border border-surface-container-high rounded-xl
overflow-hidden mb-4">

  <div className="overflow-x-auto">

    <table className="w-full text-xs">

      <thead className="bg-surface-container-low">

        <tr>

          {[ 'Name', 'SKU', 'Price', 'Category', 'Stock'].map((h) => (

            <th key={h} className="px-3 py-2.5 text-left font-bold text-on-
surface-variant uppercase tracking-wider">{h}</th>

          )))

        </tr>

      </thead>

      <tbody className="divide-y divide-surface-container-low">

        {preview.slice(0, 8).map((row, i) => (

          <tr key={` ${row.row_number} || i}-${row.sku || row.name || i}`
className="hover:bg-surface-container-low/50">

            <td className="px-3 py-2.5 font-medium text-on-surface truncate
max-w-[150px]">{row.name}</td>

```

```

        <td className="px-3 py-2.5 font-mono text-on-surface-variant">{row.sku}</td>

        <td className="px-3 py-2.5 text-on-surface">{formatCurrencyINR(row.price)}</td>

        <td className="px-3 py-2.5 text-on-surface-variant">{row.category}</td>

        <td className="px-3 py-2.5 text-on-surface">{row.initial_stock}</td>
    </tr>
  )}
</tbody>
</table>
</div>
{preview.length > 8 && (
  <p className="text-xs text-on-surface-variant text-center py-2 border-t border-surface-container-low">
    +{preview.length - 8} more rows
  </p>
)}
</div>

{errors.length > 0 && (
  <div className="bg-tertiary-fixed/20 rounded-xl p-4 mb-4">
    <p className="text-xs font-bold text-tertiary mb-2">Warning:
    {errors.length} rows will be skipped</p>
    {errors.slice(0, 3).map((error, index) => (
      <p key={` ${error.row || index} - ${index}`} className="text-xs text-on-surface-variant">

```

```

      Row {error.row}: {getErrorMessage(error)}
    </p>
  )}
</div>
})

<div className="flex gap-3">
  <button
    onClick={() => setStep('mapping')}
    className="px-6 py-3 bg-surface-container-highest rounded-xl text-sm
font-semibold text-on-surface hover:bg-surface-container-high transition-colors"
  >
    Back
  </button>
  <button
    onClick={() => importMutation.mutate()}
    disabled={importMutation.isPending}
    className="flex-1 py-3 bg-gradient-to-br from-primary-container to-
primary text-white rounded-xl text-sm font-semibold editorial-shadow
hover:opacity-90 active:scale-95 transition-all disabled:opacity-50"
  >
    {importMutation.isPending ? 'Importing...' : `Import ${previewValidCount}
Products`}
  </button>
</div>
</div>

```

```
}}
```

```
{step === 'done' && result && (  
  <div className="py-6">  
    <div className={`w-20 h-20 rounded-full flex items-center justify-center  
mx-auto mb-6 ${  
      importFailedCompletely ? 'bg-tertiary-fixed/30' : 'bg-secondary-  
container/30'  
    }}`>  
      <span  
        className={`material-symbols-outlined text-4xl  
${importFailedCompletely ? 'text-tertiary' : 'text-secondary'}}`  
        style={{ fontVariationSettings: '"FILL' 1" }}  
      >  
        {importFailedCompletely ? 'error' : 'check_circle'}  
      </span>  
    </div>  
  
    <h3 className="text-2xl font-bold text-on-surface mb-2 text-center">  
      {importFailedCompletely ? 'Import Failed' : hasPartialFailure ? 'Import  
Finished with Issues' : 'Import Complete!'}  
    </h3>  
  
    <p className="text-on-surface-variant mb-6 text-center">  
      <span className={`font-bold text-xl ${importFailedCompletely ? 'text-  
tertiary' : 'text-secondary'}}`>  
        {result.imported_count}  
      </span>{ ' '}
```

```

products imported successfully
{result.error_count > 0 && (
  <span className="text-tertiary"> - {result.error_count} skipped</span>
)}
</p>

```

```

{(result.import_summary || previewValidCount > 0) && (
  <div className="bg-surface-container-low rounded-xl p-4 mb-4 text-sm
text-on-surface-variant">
    <p>{result.import_summary || `Preview showed ${previewValidCount}
valid row(s) before final import.`}</p>
    {previewValidCount > 0 && (
      <p className="mt-1 text-xs">
        Preview valid rows: <span className="font-semibold text-on-
surface">{previewValidCount}</span>
      </p>
    )}
  </div>
)}

```

```

{supplierWarnings.length > 0 && (
  <div className="bg-amber-100/70 rounded-xl p-4 mb-4">
    <p className="text-xs font-bold text-amber-700 mb-2">Supplier
warnings</p>
    {supplierWarnings.slice(0, 3).map((warning, index) => (
      <p key={` ${warning}-${index}`} className="text-xs text-amber-
800">{warning}</p>

```



```

    })
  </div>
})

{finalErrors.length > 0 && (
  <div className="bg-tertiary-fixed/20 rounded-xl p-4 mb-6">
    <p className="text-xs font-bold text-tertiary mb-2">Import issues</p>
    <div className="space-y-2">
      {finalErrors.slice(0, 5).map((error, index) => (
        <div key={` ${error.row || index} - ${error.final_sku || error.original_sku ||
index}`} className="text-xs text-on-surface-variant">
          <span className="font-semibold text-on-surface">
            Row {error.row || '-'}
            {error.product_name ? ` - ${error.product_name}` : ''}
          </span>
          <span>: {getErrorMessage(error)}</span>
          {error.final_sku && (
            <span className="block mt-0.5 font-mono text-[11px] text-on-
surface-variant">SKU: {error.final_sku}</span>
          )}
        </div>
      )}
    </div>
  )}
</div>
</div>
})

```



```

<div className="flex gap-3">

  <button

    onClick={importFailedCompletely ? () => setStep('mapping') : onClose}

    className="flex-1 py-3 bg-surface-container-highest rounded-xl text-sm
font-semibold text-on-surface hover:bg-surface-container-high transition-colors"

    >

      {importFailedCompletely ? 'Back to Mapping' : 'Close'}

    </button>

    {result.imported_count > 0 ? (

      <button

        onClick={onImported}

        className="flex-1 py-3 bg-gradient-to-br from-primary-container to-
primary text-white rounded-xl text-sm font-semibold editorial-shadow
hover:opacity-90 transition-all"

        >

          View Products

        </button>

      ) : (

        <button

          onClick={() => setStep('mapping')}

          className="flex-1 py-3 bg-gradient-to-br from-primary-container to-
primary text-white rounded-xl text-sm font-semibold editorial-shadow
hover:opacity-90 transition-all"

          >

            Try Again

          </button>

```

```

    }}
  </div>
</div>
}}
</div>
</div>
</div>
)
}

```



ProductModal.jsx

```
import { useState, useEffect } from 'react'

import { useMutation } from '@tanstack/react-query'

import toast from 'react-hot-toast'

import { useAuthStore } from '../store/authStore'

import { productsApi, aiApi } from '../services/api'

const INITIAL_FORM = {
  name: "", sku: "", price: "", cost_price: "", category: "", supplier_id: "",
  min_stock: '5', description: "", unit: 'units', initial_stock: '0'
}

export default function ProductModal({ product, suppliers, onClose, onSave }) {
  const { user } = useAuthStore()
  const canEditCost = user?.role === 'admin' || user?.role === 'manager'

  const isEdit = !!product
  const [form, setForm] = useState(isEdit ? {
    name: product.name || "",
    sku: product.sku || "",
    price: product.price?.toString() || "",
    cost_price: product.cost_price?.toString() || "",
    category: product.category || "",
    supplier_id: product.supplier_id || "",
    min_stock: product.min_stock?.toString() || '5',
  })
```

```

description: product.description || '',
unit: product.unit || 'units',
initial_stock: '0',
} : INITIAL_FORM)

const [generatingSKU, setGeneratingSKU] = useState(false)

const mutation = useMutation({
  mutationFn: () => isEdit
    ? productsApi.update(product.id, form)
    : productsApi.create(form),
  onSuccess: () => {
    toast.success(isEdit ? 'Product updated!' : 'Product created!')
    onSave()
  },
  onError: (err) => toast.error(err.response?.data?.error || 'Save failed'),
})

const handleGenerateSKU = async () => {
  if (!form.name) return toast.error('Enter product name first')
  setGeneratingSKU(true)
  try {
    const res = await aiApi.generateSKU(form.name, form.category)
    setForm(f => ({ ...f, sku: res.data.sku }))
    toast.success('SKU generated by AI!')
  } catch {

```

```

    toast.error('AI SKU generation failed')
  } finally {
    setGeneratingSKU(false)
  }
}

const handleSubmit = (e) => {
  e.preventDefault()
  mutation.mutate()
}

return (
  <div className="fixed inset-0 z-50 flex items-center justify-center">
    <div className="absolute inset-0 bg-black/30 backdrop-blur-sm"
      onClick={onClose} />
    <div className="relative bg-white rounded-2xl shadow-editorial-lg w-full max-
      w-lg mx-4 max-h-[90vh] overflow-y-auto scrollbar-thin">
      <div className="sticky top-0 bg-white rounded-t-2xl px-6 pt-6 pb-4 border-b
        border-surface-container-low">
        <div className="flex items-center justify-between">
          <h2 className="text-xl font-bold text-on-surface">{isEdit ? 'Edit Product' :
            'Add New Product'}</h2>
          <button onClick={onClose} className="p-2 rounded-lg hover:bg-surface-
            container-low transition-colors">
            <span className="material-symbols-outlined text-on-surface-
              variant">close</span>
          </button>
        </div>
      </div>
    </div>
  </div>
)

```

```
</div>
```

```
</div>
```

```
<form onSubmit={handleSubmit} className="p-6 space-y-4">
```

```
<div className="grid grid-cols-2 gap-4">
```

```
<div className="col-span-2">
```

```
<label className="block text-xs font-bold text-on-surface-variant uppercase tracking-wider mb-1.5">Product Name *</label>
```

```
<input required value={form.name} onChange={e => setForm({ ...form, name: e.target.value })}>
```

```
className="w-full bg-surface-container-low border-none rounded-xl px-4 py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none"
```

```
placeholder="e.g. Wireless Headphones Pro" />
```

```
</div>
```

```
<div>
```

```
<label className="block text-xs font-bold text-on-surface-variant uppercase tracking-wider mb-1.5">SKU</label>
```

```
<div className="flex gap-2">
```

```
<input value={form.sku} onChange={e => setForm({ ...form, sku: e.target.value })>
```

```
className="flex-1 bg-surface-container-low border-none rounded-xl px-4 py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none font-mono"
```

```
placeholder="AUTO-GENERATED" />
```

```
<button type="button" onClick={handleGenerateSKU} disabled={generatingSKU}>
```

```
className="px-3 py-2.5 bg-primary-fixed/40 text-primary rounded-xl
text-xs font-bold hover:bg-primary-fixed/60 transition-colors disabled:opacity-50
flex-shrink-0"
```

```
title="Generate SKU with AI">
```

```
{generatingSKU ? '...' : <span className="material-symbols-outlined text-
[16px]">psychology</span>}
```

```
</button>
```

```
</div>
```

```
</div>
```

```
<div>
```

```
<label className="block text-xs font-bold text-on-surface-variant uppercase
tracking-wider mb-1.5">Unit Price *</label>
```

```
<div className="relative">
```

```
<span className="absolute left-3 top-1/2 -translate-y-1/2 text-on-surface-
variant text-sm font-semibold">₹</span>
```

```
<input required type="number" min="0" step="0.01" value={form.price}
```

```
onChange={e => setForm({ ...form, price: e.target.value })}
```

```
className="w-full bg-surface-container-low border-none rounded-xl pl-7
pr-4 py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none"
```

```
placeholder="0.00" />
```

```
</div>
```

```
</div>
```

```
<div>
```

```
<label className="block text-xs font-bold text-on-surface-variant uppercase
tracking-wider mb-1.5">Cost Price</label>
```



```

<div className="relative">
  <span className="absolute left-3 top-1/2 -translate-y-1/2 text-on-surface-variant text-sm font-semibold">₹</span>

  <input type="number" min="0" step="0.01" value={form.cost_price}

    disabled={!canEditCost}

    title={!canEditCost ? 'Only managers or admins can edit cost price' : ''}

    onChange={e => setForm({ ...form, cost_price: e.target.value })}

    className="w-full bg-surface-container-low border-none rounded-xl pl-7
pr-4 py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none
disabled:opacity-50"

    placeholder="0.00" />
</div>
</div>

<div>
  <label className="block text-xs font-bold text-on-surface-variant uppercase
tracking-wider mb-1.5">Category *</label>

  <input required value={form.category} onChange={e => setForm({ ...form,
category: e.target.value })}

    className="w-full bg-surface-container-low border-none rounded-xl px-4
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none"

    placeholder="Electronics, Apparel..." list="cat-list" />

  <datalist id="cat-list">

    {[ 'Electronics', 'Apparel', 'Furniture', 'Accessories', 'Food & Beverage',
'Tools', 'Stationery' ].map(c =>

      <option key={c} value={c} />)}

  </datalist>

```



```
</div>
```

```
<div>
```

```
  <label className="block text-xs font-bold text-on-surface-variant uppercase  
tracking-wider mb-1.5">Supplier</label>
```

```
  <select value={form.supplier_id} onChange={e => setForm({ ...form,  
supplier_id: e.target.value })}
```

```
    className="w-full bg-surface-container-low border-none rounded-xl px-4  
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none">
```

```
    <option value="">No Supplier</option>
```

```
    {suppliers.map(s => <option key={s.id} value={s.id}>{s.name}</option>)}
```

```
  </select>
```

```
</div>
```

```
<div>
```

```
  <label className="block text-xs font-bold text-on-surface-variant uppercase  
tracking-wider mb-1.5">Min Stock Alert *</label>
```

```
  <input required type="number" min="0" value={form.min_stock}  
    onChange={e => setForm({ ...form, min_stock: e.target.value })}
```

```
    className="w-full bg-surface-container-low border-none rounded-xl px-4  
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none" />
```

```
</div>
```

```
<div>
```

```
  <label className="block text-xs font-bold text-on-surface-variant uppercase  
tracking-wider mb-1.5">Unit</label>
```

```

    <select value={form.unit} onChange={e => setForm({ ...form, unit:
e.target.value })}

      className="w-full bg-surface-container-low border-none rounded-xl px-4
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none">

        {[ 'units', 'kg', 'g', 'liter', 'box', 'pack', 'pair', 'set' ].map(u =>

          <option key={u} value={u}>{u}</option>)}

      </select>
    </div>

    {!isEdit && (
      <div>

        <label className="block text-xs font-bold text-on-surface-variant
uppercase tracking-wider mb-1.5">Initial Stock</label>

        <input type="number" min="0" value={form.initial_stock}

          onChange={e => setForm({ ...form, initial_stock: e.target.value })}

          className="w-full bg-surface-container-low border-none rounded-xl px-4
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none" />

        </div>
      )}

    <div className="col-span-2">

      <label className="block text-xs font-bold text-on-surface-variant uppercase
tracking-wider mb-1.5">Description</label>

      <textarea value={form.description} onChange={e => setForm({ ...form,
description: e.target.value })}

        rows={2} placeholder="Optional product description..."

```

```
className="w-full bg-surface-container-low border-none rounded-xl px-4  
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none resize-none" />
```

```
</div>
```

```
</div>
```

```
<div className="flex gap-3 pt-2">
```

```
<button type="button" onClick={onClose}
```

```
className="flex-1 py-3 bg-surface-container-highest rounded-xl text-sm  
font-semibold text-on-surface hover:bg-surface-container-high transition-colors">
```

```
Cancel
```

```
</button>
```

```
<button type="submit" disabled={mutation.isPending}
```

```
className="flex-1 py-3 bg-gradient-to-br from-primary-container to-  
primary text-white rounded-xl text-sm font-semibold editorial-shadow  
hover:opacity-90 active:scale-95 transition-all disabled:opacity-60">
```

```
{mutation.isPending ? 'Saving...' : isEdit ? 'Save Changes' : 'Add Product'}
```

```
</button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
)
```

```
}
```

SupplierImportModal.jsx

```
import { useState, useRef } from 'react'

import { useMutation } from '@tanstack/react-query'

import toast from 'react-hot-toast'

import { importExportApi } from '../services/api'

const STEPS = ['upload', 'mapping', 'preview', 'done']

const TARGET_FIELDS = [
  { key: 'name', label: 'Supplier Name', required: true, aliases: ['name', 'supplier', 'supplier_name', 'company', 'company_name'] },
  { key: 'contact_email', label: 'Contact Email', aliases: ['email', 'contact_email', 'contact email'] },
  { key: 'phone', label: 'Phone', aliases: ['phone', 'mobile', 'phone_number', 'contact_number', 'phone number', 'contact number'] },
  { key: 'address', label: 'Address', aliases: ['address', 'full_address', 'location'] },
  { key: 'lead_time_days', label: 'Lead Time (Days)', aliases: ['lead_time', 'lead_time_days', 'delivery_days', 'lead time days', 'lead time'] },
  { key: 'notes', label: 'Notes', aliases: ['notes', 'remark', 'remarks', 'description'] },
]

export default function SupplierImportModal({ onClose, onImported }) {
  const [step, setStep] = useState('upload')
  const [file, setFile] = useState(null)
  const [mapping, setMapping] = useState({})
  const [headers, setHeaders] = useState([])
```

```
const [preview, setPreview] = useState(null)
const [errors, setErrors] = useState([])
const [result, setResult] = useState(null)
const [dragging, setDragging] = useState(false)
const fileRef = useRef()
```

```
const previewMutation = useMutation({
  mutationFn: async (override) => {
    const fd = new FormData()
    fd.append('file', override?.file || file)
    fd.append('mapping', JSON.stringify(override?.mapping || mapping))
    fd.append('dry_run', 'true')
    return importExportApi.importSuppliers(fd)
  },
  onSuccess: (res) => {
    setPreview(res.data.preview)
    setErrors(res.data.errors || [])
    setStep('preview')
  },
  onError: () => toast.error('Preview failed'),
})
```

```
const importMutation = useMutation({
  mutationFn: async () => {
    const fd = new FormData()
```

```

    fd.append('file', file)

    fd.append('mapping', JSON.stringify(mapping))

    return importExportApi.importSuppliers(fd)
  },
  onSuccess: (res) => {
    setResult(res.data)
    setStep('done')
  },
  onError: () => toast.error('Import failed'),
})

const handleFile = async (f) => {
  if (!f) return
  const ext = f.name.split('.').pop().toLowerCase()
  if (!['csv', 'xlsx', 'xls'].includes(ext)) {
    return toast.error('Only CSV and Excel files supported')
  }
  setFile(f)

  if (ext === 'csv') {
    const text = await f.text()
    const firstLine = text.split('\n')[0]
    const hdrs = firstLine.split(',').map(h => h.replace(/"/g, '').trim()).filter(Boolean)
    setHeaders(hdrs)
  }
}

```

```

// Auto-mapping logic
const newMap = {}
let matchCount = 0
TARGET_FIELDS.forEach(field => {
  const match = hdrs.find(h => {
    const norm = h.toLowerCase().trim().replace(/^[a-z0-9]/g, ' ').replace(/\s+/g,
    '_')
    return field.aliases.includes(norm) ||
    field.aliases.includes(h.toLowerCase().trim())
  })
  if (match) {
    newMap[field.key] = match
    matchCount++
  }
})

setMapping(newMap)

if (newMap.name && matchCount >= 3) {
  toast.success('Fields auto-mapped successfully!')
  previewMutation.mutate({ file: f, mapping: newMap })
  return
} else if (newMap.name) {
  toast('Partial mapping found. Please review.', { icon: 'i'})
} else {

```



```

    toast('No mapping found. Please map manually.', { icon: 'i'})
  }
} else {
  setHeaders([])
}
setStep('mapping')
}

const handleDrop = (e) => {
  e.preventDefault()
  setDragging(false)
  const f = e.dataTransfer.files[0]
  if (f) handleFile(f)
}

return (
  <div className="fixed inset-0 z-50 flex items-center justify-center">
    <div className="absolute inset-0 bg-black/30 backdrop-blur-sm"
      onClick={onClose} />
    <div className="relative bg-white rounded-2xl shadow-editorial-lg w-full max-w-2xl mx-4 max-h-[90vh] overflow-hidden flex flex-col">
      { /* Header */ }
      <div className="px-6 pt-6 pb-4 border-b border-surface-container-low flex items-center justify-between flex-shrink-0">
        <div>
          <h2 className="text-xl font-bold text-on-surface">Import Suppliers</h2>

```



```
<p className="text-xs text-on-surface-variant mt-0.5">CSV or Excel • Smart  
local auto-mapping</p>
```

```
</div>
```

```
<button onClick={onClose} className="p-2 rounded-lg hover:bg-surface-  
container-low transition-colors">
```

```
<span className="material-symbols-outlined text-on-surface-  
variant">close</span>
```

```
</button>
```

```
</div>
```

```
{/* Step indicator */}
```

```
<div className="flex px-6 py-3 gap-2 flex-shrink-0 border-b border-surface-  
container-low">
```

```
{STEPS.map((s, i) => {
```

```
<div key={s} className="flex items-center gap-2">
```

```
<div className={`w-6 h-6 rounded-full flex items-center justify-center text-  
xs font-bold transition-colors ${
```

```
STEPS.indexOf(step) >= i ? 'bg-primary text-white' : 'bg-surface-container  
text-on-surface-variant'
```

```
}`}>{i + 1}</div>
```

```
<span className={`text-xs font-medium capitalize hidden sm:block ${
```

```
step === s ? 'text-primary font-bold' : 'text-on-surface-variant'
```

```
}`}>{s}</span>
```

```
{i < STEPS.length - 1 && <div className="w-8 h-px bg-surface-container-  
high" />}
```

```
</div>
```

```
}}}
```

```
</div>
```

```
{/* Content */}
```

```
<div className="overflow-y-auto flex-1 p-6 scrollbar-thin">
```

```
  {/* STEP 1: Upload */}
```

```
  {step === 'upload' && {
```

```
    <div
```

```
      onDragOver={e => { e.preventDefault(); setDragging(true) }}
```

```
      onDragLeave={() => setDragging(false)}
```

```
      onDrop={handleDrop}
```

```
      className={`border-2 border-dashed rounded-2xl p-12 text-center
        transition-colors cursor-pointer ${
          dragging ? 'border-primary bg-primary-fixed/10' : 'border-outline-variant
        hover:border-primary/50 hover:bg-surface-container-low'
        }}
      onClick={() => fileRef.current?.click()}
    >
```

```
      <span className="material-symbols-outlined text-5xl text-on-surface-
        variant mb-4 block">upload_file</span>
```

```
      <p className="text-base font-bold text-on-surface mb-1">Drop your file
        here</p>
```

```
      <p className="text-sm text-on-surface-variant mb-4">or click to
        browse</p>
```

```
      <p className="text-xs text-outline">Supports: .csv, .xlsx, .xls • Max
        10MB</p>
```

```
      <input ref={fileRef} type="file" accept=".csv,.xlsx,.xls" className="hidden"
```

```

        onChange={e => handleFile(e.target.files[0])} />
    </div>
  )}

  { /* STEP 2: Mapping */
    {step === 'mapping' && (
      <div>
        <div className="flex items-center justify-between mb-4">
          <div>
            <p className="text-sm font-bold text-on-surface">Column Mapping</p>
            <p className="text-xs text-on-surface-variant mt-0.5">
              File: <span className="font-mono text-primary">{file?.name}</span>
            </p>
          </div>
          <button onClick={() => { setStep('upload'); setFile(null); setHeaders([]) }}
            className="text-xs text-on-surface-variant hover:text-on-surface">
            ← Change file
          </button>
        </div>

        {headers.length === 0 ? (
          <div className="bg-surface-container-low rounded-xl p-4 text-sm text-on-surface-variant text-center">
            Excel file detected. Mapping will be applied on import.
          </div>
        ) : (

```

```

): {
  <div className="space-y-3">
    {TARGET_FIELDS.map(field => {
      <div key={field.key} className="flex items-center gap-4">
        <div className="w-36 flex-shrink-0">
          <span className="text-xs font-bold text-on-surface">
            {field.label}
            {field.required && <span className="text-tertiary ml-0.5">*</span>}
          </span>
        </div>
        <div className="flex-1">
          <select
            value={mapping[field.key] || ""}
            onChange={e => setMapping({ ...mapping, [field.key]: e.target.value ||
undefined })}
            className="w-full bg-surface-container-low border-none rounded-xl
px-3 py-2 text-sm focus:ring-2 focus:ring-primary/30 outline-none"
          >
            <option value="">— Skip field —</option>
            {headers.map(h => <option key={h} value={h}>{h}</option>)}
          </select>
        </div>
        {mapping[field.key] && {
          <span className="material-symbols-outlined text-secondary text-
[18px] flex-shrink-0">check_circle</span>
        }}
      )}
    }
  </div>
}

```

```

    </div>

  )})
</div>
})

<div className="flex gap-3 mt-6">

  <button onClick={() => previewMutation.mutate()}

    disabled={previewMutation.isPending || (!mapping.name &&
headers.length > 0)}

    className="flex-1 py-3 bg-gradient-to-br from-primary-container to-
primary text-white rounded-xl text-sm font-semibold editorial-shadow
hover:opacity-90 active:scale-95 transition-all disabled:opacity-50">

      {previewMutation.isPending ? 'Analyzing...' : 'Preview Import →'}

    </button>
  </div>
</div>
})

{/* STEP 3: Preview */}

{step === 'preview' && preview && {

  <div>

    <div className="flex items-center gap-4 mb-4">

      <div className="flex-1 bg-secondary-container/20 rounded-xl p-3 text-
center">

        <p className="text-2xl font-bold text-
secondary">{previewMutation.data?.data?.valid_count || 0}</p>

```

```

        <p className="text-xs text-on-surface-variant font-medium">Valid
rows</p>

    </div>

    <div className="flex-1 bg-tertiary-fixed/30 rounded-xl p-3 text-center">

        <p className="text-2xl font-bold text-
tertiary">{previewMutation.data?.data?.error_count || 0}</p>

        <p className="text-xs text-on-surface-variant font-medium">Error
rows</p>

    </div>

</div>

{/* Preview table */}

<div className="border border-surface-container-high rounded-xl
overflow-hidden mb-4">

    <div className="overflow-x-auto">

        <table className="w-full text-xs">

            <thead className="bg-surface-container-low">

                <tr>

                    {[ 'Name', 'Email', 'Phone', 'Address', 'Lead Time' ].map(h => (

                        <th key={h} className="px-3 py-2.5 text-left font-bold text-on-
surface-variant uppercase tracking-wider whitespace-nowrap">{h}</th>

                    )))}

                </tr>

            </thead>

            <tbody className="divide-y divide-surface-container-low">

                {preview.slice(0, 8).map((row, i) => (

                    <tr key={i} className="hover:bg-surface-container-low/50">

```

```

        <td className="px-3 py-2.5 font-medium text-on-surface truncate
max-w-[150px]">{row.name}</td>

        <td className="px-3 py-2.5 text-on-surface-variant truncate max-w-
[120px]">{row.contact_email}</td>

        <td className="px-3 py-2.5 text-on-surface whitespace-
nowrap">{row.phone}</td>

        <td className="px-3 py-2.5 text-on-surface-variant truncate max-w-
[150px]">{row.address}</td>

        <td className="px-3 py-2.5 text-on-surface">{row.lead_time_days ?
`${row.lead_time_days} days` : ""}</td>

    </tr>

    )}
</tbody>
</table>
</div>

{preview.length > 8 && (
    <p className="text-xs text-on-surface-variant text-center py-2 border-t
border-surface-container-low">
        +{preview.length - 8} more rows
    </p>
    )}
</div>

{errors.length > 0 && (
    <div className="bg-tertiary-fixed/20 rounded-xl p-4 mb-4">

        <p className="text-xs font-bold text-tertiary mb-2"> ⚠️ {errors.length}
rows will be skipped:</p>

```



```

      {errors.slice(0, 3).map((e, i) => {
        <p key={i} className="text-xs text-on-surface-variant">Row {e.row}:
        {e.errors?.join(', ')}</p>
      })}
    </div>
  )}

  <div className="flex gap-3">
    <button onClick={() => setStep('mapping')}
      className="px-6 py-3 bg-surface-container-highest rounded-xl text-sm
font-semibold text-on-surface hover:bg-surface-container-high transition-colors">
      ← Back
    </button>
    <button onClick={() => importMutation.mutate()}
      disabled={importMutation.isPending}
      className="flex-1 py-3 bg-gradient-to-br from-primary-container to-
primary text-white rounded-xl text-sm font-semibold editorial-shadow
hover:opacity-90 active:scale-95 transition-all disabled:opacity-50">
      {importMutation.isPending ? 'Importing...' : 'Import
      ${previewMutation.data?.data?.valid_count || 0} Suppliers`}
    </button>
  </div>
</div>
)}

{/* STEP 4: Done */}

{step === 'done' && result && {

```



```

<div className="text-center py-6">

  <div className="w-20 h-20 bg-secondary-container/30 rounded-full flex
items-center justify-center mx-auto mb-6">

    <span className="material-symbols-outlined text-4xl text-secondary"
style={{ fontVariationSettings: "'FILL' 1" }}>check_circle</span>

  </div>

  <h3 className="text-2xl font-bold text-on-surface mb-2">Import
Complete!</h3>

  <p className="text-on-surface-variant mb-6">

    <span className="text-secondary font-bold text-
xl">{result.imported_count}</span> suppliers imported successfully

    {result.error_count > 0 && (

      <span className="text-tertiary"> · {result.error_count} skipped</span>

    )}

  </p>

  <div className="flex gap-3">

    <button onClick={onClose}

      className="flex-1 py-3 bg-surface-container-highest rounded-xl text-sm
font-semibold text-on-surface hover:bg-surface-container-high transition-colors">

      Close

    </button>

    <button onClick={onImported}

      className="flex-1 py-3 bg-gradient-to-br from-primary-container to-
primary text-white rounded-xl text-sm font-semibold editorial-shadow
hover:opacity-90 transition-all">

      View Suppliers

    </button>

```

```
</div>  
</div>  
}  
</div>  
</div>  
</div>  
)  
}
```



TransactionModal.jsx

```
import { useState } from 'react'

import { useMutation } from '@tanstack/react-query'

import toast from 'react-hot-toast'

import { transactionsApi } from '../services/api'

export default function TransactionModal({ product, onClose, onSave }) {

  const [form, setForm] = useState({

    type: 'IN', quantity: '', reason: '', reference_no: '', notes: ''

  })

  const mutation = useMutation({

    mutationFn: () => transactionsApi.create({ ...form, product_id: product.id,
    quantity: parseInt(form.quantity) }),

    onSuccess: (res) => {

      toast.success(`Stock updated! New balance: ${res.data.new_stock} units`)

      onSave()

    },

    onError: (err) => toast.error(err.response?.data?.error || 'Transaction failed'),

  })

  const handleSubmit = (e) => {

    e.preventDefault()

    if (!form.quantity || parseInt(form.quantity) < 1) return toast.error('Enter a valid
    quantity')
```

```

mutation.mutate()
}

```

```

const IN_REASONS = ['Purchase Order', 'Restock', 'Return from Customer',
'Transfer In', 'Found in Count', 'Other']

```

```

const OUT_REASONS = ['Sale', 'Transfer Out', 'Damaged/Write-off', 'Sample',
'Theft/Loss', 'Other']

```

```

return (
  <div className="fixed inset-0 z-50 flex items-center justify-center">
    <div className="absolute inset-0 bg-black/30 backdrop-blur-sm"
onClick={onClose} />
    <div className="relative bg-white rounded-2xl shadow-editorial-lg w-full max-
w-md mx-4">
      <div className="px-6 pt-6 pb-4 border-b border-surface-container-low">
        <div className="flex items-center justify-between mb-1">
          <h2 className="text-xl font-bold text-on-surface">Stock Movement</h2>
          <button onClick={onClose} className="p-2 rounded-lg hover:bg-surface-
container-low transition-colors">
            <span className="material-symbols-outlined text-on-surface-
variant">close</span>
          </button>
        </div>
        <p className="text-sm text-on-surface-variant">
          <span className="font-semibold">{product.name}</span> — Current:
          <span className="font-bold text-on-surface">{product.current_stock}
          {product.unit || 'units'}</span>
        </p>
      </div>
    </div>
  </div>
)

```

```
</div>
```

```
<form onSubmit={handleSubmit} className="p-6 space-y-4">
```

```
  { /* Type toggle */ }
```

```
  <div>
```

```
    <label className="block text-xs font-bold text-on-surface-variant uppercase tracking-wider mb-2">Movement Type *</label>
```

```
    <div className="grid grid-cols-2 gap-2">
```

```
      { ['IN', 'OUT'].map(type => (
```

```
        <button
```

```
          key={type} type="button"
```

```
          onClick={() => setForm({ ...form, type, reason: " " })}
```

```
          className={ py-3 rounded-xl text-sm font-bold transition-all flex items-center justify-center gap-2 ${
```

```
            form.type === type
```

```
            ? type === 'IN'
```

```
            ? 'bg-secondary-container/30 text-secondary ring-2 ring-secondary'
```

```
            : 'bg-primary-fixed/40 text-primary ring-2 ring-primary'
```

```
            : 'bg-surface-container-low text-on-surface-variant hover:bg-surface-container'
```

```
          ) }
```

```
        >
```

```
        <span className="material-symbols-outlined text-[18px]">{type === 'IN' ? 'add_circle' : 'remove_circle'}</span>
```

```
        Stock {type === 'IN' ? 'In' : 'Out'}
```

```
      </button>
```

```

    )}
  </div>
</div>

<div>
  <label className="block text-xs font-bold text-on-surface-variant uppercase
tracking-wider mb-1.5">Quantity *</label>

  <input
    required type="number" min="1" value={form.quantity}
    onChange={e => setForm({ ...form, quantity: e.target.value })}
    className="w-full bg-surface-container-low border-none rounded-xl px-4
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none text-center text-2xl
font-bold"
    placeholder="0"
  />
  {form.type === 'OUT' && parseInt(form.quantity) > product.current_stock &&
  (
    <p className="text-xs text-tertiary mt-1 font-medium"> ⚠ Exceeds current
stock ({product.current_stock})</p>
  )}
</div>

<div>
  <label className="block text-xs font-bold text-on-surface-variant uppercase
tracking-wider mb-1.5">Reason *</label>

  <select required value={form.reason} onChange={e => setForm({ ...form,
reason: e.target.value })}

```

```
className="w-full bg-surface-container-low border-none rounded-xl px-4  
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none">
```

```
<option value="">Select reason...</option>
```

```
{{(form.type === 'IN' ? IN_REASONS : OUT_REASONS).map(r =>
```

```
<option key={r} value={r}>{r}</option>}}
```

```
</select>
```

```
</div>
```

```
<div>
```

```
<label className="block text-xs font-bold text-on-surface-variant uppercase  
tracking-wider mb-1.5">Reference No. (optional)</label>
```

```
<input value={form.reference_no} onChange={e => setForm({ ...form,  
reference_no: e.target.value })}>
```

```
className="w-full bg-surface-container-low border-none rounded-xl px-4  
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none"
```

```
placeholder="PO#, Invoice#, etc." />
```

```
</div>
```

```
<div>
```

```
<label className="block text-xs font-bold text-on-surface-variant uppercase  
tracking-wider mb-1.5">Notes (optional)</label>
```

```
<textarea value={form.notes} onChange={e => setForm({ ...form, notes:  
e.target.value })}>
```

```
rows={2} className="w-full bg-surface-container-low border-none  
rounded-xl px-4 py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none  
resize-none" />
```

```
</div>
```



```

<div className="flex gap-3 pt-1">
  <button type="button" onClick={onClose}
    className="flex-1 py-3 bg-surface-container-highest rounded-xl text-sm
font-semibold text-on-surface hover:bg-surface-container-high transition-colors">
    Cancel
  </button>
  <button type="submit" disabled={mutation.isPending}
    className={`flex-1 py-3 text-white rounded-xl text-sm font-semibold
editorial-shadow active:scale-95 transition-all disabled:opacity-60 ${
      form.type === 'IN'
        ? 'bg-secondary hover:opacity-90'
        : 'bg-gradient-to-br from-primary-container to-primary hover:opacity-90'
    }}>
    {mutation.isPending ? 'Processing...' : `Record Stock ${form.type}`}
  </button>
</div>
</form>
</div>
</div>
)
}

```


CategoryBadge.jsx

```
export default function CategoryBadge({ children, className = "" }) {  
  return (  
    <span  
      className={`inline-flex max-w-full items-center rounded-full bg-surface-  
container px-2.5 py-1 text-xs font-medium text-on-surface-variant ring-1 ring-  
outline-variant/40 ${className}`.trim()}  
      title={children}  
    >  
      <span className="truncate">{children || 'Uncategorized'}</span>  
    </span>  
  );  
}
```



StatCard.jsx

```
import StatusBadge from './StatusBadge';

function resolveBadgeVariant(variant) {

  if (variant === 'success' || variant === 'danger' || variant === 'warning' || variant
  === 'neutral' || variant === 'info') {

    return variant;

  }

  return 'neutral';

}

export default function StatCard({ label, value, badge, badgeVariant = 'neutral', icon,
loading }) {

  if (loading) {

    return (

      <div className="rounded-xl bg-surface-container-lowest p-5 shadow-sm ring-1
ring-outline-variant/20">

        <div className="skeleton mb-4 h-4 w-24 rounded" />

        <div className="skeleton h-8 w-32 rounded" />

      </div>

    );

  }

  return (

    <div className="rounded-xl bg-surface-container-lowest p-5 shadow-sm ring-1
ring-outline-variant/20 transition-shadow duration-200 hover:shadow-md">
```

```

<div className="flex items-start justify-between gap-3">
  <div className="space-y-2">
    <p className="text-sm font-medium text-on-surface-variant">{label}</p>
    <p className="text-2xl font-bold tracking-tight text-on-surface sm:text-3xl">{value}</p>
  </div>
  {icon ? (
    <div className="flex h-10 w-10 shrink-0 items-center justify-center rounded-xl bg-surface-container text-on-surface-variant">
      <span className="material-symbols-outlined text-[20px]">{icon}</span>
    </div>
  ) : null}
</div>
{badge ? (
  <div className="mt-4 flex items-center justify-end">
    <StatusBadge
      variant={resolveBadgeVariant(badgeVariant)}>{badge}</StatusBadge>
  </div>
  ) : null}
</div>
);
}

```

StatusBadge.jsx

```
const STATUS_VARIANTS = {  
  success: 'bg-secondary-container/20 text-secondary ring-1 ring-secondary/10',  
  danger: 'bg-tertiary-fixed/40 text-tertiary ring-1 ring-tertiary/10',  
  warning: 'bg-amber-100 text-amber-700 ring-1 ring-amber-200 dark:bg-amber-500/15 dark:text-amber-300 dark:ring-amber-400/20',  
  neutral: 'bg-surface-container text-on-surface-variant ring-1 ring-outline-variant/40',  
  info: 'bg-primary-fixed/40 text-primary ring-1 ring-primary/10',  
};  
  
export default function StatusBadge({ children, variant = 'neutral', className = '' }) {  
  return (  
    <span  
      className={inline-flex items-center gap-1.5 rounded-full px-2.5 py-1 text-xs  
font-semibold whitespace-nowrap ${STATUS_VARIANTS[variant] ||  
STATUS_VARIANTS.neutral} ${className}`.trim()}  
    >  
      {children}  
    </span>  
  );  
}
```

StockBar.jsx

```
const BAR_VARIANTS = {
  in_stock: 'bg-secondary',
  low: 'bg-amber-400 dark:bg-amber-300',
  out_of_stock: 'bg-tertiary',
};

export default function StockBar({ current = 0, min = 0, status = 'in_stock', unit =
'units' }) {
  const safeCurrent = Number(current) || 0;
  const safeMin = Number(min) || 0;
  const percent = safeMin > 0
    ? Math.max(0, Math.min(100, (safeCurrent / (safeMin * 2)) * 100))
    : (safeCurrent > 0 ? 100 : 0);

  return (
    <div className="flex min-w-[10rem] flex-col gap-1">
      <div className="flex items-center justify-between gap-3 text-[11px] font-
medium text-on-surface-variant">
        <span className="truncate">{safeCurrent} {unit}</span>
        <span>{Math.round(percent)}%</span>
      </div>
      <div className="h-2 overflow-hidden rounded-full bg-surface-container-high">
        <div
          className={`h-full rounded-full transition-all duration-300
${BAR_VARIANTS[status] || BAR_VARIANTS.in_stock}`}
        >

```

```

        style={{ width: `${Math.max(2, percent)}%` }}
      />
    </div>
  </div>
);
}

```



AllInsights.jsx

```
import { useState, useRef, useEffect } from 'react'

import { useQuery, useMutation } from '@tanstack/react-query'

import { format } from 'date-fns'

import toast from 'react-hot-toast'

import { aiApi, productsApi } from '../services/api'

const QUICK_PROMPTS = [
  'Which products are at risk of stockout in the next 30 days?',
  'What are my top-selling categories?',
  'Show me reorder recommendations',
  'Identify dead stock items',
  'How can I reduce inventory carrying costs?',
  'What should I reorder this week?',
]

export default function AllInsights() {
  const [messages, setMessages] = useState([
    {
      role: 'assistant',
      content: "Good day! I'm your AI inventory assistant powered by Gemini. I can analyze your inventory data, predict demand, identify dead stock, suggest reorder quantities, and answer any inventory-related questions. What would you like to explore?",
      timestamp: new Date()
    }
  ])

```

```

  })

  const [input, setInput] = useState("")

  const [activePanel, setActivePanel] = useState('chat') // chat / reorder / deadstock

  const bottomRef = useRef()

  const inputRef = useRef()

  const { data: productsData } = useQuery({
    queryKey: ['products-for-ai'],
    queryFn: () => productsApi.list({ limit: 200 }).then(r => r.data.products),
  })

  const chatMutation = useMutation({
    mutationFn: (msgs) => aiApi.chat(msgs),
    onSuccess: (res) => {
      setMessages(prev => [...prev, {
        role: 'assistant',
        content: res.data.response,
        timestamp: new Date()
      }])
    },
    onError: (err) => {
      const msg = err.response?.data?.error || 'AI unavailable. Please check your Gemini API key.'

      setMessages(prev => [...prev, {
        role: 'assistant',

```



```
    content: ` ⚠️ ${msg}`,  
    timestamp: new Date(),  
    error: true  
  })  
}  
})
```

```
const reorderMutation = useMutation({  
  mutationFn: () => aiApi.reorderSuggestions(),  
  onError: (err) => toast.error(err.response?.data?.error || 'AI unavailable'),  
})
```

```
const deadStockMutation = useMutation({  
  mutationFn: () => aiApi.deadStock(60),  
  onError: (err) => toast.error(err.response?.data?.error || 'AI unavailable'),  
})
```

```
useEffect(() => {  
  bottomRef.current?.scrollIntoView({ behavior: 'smooth' })  
}, [messages])
```

```
const sendMessage = (text) => {  
  const msg = text || input.trim()  
  if (!msg) return  
  const userMsg = { role: 'user', content: msg, timestamp: new Date() }
```

```

const newMessages = [...messages, userMsg]
setMessages(newMessages)
setInput('')

chatMutation.mutate(newMessages.map(m => ({ role: m.role, content: m.content
})))
}

```

```

const handleKeyDown = (e) => {
  if (e.key === 'Enter' && !e.shiftKey) {
    e.preventDefault()
    sendMessage()
  }
}

return (
  <div className="h-[calc(100vh-4rem)] flex bg-surface">
    {/* Chat Area */}
    <section className={`flex flex-col ${activePanel !== 'chat' ? 'hidden md:flex
md:flex-1' : 'flex-1'}`}>
      {/* Chat header */}
      <div className="px-8 py-5 border-b border-surface-container-low flex items-
center justify-between flex-shrink-0">
        <div className="flex items-center gap-3">
          <div className="w-9 h-9 rounded-xl bg-primary flex items-center justify-
center text-white">
            <span className="material-symbols-outlined text-[18px]" style={{
fontVariationSettings: '"FILL' 1" }}>smart_toy</span>

```

```

</div>

<div>

  <h2 className="font-bold text-on-surface">AI Inventory Assistant</h2>

  <div className="flex items-center gap-1.5">

    <span className="w-1.5 h-1.5 rounded-full bg-secondary animate-pulse"
  />

    <p className="text-xs text-on-surface-variant">Powered by Gemini 2.0
Flash</p>

  </div>

</div>

</div>

<div className="flex gap-2">

  <button

    onClick={() => { setActivePanel('reorder'); reorderMutation.mutate() }}

    className={`flex items-center gap-2 px-4 py-2 rounded-xl text-xs font-
semibold transition-all ${activePanel === 'reorder' ? 'bg-primary text-white' : 'bg-
surface-container-highest text-on-surface hover:bg-surface-container-high'}}`>

    <span className="material-symbols-outlined text-
[16px]">local_shipping</span>

    Reorder

  </button>

  <button

    onClick={() => { setActivePanel('deadstock'); deadStockMutation.mutate() }}

    className={`flex items-center gap-2 px-4 py-2 rounded-xl text-xs font-
semibold transition-all ${activePanel === 'deadstock' ? 'bg-primary text-white' : 'bg-
surface-container-highest text-on-surface hover:bg-surface-container-high'}}`>

    <span className="material-symbols-outlined text-
[16px]">inventory</span>

```

```

        Dead Stock

    </button>

</div>

</div>

    {/* Messages */}

    <div className="flex-1 overflow-y-auto px-8 py-6 space-y-6 scrollbar-thin">

        {/* Quick prompts - only show at start */}

        {messages.length === 1 && (

            <div className="grid grid-cols-2 gap-2 max-w-xl">

                {QUICK_PROMPTS.map(prompt => (

                    <button key={prompt} onClick={() => sendMessage(prompt)}

                        className="text-left px-4 py-3 bg-surface-container-lowest rounded-xl
text-xs font-medium text-on-surface hover:bg-surface-container-low border border-
surface-container-high transition-colors editorial-shadow">

                            {prompt}

                        </button>

                    )))}

                </div>

            )}

        {messages.map((msg, i) => (

            <div key={i} className={`flex gap-3 max-w-3xl ${msg.role === 'user' ? 'ml-
auto flex-row-reverse' : ''}`>

                {/* Avatar */}

```

```

    <div className={`w-8 h-8 rounded-xl flex items-center justify-center flex-
shrink-0 ${
      msg.role === 'assistant' ? 'bg-primary text-white' : 'bg-surface-container-
high text-on-surface'
    }}>

      <span className="material-symbols-outlined text-[16px]" style={msg.role
=== 'assistant' ? { fontVariationSettings: '"FILL' 1" } : {}}>

        {msg.role === 'assistant' ? 'smart_toy' : 'person'}

      </span>

    </div>

    { /* Bubble */ }

    <div className="space-y-1">

      <div className={`rounded-2xl px-5 py-4 text-sm leading-relaxed
whitespace-pre-wrap ${
        msg.role === 'user'
          ? 'bg-primary text-white rounded-tr-sm'
          : msg.error
            ? 'bg-tertiary-fixed/20 text-on-surface border border-tertiary-fixed
rounded-tl-sm'
            : 'bg-surface-container-lowest text-on-surface shadow-sm border
border-surface-container-high rounded-tl-sm'
        }}>

        {msg.content}

      </div>

      <p className={`text-[10px] text-on-surface-variant px-1 ${msg.role ===
'user' ? 'text-right' : ''}}>

```

```

    {format(new Date(msg.timestamp), 'h:mm a')}}
  </p>
</div>
</div>
)))

{chatMutation.isPending && (
  <div className="flex gap-3 max-w-3xl">
    <div className="w-8 h-8 rounded-xl bg-primary flex items-center justify-
center text-white flex-shrink-0">
      <span className="material-symbols-outlined text-[16px]" style={{
fontVariationSettings: "'FILL' 1" }}>smart_toy</span>
    </div>
    <div className="bg-surface-container-lowest rounded-2xl rounded-tl-sm
px-5 py-4 shadow-sm border border-surface-container-high">
      <div className="flex items-center gap-2">
        <div className="flex gap-1">
          {[0, 1, 2].map(i => (
            <span key={i} className="w-2 h-2 bg-primary/40 rounded-full
animate-bounce"
              style={{ animationDelay: `${i * 0.15}s` }} />
          ))}
        </div>
        <span className="text-xs text-on-surface-variant">Analyzing your
inventory...</span>
      </div>
    </div>
  </div>
)

```

```

    </div>

  }}

  <div ref={bottomRef} />

</div>

{ /* Input */ }

<div className="px-8 py-5 border-t border-surface-container-low flex-shrink-0">

  <div className="flex items-end gap-3 bg-surface-container-lowest rounded-2xl border border-surface-container-high p-3 shadow-sm">

    <textarea

      ref={inputRef}

      value={input}

      onChange={e => setInput(e.target.value)}

      onKeyDown={handleKeyDown}

      placeholder="Ask about inventory, demand, reorders, dead stock..."

      rows={1}

      className="flex-1 bg-transparent border-none resize-none text-sm focus:ring-0 outline-none text-on-surface placeholder:text-on-surface-variant max-h-32 overflow-y-auto scrollbar-thin"

      style={{ minHeight: '24px' }}

    />

    <button

      onClick={() => sendMessage()}

      disabled={!input.trim() || chatMutation.isPending}

```



```

        className="w-9 h-9 rounded-xl bg-primary text-white flex items-center
justify-center flex-shrink-0 hover:opacity-90 active:scale-95 transition-all
disabled:opacity-40"

        >

        <span className="material-symbols-outlined text-[18px]">send</span>

    </button>

</div>

<p className="text-[10px] text-on-surface-variant text-center mt-2">Press
Enter to send · Shift+Enter for new line</p>

</div>

</section>

{ /* Side Panel - Reorder or Dead Stock */ }

{ activePanel !== 'chat' && {

    <aside className="w-full md:w-[420px] border-l border-surface-container-low
flex flex-col bg-surface-container-lowest overflow-hidden">

        <div className="px-6 py-5 border-b border-surface-container-low flex items-
center justify-between flex-shrink-0">

            <h3 className="font-bold text-on-surface">

                { activePanel === 'reorder' ? ' 🛠️ Reorder Suggestions' : ' 📦 Dead Stock
Analysis' }

            </h3>

            <button onClick={() => setActivePanel('chat')}

                className="p-2 rounded-lg hover:bg-surface-container-low transition-
colors">

                <span className="material-symbols-outlined text-on-surface-
variant">close</span>

            </button>

```



```

</div>

<div className="flex-1 overflow-y-auto scrollbar-thin p-6">
  { /* Reorder Panel */ }
  {activePanel === 'reorder' && (
    <>
      {reorderMutation.isPending ? (
        <div className="space-y-4">
          {[...Array(4)].map((_, i) => <div key={i} className="skeleton h-28
rounded-xl" />)}
        </div>
      ) : reorderMutation.data ? (
        <>
          {reorderMutation.data.data.summary && (
            <div className="bg-primary-fixed/30 rounded-xl p-4 mb-4 text-sm
text-on-surface leading-relaxed">
              {reorderMutation.data.data.summary}
            </div>
          )}
        <div className="space-y-3">
          {(reorderMutation.data.data.recommendations || []).map((rec, i) => (
            <div key={i} className={`rounded-xl p-4 border ${
              rec.urgency === 'critical' ? 'border-tertiary/30 bg-tertiary-fixed/10' :
              rec.urgency === 'high' ? 'border-yellow-200 bg-yellow-50' :
              'border-surface-container-high bg-white'

```

```

    }}>
    <div className="flex items-start justify-between mb-2">
        <p className="text-sm font-bold text-on-
surface">{rec.product_name}</p>
        <span className={`text-[10px] font-bold uppercase px-2 py-0.5
rounded-full ${
            rec.urgency === 'critical' ? 'bg-tertiary text-white' :
            rec.urgency === 'high' ? 'bg-yellow-400 text-yellow-900' :
            'bg-surface-container text-on-surface-variant'
        }}>{rec.urgency}</span>
    </div>
    <div className="grid grid-cols-2 gap-2 text-xs text-on-surface-variant
mb-2">
        <div>Current: <span className="font-bold text-on-
surface">{rec.current_stock}</span></div>
        <div>Order qty: <span className="font-bold text-
secondary">{rec.suggested_order_qty}</span></div>
        <div className="col-span-2">Stockout in: <span className="font-
bold text-tertiary">{rec.estimated_stockout_days} days</span></div>
    </div>
    <p className="text-xs text-on-surface-variant leading-
relaxed">{rec.reason}</p>
</div>
)}}
{({reorderMutation.data.data.recommendations || []}).length === 0 && (
    <div className="text-center py-8">
        <span className="material-symbols-outlined text-4xl text-secondary
block mb-2" style={{ fontVariationSettings: '"FILL' 1" }}>check_circle</span>

```

```
up!</p>
```

```
<p className="text-xs text-on-surface-variant mt-1">No urgent  
reorders needed right now.</p>
```

```
</div>
```

```
}}
```

```
</div>
```

```
</>
```

```
): {
```

```
<div className="text-center py-12">
```

```
<span className="material-symbols-outlined text-5xl text-on-surface-  
variant mb-3 block">local_shipping</span>
```

```
<p className="text-sm text-on-surface-variant">Click "Reorder" to  
analyze your stock levels</p>
```

```
</div>
```

```
}}
```

```
</>
```

```
}}
```

```
{/* Dead Stock Panel */}
```

```
{activePanel === 'deadstock' && {
```

```
<>
```

```
{deadStockMutation.isPending ? {
```

```
<div className="space-y-4">
```

```
{[...Array(4)].map((_, i) => <div key={i} className="skeleton h-28  
rounded-xl" />)}
```

```
</div>
```

```

) : deadStockMutation.data ? (
  <>
  <div className="bg-surface-container-low rounded-xl p-4 mb-4 text-xs
font-semibold text-on-surface-variant">
    Total dead stock value:{' '}
    <span className="text-tertiary text-base font-bold">
      ${{deadStockMutation.data.data.total_dead_stock_value ||
0).toLocaleString()}}
    </span>
  </div>
  {deadStockMutation.data.data.insight && (
    <div className="bg-primary-fixed/20 rounded-xl p-4 mb-4 text-sm
text-on-surface leading-relaxed">
      {deadStockMutation.data.data.insight}
    </div>
  )}
  <div className="space-y-3">
    {(deadStockMutation.data.data.recommendations || []).map((rec, i) => (
      <div key={i} className="bg-white rounded-xl p-4 border border-
surface-container-high">
        <div className="flex items-start justify-between mb-2">
          <p className="text-sm font-bold text-on-
surface">{rec.product_name}</p>
          <span className="text-[10px] font-bold uppercase px-2 py-0.5
rounded-full bg-surface-container text-on-surface-variant">
            {rec.action?.replace(/_/g, ' ')}
          </span>

```

```

</div>

{rec.suggested_discount_pct && (
  <p className="text-sm font-bold text-primary mb-1">
    Suggest {rec.suggested_discount_pct}% discount
  </p>
)}

<p className="text-xs text-on-surface-variant leading-relaxed">{rec.reason}</p>
</div>
)}}

{((deadStockMutation.data.data.recommendations || []).length === 0 && (
  <div className="text-center py-8">
    <span className="material-symbols-outlined text-4xl text-secondary block mb-2" style={{ fontVariationSettings: "'FILL' 1" }}>check_circle</span>
    <p className="text-sm font-semibold text-on-surface">No dead stock!</p>
    <p className="text-xs text-on-surface-variant mt-1">All products have recent sales activity.</p>
  </div>
)}
</div>
</>
):(
  <div className="text-center py-12">
    <span className="material-symbols-outlined text-5xl text-on-surface-variant mb-3 block">inventory</span>

```

```
<p className="text-sm text-on-surface-variant">Analyzing dead  
stock...</p>
```

```
</div>
```

```
}}
```

```
</>
```

```
}}
```

```
</div>
```

```
</aside>
```

```
}}
```

```
</div>
```

```
)
```

```
}
```



Dashboard.jsx

```
import { useState } from 'react';

import { useQuery, useMutation } from '@tanstack/react-query';

import {
  AreaChart,
  Area,
  BarChart,
  Bar,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip,
  ResponsiveContainer,
  PieChart,
  Pie,
  Cell,
} from 'recharts';

import { format, parseISO } from 'date-fns';

import toast from 'react-hot-toast';

import { reportsApi, aiApi, alertsApi } from '../services/api';

import { useAuthStore } from '../store/authStore';

import StatCard from '../components/ui/StatCard';

import StatusBadge from '../components/ui/StatusBadge';

import { formatCompactCurrencyINR, formatCurrencyINR } from '../utils/currency';
```

```
const PIE_COLORS = ['#3525cd', '#006c49', '#960014', '#d44e3', '#4edea3',  
'#ffb3ad'];
```

```
function formatChartData(value, pattern) {  
  try {  
    return format(parseISO(value), pattern);  
  } catch {  
    return value;  
  }  
}
```

```
function InsightCard({ insight, index }) {  
  const variant = insight?.type === 'warning'  
    ? 'danger'  
    : insight?.type === 'opportunity'  
    ? 'success'  
    : 'info';  
  
  return (  
    <div  
      className="rounded-xl bg-surface-container-low p-4 ring-1 ring-outline-  
variant/20"  
      >  
        <div className="mb-2 flex items-center justify-between gap-3">  
          <p className="text-sm font-semibold text-on-surface">{insight?.title ||  
'Insight'}</p>
```



```

      <StatusBadge variant={variant}>{insight?.type || 'info'}</StatusBadge>
    </div>

    <p className="text-sm leading-6 text-on-surface-variant">{insight?.description}</p>
  </div>
);
}

```

```

export default function Dashboard() {
  const { user } = useAuthStore();
  const [aiInsights, setAiInsights] = useState(null);

  const { data: dashData, isLoading } = useQuery({
    queryKey: ['dashboard'],
    queryFn: () => reportsApi.dashboard().then((r) => r.data),
    enabled: !!user,
  });

  const { data: alerts } = useQuery({
    queryKey: ['alerts'],
    queryFn: () => alertsApi.list({ resolved: 'false', limit: 5 }).then((r) => r.data.alerts),
    enabled: !!user,
  });

  const insightsMutation = useMutation({

```

```

mutationFn: () => aiApi.dashboardInsights(dashData),
onSuccess: (res) => setAiInsights(res.data.insights),
onError: () => toast.error('AI insights unavailable. Check your Gemini API key.'),
});

const kpis = dashData?.kpis;
const salesTrend = dashData?.sales_trend || [];
const categories = dashData?.category_distribution || [];
const recent = dashData?.recent_transactions || [];

return (
  <div className="container mx-auto px-6 py-8">
    <div className="space-y-8">
      <div className="flex flex-col gap-4 lg:flex-row lg:items-end lg:justify-between">
        <div>
          <h2 className="text-3xl font-bold tracking-tight text-on-surface">Inventory Command</h2>
          <p className="mt-1 text-sm text-on-surface-variant">
            {format(new Date(), 'MMMM d, yyyy')} | Welcome back, {user?.name?.split('')[0]}
          </p>
        </div>
        <button
          onClick={() => insightsMutation.mutate()}
          disabled={insightsMutation.isPending || !dashData}
        >

```

```
className="inline-flex cursor-pointer items-center gap-2 self-start rounded-  
xl bg-surface-container-highest px-4 py-2.5 text-sm font-medium text-on-surface  
shadow-sm transition-all duration-200 hover:-translate-y-0.5 hover:shadow-md  
disabled:cursor-not-allowed disabled:opacity-50"
```

```
>
```

```
<span className="material-symbols-outlined text-  
[18px]">psychology</span>
```

```
{insightsMutation.isPending ? 'Analyzing...' : 'AI Insights'}
```

```
</button>
```

```
</div>
```

```
<div className="grid grid-cols-1 gap-6 md:grid-cols-2 xl:grid-cols-5">
```

```
<StatCard label="Total SKUs" value={kpis?.total_skus?.toLocaleString() || '-'}  
badge="+2.4%" badgeVariant="success" icon="inventory_2" loading={isLoading} />
```

```
<StatCard label="Total Stock Value" value={kpis ?  
formatCompactCurrencyINR(kpis.total_stock_value) : '-'} badge="Portfolio value"  
badgeVariant="info" icon="payments" loading={isLoading} />
```

```
<StatCard label="Monthly Profit" value={kpis?.profit_last_30d !== null ?  
formatCurrencyINR(kpis.profit_last_30d) : '-'} badge="30 Days"  
badgeVariant="success" icon="trending_up" loading={isLoading} />
```

```
<StatCard label="Out of Stock" value={kpis?.out_of_stock ?? '-'}  
badge={kpis?.out_of_stock > 0 ? 'Critical' : 'Clear'} badgeVariant={kpis?.out_of_stock  
> 0 ? 'danger' : 'success'} icon="error" loading={isLoading} />
```

```
<StatCard label="Low Stock" value={kpis?.low_stock ?? '-'}  
badge={kpis?.low_stock > 0 ? 'Attention' : 'Healthy'} badgeVariant={kpis?.low_stock  
> 0 ? 'warning' : 'success'} icon="warning" loading={isLoading} />
```

```
</div>
```

```
<div className="grid grid-cols-1 gap-6 xl:grid-cols-3">
```

```
<div className="xl:col-span-2">
```

```

<div className="grid grid-cols-1 gap-6">

  <div className="rounded-xl bg-surface-container-lowest p-6 shadow-sm
ring-1 ring-outline-variant/20 transition-shadow duration-200 hover:shadow-md">

    <div className="mb-6 flex flex-col gap-4 lg:flex-row lg:items-center
lg:justify-between">

      <div>

        <h3 className="text-lg font-semibold text-on-surface">Stock Movement
Trend</h3>

        <p className="mt-1 text-sm text-on-surface-variant">Last 7 days of
inbound and outbound inventory</p>

      </div>

      <div className="flex flex-wrap items-center gap-3 text-xs font-medium
text-on-surface-variant">

        <span className="inline-flex items-center gap-2"><span className="h-
2.5 w-2.5 rounded-full bg-secondary" />Stock In</span>

        <span className="inline-flex items-center gap-2"><span className="h-
2.5 w-2.5 rounded-full bg-primary" />Stock Out</span>

      </div>

    </div>

    {isLoading ? (

      <div className="skeleton h-56 rounded-xl" />

    ) : (

      <ResponsiveContainer width="100%" height={220}>

        <AreaChart data={salesTrend}>

          <defs>

            <linearGradient id="colorIn" x1="0" y1="0" x2="0" y2="1">

              <stop offset="5%" stopColor="#006c49" stopOpacity={0.18} />

```

```

        <stop offset="95%" stopColor="#006c49" stopOpacity={0} />
      </linearGradient>

      <linearGradient id="colorOut" x1="0" y1="0" x2="0" y2="1">

        <stop offset="5%" stopColor="#3525cd" stopOpacity={0.18} />

        <stop offset="95%" stopColor="#3525cd" stopOpacity={0} />

      </linearGradient>

    </defs>

    <CartesianGrid strokeDasharray="3 3" stroke="#d7dbe1"
    vertical={false} />

    <XAxis dataKey="date" tick={{ fontSize: 11, fill: '#6b7280' }}
    tickFormatter={(v) => formatChartDate(v, 'MMM d')} axisLine={false}
    tickLine={false} />

    <YAxis tick={{ fontSize: 11, fill: '#6b7280' }} axisLine={false}
    tickLine={false} />

    <Tooltip
      formatter={(v, n) => [v ?? 0, n === 'in' ? 'Stock In' : 'Stock Out']}
      labelFormatter={(l) => formatChartDate(l, 'MMM d, yyyy')}
    />

    <Area type="monotone" dataKey="in" stroke="#006c49"
    strokeWidth={2.5} fill="url(#colorIn)" />

    <Area type="monotone" dataKey="out" stroke="#3525cd"
    strokeWidth={2.5} fill="url(#colorOut)" />

  </AreaChart>

</ResponsiveContainer>

}}

</div>

```

```

<div className="rounded-xl bg-surface-container-lowest p-6 shadow-sm
ring-1 ring-outline-variant/20 transition-shadow duration-200 hover:shadow-md">

  <div className="mb-6 flex flex-col gap-4 lg:flex-row lg:items-center
lg:justify-between">

    <div>

      <h3 className="text-lg font-semibold text-on-surface">Revenue vs
Profit</h3>

      <p className="mt-1 text-sm text-on-surface-variant">Financial
performance across the last week</p>

    </div>

    <div className="flex flex-wrap items-center gap-3 text-xs font-medium
text-on-surface-variant">

      <span className="inline-flex items-center gap-2"><span className="h-
2.5 w-2.5 rounded-full bg-[#3525cd]" />Revenue</span>

      <span className="inline-flex items-center gap-2"><span className="h-
2.5 w-2.5 rounded-full bg-[#4edea3]" />Profit</span>

    </div>

  </div>

  {isLoading ? (
    <div className="skeleton h-56 rounded-xl" />
  ) : (
    <ResponsiveContainer width="100%" height={220}>

      <BarChart data={salesTrend}>

        <CartesianGrid strokeDasharray="3 3" stroke="#d7dbe1"
vertical={false} />

        <XAxis dataKey="date" tick={{ fontSize: 11, fill: '#6b7280' }}
tickFormatter={(v) => formatChartDate(v, 'MMM d')} axisLine={false}
tickLine={false} />

```



```

        <YAxis tick={{ fontSize: 11, fill: '#6b7280' }} axisLine={false}
tickLine={false} />

        <Tooltip

        formatter={(v, n) => [formatCurrencyINR(v || 0), n === 'revenue' ?
'Revenue' : 'Profit']}

        labelFormatter={(l) => formatChartDate(l, 'MMM d, yyyy')}

        />

        <Bar dataKey="revenue" fill="#3525cd" radius={[6, 6, 0, 0]} />

        <Bar dataKey="profit" fill="#4edea3" radius={[6, 6, 0, 0]} />

    </BarChart>
  </ResponsiveContainer>
)}
</div>
</div>
</div>

<div className="space-y-6">

  <div className="rounded-xl bg-surface-container-lowest p-6 shadow-sm
ring-1 ring-outline-variant/20 transition-shadow duration-200 hover:shadow-md">

    <div className="mb-4">

      <h3 className="text-lg font-semibold text-on-surface">By Category</h3>

      <p className="mt-1 text-sm text-on-surface-variant">Current stock value
distribution</p>

    </div>

    {isLoading ? (

      <div className="skeleton h-56 rounded-xl" />

```

```

) : categories.length > 0 ? {
  <>
  <ResponsiveContainer width="100%" height={180}>
    <PieChart>
      <Pie data={categories} cx="50%" cy="50%" innerRadius={48}
outerRadius={78} dataKey="value" paddingAngle={2}>
        {categories.map((category, index) => {
          <Cell key={category.id || category.name || `category-cell-${index}`}
fill={PIE_COLORS[index % PIE_COLORS.length]} />
        })}
      </Pie>
      <Tooltip formatter={(v) => [formatCurrencyINR(v || 0), 'Value']} />
    </PieChart>
  </ResponsiveContainer>
  <div className="mt-3 space-y-2">
    {categories.slice(0, 4).map((category, index) => {
      <div key={category.id || category.name || `category-${index}`}
className="flex items-center justify-between gap-3 rounded-lg px-1 py-1 text-sm">
        <div className="flex min-w-0 items-center gap-2">
          <span className="h-2.5 w-2.5 flex-shrink-0 rounded-full" style={{
backgroundColor: PIE_COLORS[index % PIE_COLORS.length] }} />
          <span className="truncate text-on-surface-variant">{category.name}</span>
        </div>
        <span className="font-semibold text-on-surface">{category.count}
SKUs</span>
      </div>
    })}
  </div>
} : null

```



```

    )}
  </div>
</>
): (
  <div className="flex h-56 items-center justify-center rounded-xl bg-
surface-container-low text-sm text-on-surface-variant">
    No data yet
  </div>
)}
</div>

{alerts && alerts.length > 0 ? (
  <div className="rounded-xl border border-tertiary/10 bg-tertiary-fixed/20
p-5 shadow-sm">
    <div className="mb-4 flex items-center gap-3">
      <div className="flex h-9 w-9 items-center justify-center rounded-xl bg-
tertiary text-white">
        <span className="material-symbols-outlined text-
[18px]">warning</span>
      </div>
      <div className="min-w-0">
        <h3 className="text-sm font-semibold text-on-surface">Stock
Alerts</h3>
        <p className="text-xs text-on-surface-variant">Items needing attention
right now</p>
      </div>

```

```

        <StatusBadge variant="danger" className="ml-
auto">{alerts.length}</StatusBadge>

    </div>

    <div className="space-y-3">

        {alerts.slice(0, 3).map((alert, index) => {

            <div key={alert.id || alert.product?.id || `alert-${index}`}
className="rounded-lg bg-surface-container-lowest/60 p-3">

                <p className="text-sm font-medium text-on-
surface">{alert.product?.name || 'Product alert'}</p>

                <p className="mt-1 text-xs leading-5 text-on-surface-
variant">{alert.message}</p>

            </div>

        })}

    </div>

    <a href="/transactions" className="mt-4 inline-flex items-center text-xs
font-semibold text-primary hover:underline">

        Manage alerts

    </a>

</div>

) : null}

</div>

</div>

<div className="grid grid-cols-1 gap-6 xl:grid-cols-3">

    <div className="rounded-xl bg-surface-container-lowest p-6 shadow-sm ring-
1 ring-outline-variant/20 xl:col-span-2">

```

```

<div className="mb-5 flex flex-col gap-3 sm:flex-row sm:items-center
sm:justify-between">

  <div>

    <h3 className="text-lg font-semibold text-on-surface">Recent
Transactions</h3>

    <p className="mt-1 text-sm text-on-surface-variant">Latest inventory
movements across your workspace</p>

  </div>

  <a href="/transactions" className="text-sm font-semibold text-primary
hover:underline">View all</a>

</div>

{isLoading ? (

  <div className="space-y-3">

    {[...Array(5)].map((_, index) => <div key={`recent-skeleton-${index}`}
className="skeleton h-14 rounded-xl" />)}

  </div>

) : recent.length > 0 ? (

  <div className="space-y-3">

    {recent.map((tx, index) => (

      <div

        key={tx.id || `${tx.product?.id} || tx.product?.name || 'transaction'`-
`${index}`}

        className="flex items-center gap-4 rounded-xl border border-
transparent px-3 py-3 transition-colors duration-200 hover:bg-surface-container-
low"

      >

        <div className={`flex h-10 w-10 flex-shrink-0 items-center justify-center
rounded-xl ${`

```

```

      tx.type === 'IN' ? 'bg-secondary-container/20 text-secondary' : 'bg-
primary-fixed/40 text-primary'
    }}>

    <span className="material-symbols-outlined text-[18px]">{tx.type ===
'IN' ? 'add' : 'remove'}</span>

  </div>

  <div className="min-w-0 flex-1">

    <p className="truncate text-sm font-semibold text-on-
surface">{tx.product?.name || 'Product'}</p>

    <p className="truncate text-xs text-on-surface-variant">{tx.reason ||
'Stock adjustment'}</p>

  </div>

  <div className="text-right">

    <p className={`text-sm font-bold ${tx.type === 'IN' ? 'text-secondary' :
'text-primary'}`}>

      {tx.type === 'IN' ? '+' : '-'}{tx.quantity}

    </p>

    <p className="text-xs text-on-surface-variant">

      {tx.created_at ? format(parseISO(tx.created_at), 'MMM d') : '-'}

    </p>

  </div>

</div>

)}}

</div>

):(

  <p className="py-10 text-center text-sm text-on-surface-variant">No
recent transactions</p>

```

```

    }}
  </div>

  <div className="rounded-xl bg-surface-container-lowest p-5 shadow-sm ring-
1 ring-outline-variant/20">

    <div className="mb-4 flex items-center gap-3">

      <div className="flex h-9 w-9 items-center justify-center rounded-xl bg-
primary text-white">

        <span className="material-symbols-outlined text-[18px]" style={{
fontVariationSettings: "'FILL' 1" }}>smart_toy</span>

      </div>

      <div>

        <h3 className="text-sm font-semibold text-on-surface">AI Analysis</h3>

        <p className="text-xs text-on-surface-variant">Quick recommendations
from current dashboard data</p>

      </div>

    </div>

    {insightsMutation.isPending ? (

      <div className="space-y-3">

        {[...Array(3)].map((_, index) => <div key={`insight-skeleton-${index}`}
className="skeleton h-20 rounded-xl" />)}

      </div>

    ) : aiInsights ? (

      <div className="space-y-3">

        {aiInsights.map((insight, index) => (

```

```

      <InsightCard key={insight?.id || insight?.title || `insight-card-${index}`}
insight={insight} index={index} />
    )}}
  </div>
): {
  <div className="rounded-xl bg-surface-container-low p-4 text-center">
    <p className="mb-3 text-sm leading-6 text-on-surface-variant">
      Generate a quick read on performance, risks, and opportunities from your
      latest inventory data.
    </p>
    <button
      onClick={() => insightsMutation.mutate()}
      disabled={!dashData}
      className="text-sm font-semibold text-primary transition-opacity
      hover:opacity-80 disabled:opacity-40"
    >
      Generate insights
    </button>
  </div>
  )}
</div>
</div>
</div>
</div>
);

```

Login.jsx

```
import { useState } from 'react'

import { Link, useNavigate } from 'react-router-dom'

import { useMutation } from '@tanstack/react-query'

import toast from 'react-hot-toast'

import { authApi } from '../services/api'

import { useAuthStore } from '../store/authStore'

export default function Login() {

  const [form, setForm] = useState({ email: "", password: "" })

  const [showForgotPassword, setShowForgotPassword] = useState(false)

  const [resetEmail, setResetEmail] = useState("")

  const { login } = useAuthStore()

  const navigate = useNavigate()

  const mutation = useMutation({

    mutationFn: () => authApi.login(form),

    onSuccess: (res) => {

      login(res.data.user, res.data.token)

      toast.success(`Welcome back, ${res.data.user.name}!`)

      navigate('/')

    },

    onError: (err) => {

      toast.error(err.response?.data?.error || 'Login failed')

    }

  })
```



```
}}
```

```
const forgotPasswordMutation = useMutation({  
  mutationFn: () => authApi.forgotPassword({ email: resetEmail }),  
  onSuccess: (res) => {  
    toast.success(res.data?.message || 'If your account exists, a reset link has been  
sent')  
    setShowForgotPassword(false)  
    setResetEmail('')  
  },  
  onError: (err) => {  
    const firstValidationError = err.response?.data?.errors?.[0]?.msg  
    toast.error(firstValidationError || err.response?.data?.error || 'Could not send  
reset email')  
  }  
})
```

```
const handleSubmit = (e) => {  
  e.preventDefault()  
  mutation.mutate()  
}
```

```
const handleForgotPasswordSubmit = (e) => {  
  e.preventDefault()  
  forgotPasswordMutation.mutate()
```



```
}
```

```
const toggleForgotPassword = () => {  
  setShowForgotPassword((current) => !current)  
  setResetEmail((current) => current || form.email)  
}
```

```
return (  
  <div className="min-h-screen bg-surface flex items-center justify-center p-4">  
    <div className="w-full max-w-md">  
      <div className="flex items-center gap-3 mb-10 justify-center">  
        <div className="w-12 h-12 rounded-2xl bg-primary-container flex items-center justify-center text-white shadow-editorial shrink-0">  
          <span className="material-symbols-outlined text-2xl">inventory_2</span>  
        </div>  
        <div>  
          <h1 className="text-xl font-bold text-on-surface tracking-tight">StockGrid</h1>  
          <p className="text-[10px] uppercase tracking-widest text-on-surface-variant font-bold">Inventory Intelligence</p>  
        </div>  
      </div>  
      <div className="bg-surface-container-lowest rounded-2xl shadow-editorial p-6 sm:p-8">  
        <h2 className="text-2xl font-bold text-on-surface mb-1">Welcome back</h2>
```

```
<p className="text-sm text-on-surface-variant mb-8">Sign in to your  
inventory workspace</p>
```

```
<form onSubmit={handleSubmit} className="space-y-5">
```

```
<div>
```

```
<label className="block text-xs font-bold text-on-surface-variant uppercase  
tracking-wider mb-2">Email</label>
```

```
<input
```

```
  type="email"
```

```
  required
```

```
  value={form.email}
```

```
  onChange={(e) => setForm({ ...form, email: e.target.value })}
```

```
  className="w-full bg-surface-container-low text-on-surface border-none  
rounded-xl px-4 py-3 text-sm focus:ring-2 focus:ring-primary/30 outline-none  
transition-all"
```

```
  placeholder="you@company.com"
```

```
</div>
```

```
<div>
```

```
<label className="block text-xs font-bold text-on-surface-variant uppercase  
tracking-wider mb-2">Password</label>
```

```
<input
```

```
  type="password"
```

```
  required
```

```
  value={form.password}
```

```
  onChange={(e) => setForm({ ...form, password: e.target.value })}
```

```
className="w-full bg-surface-container-low text-on-surface border-none
rounded-xl px-4 py-3 text-sm focus:ring-2 focus:ring-primary/30 outline-none
transition-all"
```

```
placeholder="Enter your password"
```

```
/>
```

```
</div>
```

```
<div className="flex justify-end">
```

```
<button
```

```
type="button"
```

```
onClick={toggleForgotPassword}
```

```
className="text-sm font-semibold text-primary hover:underline"
```

```
>
```

```
Forgot password?
```

```
</button>
```

```
</div>
```

```
<button
```

```
type="submit"
```

```
disabled={mutation.isPending}
```

```
className="w-full bg-gradient-to-br from-primary-container to-primary
text-white py-3 rounded-xl font-semibold text-sm shadow-editorial hover:opacity-
90 active:scale-95 transition-all disabled:opacity-60"
```

```
>
```

```
{mutation.isPending ? 'Signing in...' : 'Sign In'}
```

```
</button>
```

```
</form>
```

```
{showForgotPassword && {
```

```
  <div className="mt-6 rounded-2xl border border-outline-variant/30 bg-  
  surface-container-low p-5">
```

```
    <h3 className="text-base font-bold text-on-surface mb-1">Reset  
    password</h3>
```

```
    <p className="text-sm text-on-surface-variant mb-4">Enter your email and  
    we will send you a secure password reset link.</p>
```

```
  <form onSubmit={handleForgotPasswordSubmit} className="space-y-4">
```

```
    <div>
```

```
      <label className="block text-xs font-bold text-on-surface-variant  
      uppercase tracking-wider mb-2">Email</label>
```

```
      <input
```

```
        type="email"
```

```
        required
```

```
        value={resetEmail}
```

```
        onChange={(e) => setResetEmail(e.target.value)}
```

```
        className="w-full bg-surface-container-low text-on-surface border  
        border-outline-variant/30 rounded-xl px-4 py-3 text-sm focus:ring-2 focus:ring-  
        primary/30 outline-none transition-all"
```

```
        placeholder="you@company.com"
```

```
      />
```

```
    </div>
```

```
  <div className="flex gap-3">
```

```

<button
  type="button"
  onClick={() => setShowForgotPassword(false)}

  className="flex-1 rounded-xl bg-surface-container-lowest border
border-outline-variant/30 py-3 text-sm font-semibold text-on-surface hover:bg-
surface-container transition-colors"

  >
    Cancel
  </button>

  <button
    type="submit"
    disabled={forgotPasswordMutation.isPending}

    className="flex-1 bg-gradient-to-br from-primary-container to-primary
text-white py-3 rounded-xl font-semibold text-sm shadow-editorial hover:opacity-
90 transition-all disabled:opacity-60"

    >
      {forgotPasswordMutation.isPending ? 'Sending...' : 'Send Reset Link'}
    </button>
  </div>
</form>
</div>
}}

```

```

<p className="text-center text-sm text-on-surface-variant mt-6">
  Don't have an account?{' '}

  <Link to="/register" className="text-primary font-semibold
hover:underline">

```

```
Create workspace  
</Link>  
</p>  
</div>  
</div>  
</div>  
)  
}
```



Products.jsx

```
import { useMemo, useState } from 'react';

import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';

import toast from 'react-hot-toast';

import { productsApi, suppliersApi } from '../services/api';

import { useAuthStore } from '../store/authStore';

import ProductModal from '../components/modals/ProductModal';

import TransactionModal from '../components/modals/TransactionModal';

import BulkTransactionModal from
'../components/modals/BulkTransactionModal';

import ImportModal from '../components/modals/ImportModal';

import CategoryBadge from '../components/ui/CategoryBadge';

import StatusBadge from '../components/ui/StatusBadge';

import StockBar from '../components/ui/StockBar';

import { formatCurrencyINR } from '../utils/currency';

const STATUS_LABELS = { in_stock: 'In Stock', low: 'Low Stock', out_of_stock: 'Out of
Stock' };

const STATUS_VARIANTS = { in_stock: 'success', low: 'warning', out_of_stock:
'danger' };

function TableHeader({ children, className = '' }) {

  return (

    <th className={`px-4 py-4 text-left text-xs font-semibold uppercase tracking-
[0.14em] text-on-surface-variant ${className}`.trim()}>

      {children}

    </th>

  )
}
```



```
</th>
);
}
```

```
export default function Products() {
  const { user } = useAuthStore();
  const qc = useQueryClient();
  const canEdit = ['admin', 'manager'].includes(user?.role);

  const [filters, setFilters] = useState({ search: "", category: "", stock_status: "",
supplier_id: "" });
  const [page, setPage] = useState(1);
  const [productModal, setProductModal] = useState(null);
  const [txModal, setTxModal] = useState(null);
  const [bulkTxModal, setBulkTxModal] = useState(false);
  const [importModal, setImportModal] = useState(false);
  const [selected, setSelected] = useState([]);

  const { data, isLoading } = useQuery({
    queryKey: ['products', filters, page],
    queryFn: () => productsApi.list({ ...filters, page, limit: 20 }).then((r) => r.data),
    keepPreviousData: true,
  });

  const { data: suppliersData } = useQuery({
```



```
queryKey: ['suppliers'],
queryFn: () => suppliersApi.list().then((r) => r.data.suppliers),
});

const { data: categoriesData } = useQuery({
  queryKey: ['categories'],
  queryFn: () => productsApi.categories().then((r) => r.data.categories),
});

const deleteMutation = useMutation({
  mutationFn: (id) => productsApi.delete(id),
  onSuccess: () => {
    qc.invalidateQueries(['products']);
    toast.success('Product deleted');
  },
  onError: () => toast.error('Delete failed'),
});

const bulkDeleteMutation = useMutation({
  mutationFn: (ids) => productsApi.bulkDelete(ids),
  onSuccess: (res) => {
    qc.invalidateQueries(['products']);
    toast.success(res.data.message);
    setSelected([]);
  },
});
```

```

    onError: () => toast.error('Bulk delete failed'),
  });

  const products = data?.products || [];

  const pagination = data?.pagination || {};

  const allVisibleSelected = products.length > 0 && selected.length ===
products.length;

  const pageButtons = useMemo(() => {
    const totalPages = pagination.pages || 0;
    return Array.from({ length: Math.min(5, totalPages) }, (_, index) => index + 1);
  }, [pagination.pages]);

  const handleBulkDelete = () => {
    if (confirm(`Are you sure you want to delete ${selected.length} products?`)) {
      bulkDeleteMutation.mutate(selected);
    }
  };

  const toggleSelect = (id) => {
    setSelected((prev) => (prev.includes(id) ? prev.filter((x) => x !== id) : [...prev,
id]));
  };

  const handleExport = async () => {

```

```
try {  
  const res = await productsApi.list({ ...filters, limit: 9999 });  
  const prods = res.data.products;  
  const rows = prods.map((product) => ({  
    Name: product.name,  
    SKU: product.sku,  
    Category: product.category,  
    Price: product.price,  
    Stock: product.current_stock,  
    'Min Stock': product.min_stock,  
    Status: STATUS_LABELS[product.stock_status] || product.stock_status,  
  }));  
  const headers = Object.keys(rows[0] || {});  
  const csv = [headers.join(','), ...rows.map((row) => headers.map((header) =>  
    `${row[header] ?? ''}`).join(',')].join("\n");  
  const blob = new Blob([csv], { type: 'text/csv' });  
  const url = URL.createObjectURL(blob);  
  const anchor = document.createElement('a');  
  anchor.href = url;  
  anchor.download = 'products.csv';  
  anchor.click();  
  URL.revokeObjectURL(url);  
} catch {  
  toast.error('Export failed');  
}
```

```

};

return (
  <div className="container mx-auto px-6 py-8">
    <div className="space-y-6">
      <div className="flex flex-col gap-4 lg:flex-row lg:items-end lg:justify-between">
        <div>
          <nav className="mb-2 flex items-center gap-1 text-xs font-semibold uppercase tracking-[0.14em] text-on-surface-variant">
            <span>Inventory</span>
            <span className="material-symbols-outlined text-[14px]">chevron_right</span>
            <span className="text-primary">Products</span>
          </nav>
          <h2 className="text-3xl font-bold tracking-tight text-on-surface">Products</h2>
          <p className="mt-1 text-sm text-on-surface-variant">Track, update, and review the items in your catalog.</p>
        </div>

        <div className="flex flex-wrap items-center gap-3">
          <button
            onClick={() => setImportModal(true)}

            className="inline-flex cursor-pointer items-center gap-2 rounded-xl bg-surface-container-highest px-5 py-2.5 text-sm font-semibold text-on-surface shadow-sm transition-all duration-200 hover:-translate-y-0.5 hover:shadow-md"

```

```

>
<span className="material-symbols-outlined text-lg">upload_file</span>
  Import
</button>
<button
  onClick={handleExport}
  className="inline-flex cursor-pointer items-center gap-2 rounded-xl bg-
surface-container-highest px-5 py-2.5 text-sm font-semibold text-on-surface
shadow-sm transition-all duration-200 hover:-translate-y-0.5 hover:shadow-md"
>
  <span className="material-symbols-outlined text-lg">download</span>
  Export
</button>
{canEdit ? (
  <button
    onClick={() => setProductModal('new')}
    className="inline-flex cursor-pointer items-center gap-2 rounded-xl bg-
gradient-to-br from-primary-container to-primary px-6 py-2.5 text-sm font-
semibold text-white shadow-sm transition-all duration-200 hover:-translate-y-0.5
hover:shadow-md"
    >
      <span className="material-symbols-outlined text-lg">add</span>
      Add Product
    </button>
  ) : null}
</div>
</div>

```

```

<div className="grid grid-cols-1 gap-4 lg:grid-cols-12">

  <div className="lg:col-span-4">

    <div className="flex items-center gap-2 rounded-xl bg-surface-container-
low px-4 py-3 ring-1 ring-outline-variant/20">

      <span className="material-symbols-outlined text-[18px] text-on-surface-
variant">search</span>

      <input

        type="text"

        placeholder="Search name or SKU..."

        value={filters.search}

        onChange={(e) => { setFilters({ ...filters, search: e.target.value });
setPage(1); }}

        className="w-full bg-transparent text-sm text-on-surface outline-none
placeholder:text-on-surface-variant"

      />

    </div>

  </div>

  <div className="lg:col-span-2">

    <select

      value={filters.category}

      onChange={(e) => { setFilters({ ...filters, category: e.target.value });
setPage(1); }}

      className="w-full rounded-xl bg-surface-container-low px-4 py-3 text-sm
text-on-surface outline-none ring-1 ring-outline-variant/20"

    >

      <option value="">All Categories</option>

```

```

      {(categoriesData || []).map((category, index) => (
        <option key={category || `category-option-${index}`}
value={category}>{category}</option>
      ))}
    </select>
  </div>

  <div className="lg:col-span-3">
    <select
      value={filters.stock_status}
      onChange={(e) => { setFilters({ ...filters, stock_status: e.target.value });
setPage(1); }}
      className="w-full rounded-xl bg-surface-container-low px-4 py-3 text-sm
text-on-surface outline-none ring-1 ring-outline-variant/20"
    >
      <option value="">All Status</option>
      <option value="in_stock">In Stock</option>
      <option value="low">Low Stock</option>
      <option value="out_of_stock">Out of Stock</option>
    </select>
  </div>

  <div className="lg:col-span-3">
    <select
      value={filters.supplier_id}
      onChange={(e) => { setFilters({ ...filters, supplier_id: e.target.value });
setPage(1); }}
      className="w-full rounded-xl bg-surface-container-low px-4 py-3 text-sm
text-on-surface outline-none ring-1 ring-outline-variant/20"

```



```

>
<option value="">All Suppliers</option>

{{suppliersData || []}.map((supplier, index) => (
  <option key={supplier.id || `${supplier.name}-${index}`}
value={supplier.id}>{supplier.name}</option>
))}
</select>
</div>
</div>

{selected.length > 0 ? (
  <div className="flex flex-col gap-4 rounded-xl border border-secondary/10
bg-secondary-container/20 px-5 py-4 shadow-sm lg:flex-row lg:items-center
lg:justify-between">
    <div className="flex items-center gap-3 text-sm font-semibold text-on-
secondary-container">
      <span className="material-symbols-outlined text-
secondary">library_add_check</span>
      {selected.length} product(s) selected
    </div>
    <div className="flex flex-wrap items-center gap-2">
      <button onClick={() => setBulkTxModal(true)} className="rounded-lg bg-
secondary px-4 py-2 text-sm font-semibold text-white shadow-sm transition
hover:bg-secondary/90">
        Bulk Stock Update
      </button>

```



```

        <button onClick={() => setSelected([])} className="rounded-lg border
border-outline-variant/40 bg-surface-container-lowest px-4 py-2 text-sm font-
semibold text-on-surface-variant transition hover:bg-surface-container-low">

        Clear

    </button>

    {canEdit ? (

        <button

            onClick={handleBulkDelete}

            className="inline-flex h-10 w-10 items-center justify-center rounded-lg
bg-tertiary/10 text-tertiary transition hover:bg-tertiary hover:text-white"

            title="Delete Selected"

        >

            <span className="material-symbols-outlined">delete</span>

        </button>

    ) : null}

</div>

</div>

) : null}

```

```

<div className="overflow-hidden rounded-xl bg-surface-container-lowest
shadow-sm ring-1 ring-outline-variant/20">

    <div className="overflow-x-auto">

        <table className="min-w-full text-left">

            <thead className="bg-surface-container-low">

                <tr>

                    <TableHeader className="w-14">

```

```

<div className="flex items-center justify-center">
  <input
    type="checkbox"
    className="h-4 w-4 rounded border-outline-variant text-primary
focus:ring-primary"
    checked={allVisibleSelected}
    onChange={(e) => setSelected(e.target.checked ?
products.map((product) => product.id) : [])}
  />
</div>
</TableHeader>
<TableHeader className="min-w-[16rem]">Product</TableHeader>
<TableHeader className="min-w-[10rem]">Category</TableHeader>
<TableHeader className="min-w-[10rem]">Supplier</TableHeader>
<TableHeader className="min-w-[12rem]">Stock Level</TableHeader>
<TableHeader className="min-w-[8rem]">Unit Price</TableHeader>
<TableHeader className="min-w-[9rem]">Status</TableHeader>
<TableHeader className="w-28 text-right">Actions</TableHeader>
</tr>
</thead>
<tbody className="divide-y divide-surface-container-low">
  {isLoading ? (
    [...Array(8)].map((_, rowIndex) => (
      <tr key={`product-skeleton-${rowIndex}`}>
        {[...Array(8)].map((_, cellIndex) => (

```

```

        <td key={`product-skeleton-${rowIndex}-${cellIndex}`}
className="px-4 py-5">
            <div className="skeleton h-4 rounded" />
        </td>
    )}
</tr>
))
): products.length === 0 ? (
    <tr>
        <td colspan={8} className="px-6 py-16 text-center text-sm text-on-
surface-variant">
            <span className="material-symbols-outlined mb-3 block text-4xl text-
outline">inventory_2</span>
            No products found.
            {canEdit ? (
                <button onClick={() => setProductModal('new')} className="ml-1
font-semibold text-primary hover:underline">
                    Add your first product
                </button>
            ) : null}
        </td>
    </tr>
): (
    products.map((product, index) => (
        <tr key={product.id || `${product.name}-${index}`} className="group
transition hover:bg-surface-container-low/70">
            <td className="px-4 py-5 align-middle">

```

```

<div className="flex items-center justify-center">
  <input
    type="checkbox"
    checked={selected.includes(product.id)}
    onChange={() => toggleSelect(product.id)}
    className="h-4 w-4 rounded border-outline-variant text-primary
focus:ring-primary"
  />
</div>
</td>
<td className="px-4 py-5 align-middle">
  <div className="flex items-center gap-4">
    <div className="flex h-12 w-12 flex-shrink-0 items-center justify-
center rounded-xl bg-surface-container text-on-surface-variant">
      <span className="material-symbols-outlined">inventory_2</span>
    </div>
    <div className="min-w-0">
      <p className="max-w-[16rem] truncate text-sm font-semibold text-
on-surface" title={product.name}>
        {product.name}
      </p>
      <p className="mt-1 max-w-[16rem] truncate text-xs font-medium
text-on-surface-variant" title={product.sku}>
        SKU: <span className="font-mono">{product.sku}</span>
      </p>
    </div>
  </div>

```

```

        </div>
    </td>

    <td className="px-4 py-5 align-middle">

        <CategoryBadge>{product.category}</CategoryBadge>

    </td>

    <td className="px-4 py-5 align-middle">

        <p className="max-w-[10rem] truncate text-sm font-medium text-on-
surface" title={product.supplier?.name || product.supplier_name || '-'}>

            {product.supplier?.name || product.supplier_name || '-'}

        </p>
    </td>

    <td className="px-4 py-5 align-middle">

        <StockBar current={product.current_stock} min={product.min_stock}
status={product.stock_status} unit={product.unit || 'units'} />

    </td>

    <td className="px-4 py-5 align-middle text-sm font-semibold text-on-
surface">

        {formatCurrencyINR(product.price || 0)}

    </td>

    <td className="px-4 py-5 align-middle">

        <StatusBadge variant={STATUS_VARIANTS[product.stock_status] ||
'neutral'}>

            <span className="h-1.5 w-1.5 rounded-full bg-current opacity-80" />

            {STATUS_LABELS[product.stock_status] || product.stock_status}

        </StatusBadge>

    </td>

```

```

<td className="px-4 py-5 align-middle">

  <div className="flex items-center justify-end gap-1 opacity-100
transition-opacity lg:opacity-0 lg:group-hover:opacity-100">

    <button

      onClick={() => setTxModal(product)}

      className="cursor-pointer rounded-lg p-2 text-secondary transition-
colors hover:bg-secondary-container/20"

      title="Stock movement"

    >

      <span className="material-symbols-outlined text-
[18px]">swap_vert</span>

    </button>

    {canEdit ? (

      <>

        <button

          onClick={() => setProductModal(product)}

          className="cursor-pointer rounded-lg p-2 text-on-surface-variant
transition-colors hover:bg-surface-container-high"

          title="Edit product"

        >

          <span className="material-symbols-outlined text-
[18px]">edit</span>

        </button>

        <button

          onClick={() => {

            if (confirm(`Delete "${product.name}"`))
deleteMutation.mutate(product.id);

```

```

    }}

    className="cursor-pointer rounded-lg p-2 text-tertiary transition-
colors hover:bg-tertiary-fixed/20"

    title="Delete product"

    >

    <span className="material-symbols-outlined text-
[18px]">delete</span>

    </button>

  </>
) : null}
</div>
</td>
</tr>
))
}}
</tbody>
</table>
</div>

```

```

{pagination.pages > 1 ? {
  <div className="flex flex-col gap-3 border-t border-surface-container-low
px-6 py-4 sm:flex-row sm:items-center sm:justify-between">

    <p className="text-xs text-on-surface-variant">

      Showing {(page - 1) * 20 + 1} to {Math.min(page * 20, pagination.total)} of
{pagination.total}

    </p>

```



```

<div className="flex items-center gap-1">

  <button
    onClick={() => setPage((p) => Math.max(1, p - 1))}
    disabled={page === 1}
    className="rounded-lg p-2 text-on-surface-variant transition-colors
    hover:bg-surface-container-high disabled:opacity-40"
  >

    <span className="material-symbols-outlined text-[18px]">chevron_left</span>

  </button>

  {pageButtons.map((pageNumber) => (
    <button
      key={`page-${pageNumber}`}
      onClick={() => setPage(pageNumber)}
      className={`h-9 w-9 rounded-lg text-xs font-semibold transition-colors
    ${
      page === pageNumber ? 'bg-primary text-white' : 'text-on-surface-variant hover:bg-surface-container-high'
    }`}
    >

      {pageNumber}

    </button>

  )]}

  <button
    onClick={() => setPage((p) => Math.min(pagination.pages, p + 1))}
    disabled={page === pagination.pages}

```



```
        className="rounded-lg p-2 text-on-surface-variant transition-colors
        hover:bg-surface-container-high disabled:opacity-40"
```

```
    >
```

```
        <span className="material-symbols-outlined text-
        [18px]">chevron_right</span>
```

```
    </button>
```

```
</div>
```

```
</div>
```

```
    ) : null}
```

```
</div>
```

```
</div>
```

```
{productModal !== null ? (
```

```
    <ProductModal
```

```
        product={productModal === 'new' ? null : productModal}
```

```
        suppliers={suppliersData || []}
```

```
        onClose={() => setProductModal(null)}
```

```
        onSave={() => { qc.invalidateQueries(['products']); setProductModal(null); }}
```

```
    />
```

```
) : null}
```

```
{txModal ? (
```

```
    <TransactionModal
```

```
        product={txModal}
```

```
        onClose={() => setTxModal(null)}
```

```
        onSave={() => {
```

```

      qc.invalidateQueries({ queryKey: ['products'] });
      qc.invalidateQueries({ queryKey: ['alerts'] });
      qc.invalidateQueries({ queryKey: ['dashboard'] });
      setTxModal(null);
    }}
  />
) : null}
{bulkTxModal ? (
  <BulkTransactionModal
    products={products.filter((product) => selected.includes(product.id))}
    onClose={() => setBulkTxModal(false)}
    onSave={() => {
      qc.invalidateQueries({ queryKey: ['products'] });
      qc.invalidateQueries({ queryKey: ['alerts'] });
      qc.invalidateQueries({ queryKey: ['dashboard'] });
      setBulkTxModal(false);
      setSelected([]);
    }}
  />
) : null}
{importModal ? (
  <ImportModal
    onClose={() => setImportModal(false)}
    onImported={() => { qc.invalidateQueries(['products']);
setImportModal(false); }}

```

```
    />  
    ): null}  
</div>  
);  
}
```



Register.jsx

```
import { useState } from 'react'

import { Link, useNavigate } from 'react-router-dom'

import { useMutation } from '@tanstack/react-query'

import toast from 'react-hot-toast'

import { authApi } from '../services/api'

import { useAuthStore } from '../store/authStore'

export default function Register() {

  const [form, setForm] = useState({ name: "", email: "", password: "",
  organizationName: "" })

  const { login } = useAuthStore()

  const navigate = useNavigate()

  const mutation = useMutation({

    mutationFn: () => authApi.register(form),

    onSuccess: (res) => {

      login(res.data.user, res.data.token)

      toast.success('Workspace created! Welcome aboard.')

      navigate('/')

    },

    onError: (err) => {

      toast.error(err.response?.data?.error || err.response?.data?.errors?.[0]?.msg ||
'Registration failed')

    }

  })

}
```

```
}}
```

```
const handleSubmit = (e) => {  
  e.preventDefault()  
  mutation.mutate()  
}
```

```
return (  
  <div className="min-h-screen bg-surface flex items-center justify-center p-4">  
    <div className="w-full max-w-md">  
      <div className="flex items-center gap-3 mb-10 justify-center">  
        <div className="w-12 h-12 rounded-2xl bg-primary-container flex items-center justify-center text-white shadow-editorial shrink-0">  
          <span className="material-symbols-outlined text-2xl">inventory_2</span>  
        </div>  
        <div>  
          <h1 className="text-xl font-bold text-on-surface tracking-tight">StockGrid</h1>  
          <p className="text-[10px] uppercase tracking-widest text-on-surface-variant font-bold">Inventory Intelligence</p>  
        </div>  
      </div>  
      <div className="bg-surface-container-lowest rounded-2xl shadow-editorial p-6 sm:p-8">  
        <h2 className="text-2xl font-bold text-on-surface mb-1">Create your workspace</h2>
```

```
<p className="text-sm text-on-surface-variant mb-8">Set up your inventory management system</p>
```

```
<form onSubmit={handleSubmit} className="space-y-4">

  {[

    { key: 'organizationName', label: 'Company Name', type: 'text', placeholder: 'Acme Corp' },

    { key: 'name', label: 'Your Name', type: 'text', placeholder: 'Alex Rivera' },

    { key: 'email', label: 'Work Email', type: 'email', placeholder: 'you@company.com' },

    { key: 'password', label: 'Password', type: 'password', placeholder: '8+ characters' },

  ].map(field => (

    <div key={field.key}>

      <label className="block text-xs font-bold text-on-surface-variant uppercase tracking-wider mb-2">
        {field.label}
      </label>

      <input

        type={field.type}

        required

        value={form[field.key]}

        onChange={e => setForm({ ...form, [field.key]: e.target.value })}

        placeholder={field.placeholder}

        className="w-full bg-surface-container-low text-on-surface border-none rounded-xl px-4 py-3 text-sm focus:ring-2 focus:ring-primary/30 outline-none transition-all"

      />

    </div>

  ))}

</form>
```

```

    />
  </div>
)}}

<button
  type="submit"
  disabled={mutation.isPending}
  className="w-full bg-gradient-to-br from-primary-container to-primary
text-white py-3 rounded-xl font-semibold text-sm shadow-editorial hover:opacity-
90 active:scale-95 transition-all disabled:opacity-60 mt-2"
  >
    {mutation.isPending ? 'Creating workspace...' : 'Create Workspace'}
  </button>
</form>
<p className="text-center text-sm text-on-surface-variant mt-6">
  Already have an account?{' '}
  <Link to="/login" className="text-primary font-semibold hover:underline">
    Sign in
  </Link>
</p>
</div>
</div>
)
}

```

Reports.jsx

```
import { useState } from 'react'

import { useQuery } from '@tanstack/react-query'

import {
  BarChart, Bar, XAxis, YAxis, CartesianGrid, Tooltip, ResponsiveContainer,
  PieChart, Pie, Cell
} from 'recharts'

import toast from 'react-hot-toast'

import { reportsApi } from '../services/api'

import { formatCompactCurrencyINR, formatCurrencyINR } from '../utils/currency'

const COLORS = ['#3525cd', '#006c49', '#960014', '#4d44e3', '#4edea3', '#ffb3ad',
  '#c3c0ff', '#6cf8bb']

function StatCard({ label, value, sub, icon, color = 'primary' }) {
  const colorMap = {
    primary: 'bg-primary-fixed/30 text-primary',
    green: 'bg-secondary-container/20 text-secondary',
    red: 'bg-tertiary-fixed/30 text-tertiary',
    purple: 'bg-primary-fixed/40 text-primary',
  }

  return (
    <div className="bg-surface-container-lowest rounded-xl editorial-shadow p-6">
      <div className="flex items-start justify-between mb-3">
```



```

    <div className={`w-10 h-10 rounded-xl flex items-center justify-center
    ${colorMap[color]}`}>
      <span className="material-symbols-outlined text-[20px]">{icon}</span>
    </div>
  </div>
  <h3 className="text-2xl font-bold text-on-surface mb-0.5">{value}</h3>
  <p className="text-xs font-bold text-on-surface-variant uppercase tracking-
  wider">{label}</p>
  {sub && <p className="text-xs text-on-surface-variant mt-1">{sub}</p>}
</div>
)
}

```

```

export default function Reports() {
  const [topDays, setTopDays] = useState(30)

  const { data: valuation, isLoading: valLoading } = useQuery({
    queryKey: ['valuation'],
    queryFn: () => reportsApi.valuation().then(r => r.data),
  })

  const { data: topProducts, isLoading: topLoading } = useQuery({
    queryKey: ['top-products', topDays],
    queryFn: () => reportsApi.topProducts({ days: topDays }).then(r => r.data),
  })

```

```
const { data: deadStock } = useQuery({  
  queryKey: ['dead-stock-report'],  
  queryFn: () => reportsApi.deadStock({ days: 60 }).then(r => r.data),  
})
```

```
const { data: dashboard } = useQuery({  
  queryKey: ['dashboard'],  
  queryFn: () => reportsApi.dashboard().then(r => r.data),  
  staleTime: 30000,  
})
```

```
const handleExport = (type) => {  
  const format = 'csv'  
  const params = new URLSearchParams({ format })  
  window.open(`/api/export/${type}?${params}`, '_blank')  
  toast.success(`Exporting ${type}...`)  
}
```

```
const valItems = valuation?.valuation || []  
const topItems = topProducts?.top_products || []  
const deadItems = deadStock?.dead_stock || []  
const catData = dashboard?.category_distribution || []  
  
const totalValue = valuation?.total_value || 0
```

```

const outOfStock = valItems.filter(p => p.status === 'out_of_stock').length
const lowStock = valItems.filter(p => p.status === 'low_stock').length
const inStock = valItems.filter(p => p.status === 'in_stock').length
const totalProfit = valItems.reduce((acc, p) => acc + (p.total_profit || 0), 0)

return (
  <div className="px-8 py-8">
    <div className="flex items-end justify-between mb-8">
      <div>
        <h2 className="text-3xl font-bold tracking-tight text-on-surface">Reports &
Analytics</h2>
        <p className="text-on-surface-variant mt-1 text-sm">Inventory intelligence
and performance insights</p>
      </div>
      <div className="flex gap-3">
        <button
          onClick={() => handleExport('products')}
          className="flex items-center gap-2 px-4 py-2.5 bg-surface-container-highest
text-on-surface rounded-xl text-sm font-semibold hover:bg-surface-container-high
transition-colors"
          >
            <span className="material-symbols-outlined text-
[18px]">download</span>
            Export Products
        </button>
        <button
          onClick={() => handleExport('transactions')}

```

```
className="flex items-center gap-2 px-4 py-2.5 bg-surface-container-highest
text-on-surface rounded-xl text-sm font-semibold hover:bg-surface-container-high
transition-colors"
```

```
>
```

```
<span className="material-symbols-outlined text-
[18px]">download</span>
```

```
Export Transactions
```

```
</button>
```

```
</div>
```

```
</div>
```

```
<div className="grid grid-cols-4 gap-6 mb-8">
```

```
<StatCard label="Total Inventory Value"
value={formatCompactCurrencyINR(totalValue)} icon="account_balance_wallet"
color="primary" sub={` ${valItems.length} SKUs tracked`} />
```

```
<StatCard label="In Stock" value={inStock} icon="check_circle" color="green"
sub="Healthy stock level" />
```

```
<StatCard label="Low Stock" value={lowStock} icon="warning" color="purple"
sub="Need attention" />
```

```
<StatCard label="Out of Stock" value={outOfStock} icon="do_not_disturb_on"
color="red" sub="Critical - reorder now" />
```

```
</div>
```

```
<div className="grid grid-cols-2 gap-6 mb-6">
```

```
<div className="bg-surface-container-lowest rounded-xl editorial-shadow p-
6">
```

```
<div className="flex items-center justify-between mb-5">
```

```
<div>
```

```

<h3 className="font-bold text-on-surface">Top Selling Products</h3>
<p className="text-xs text-on-surface-variant mt-0.5">By units sold</p>
</div>
<select
  value={topDays}
  onChange={e => setTopDays(Number(e.target.value))}
  className="bg-surface-container-low border-none rounded-lg px-3 py-1.5
text-xs font-semibold focus:ring-0 outline-none"
>
  <option value={7}>Last 7 days</option>
  <option value={30}>Last 30 days</option>
  <option value={90}>Last 90 days</option>
</select>
</div>
{topLoading ? (
  <div className="skeleton h-56 rounded-lg" />
) : topItems.length > 0 ? (
  <ResponsiveContainer width="100%" height={220}>
    <BarChart data={topItems.slice(0, 8)} layout="vertical" margin={{ left: 16
  }}>
      <CartesianGrid strokeDasharray="3 3" stroke="#edeef0" horizontal={false}
    />
      <XAxis type="number" tick={{ fontSize: 11, fill: '#464555' }} />
      <YAxis type="category" dataKey="name" tick={{ fontSize: 11, fill: '#464555'
  }} width={120} />
      <Tooltip formatter={(v) => [v, 'Units Sold']} />

```

```

        <Bar dataKey="total_sold" fill="#3525cd" radius={[0, 4, 4, 0]} />
      </BarChart>
    </ResponsiveContainer>
  ): (
    <div className="h-56 flex items-center justify-center text-sm text-on-
surface-variant">No sales data yet</div>
  )}
</div>

<div className="bg-surface-container-lowest rounded-xl editorial-shadow p-
6">
  <h3 className="font-bold text-on-surface mb-1">Stock by Category</h3>
  <p className="text-xs text-on-surface-variant mb-5">Value distribution</p>
  {catData.length > 0 ? (
    <div className="flex gap-6">
      <ResponsiveContainer width="50%" height={200}>
        <PieChart>
          <Pie data={catData} cx="50%" cy="50%" outerRadius={80}
dataKey="value" paddingAngle={2}>
            {catData.map((_, i) => <Cell key={i} fill={COLORS[i % COLORS.length]}
/>)}
          </Pie>
          <Tooltip formatter={v => [formatCurrencyINR(v), 'Value']} />
        </PieChart>
      </ResponsiveContainer>
      <div className="flex-1 space-y-2.5 self-center">

```

```

    {catData.map((cat, i) => (
      <div key={cat.name} className="flex items-center gap-2.5">
        <span className="w-3 h-3 rounded-sm flex-shrink-0" style={{
background: COLORS[i % COLORS.length] }} />
        <div className="flex-1 min-w-0">
          <p className="text-xs font-semibold text-on-surface
truncate">{cat.name}</p>
          <p className="text-[10px] text-on-surface-variant">{cat.count}
SKUs</p>
        </div>
      </div>
    )
  )}
</div>
</div>
): (
  <div className="h-56 flex items-center justify-center text-sm text-on-
surface-variant">No data yet</div>
  )}
</div>
</div>

<div className="bg-surface-container-lowest rounded-xl editorial-shadow
overflow-hidden mb-6">
  <div className="px-6 py-5 border-b border-surface-container-low flex items-
center justify-between">
    <div>

```



```
<h3 className="font-bold text-on-surface">Inventory Valuation & Profitability</h3>
```

```
<p className="text-xs text-on-surface-variant mt-0.5">Stock value and historical profit per product</p>
```

```
</div>
```

```
<div className="flex gap-6">
```

```
<span className="text-sm font-bold text-on-surface">Total Value: <span className="text-primary">{formatCurrencyINR(totalValue)}</span></span>
```

```
<span className="text-sm font-bold text-on-surface">Historical Profit: <span className="text-secondary">{formatCurrencyINR(totalProfit)}</span></span>
```

```
</div>
```

```
</div>
```

```
<div className="overflow-x-auto">
```

```
<table className="w-full text-left">
```

```
<thead>
```

```
<tr className="bg-surface-container-low">
```

```
{['Product', 'SKU', 'Category', 'Stock', 'Unit Price', 'Cost', 'Margin %', 'Profit', 'Status'].map(h => {
```

```
<th key={h} className="py-3.5 px-5 text-xs font-bold uppercase tracking-wider text-on-surface-variant whitespace-nowrap">{h}</th>
```

```
}}}
```

```
</tr>
```

```
</thead>
```

```
<tbody className="divide-y divide-surface-container-low">
```

```
{valLoading ? {
```

```
[...Array(8)].map((_, i) => {
```

```
<tr key={i}>
```



```

    {...Array(7)].map((_, j) => (
      <td key={j} className="py-4 px-5"><div className="skeleton h-4
rounded" /></td>
    )))
  </tr>
)
) : valItems.slice(0, 20).map(p => (
  <tr key={p.id} className="hover:bg-surface-container-low/50 transition-
colors">
    <td className="py-4 px-5 text-sm font-semibold text-on-
surface">{p.name}</td>
    <td className="py-4 px-5 text-xs font-mono text-on-surface-
variant">{p.sku}</td>
    <td className="py-4 px-5">
      <span className="px-2.5 py-1 rounded-lg bg-surface-container text-
[11px] font-bold text-on-surface-variant uppercase tracking-
wider">{p.category}</span>
    </td>
    <td className="py-4 px-5 text-sm font-bold text-on-
surface">{p.current_stock}</td>
    <td className="py-4 px-5 text-sm text-on-
surface">{formatCurrencyINR(p.unit_price)}</td>
    <td className="py-4 px-5 text-sm text-on-
surface">{formatCurrencyINR(p.cost_price || 0)}</td>
    <td className="py-4 px-5 text-sm font-bold text-on-
surface">{p.margin_pct}%</td>
    <td className="py-4 px-5 text-sm font-bold text-
secondary">{formatCurrencyINR(p.total_profit || 0)}</td>
    <td className="py-4 px-5">

```

```

        <span className={`inline-flex items-center gap-1.5 px-2.5 py-1 rounded-
full text-xs font-bold ${
            p.status === 'in_stock' ? 'bg-secondary-container/20 text-on-secondary-
container' :
            p.status === 'low_stock' ? 'bg-yellow-100 text-yellow-800' :
            'bg-tertiary-fixed/30 text-tertiary'
        }}>
        <span className="w-1.5 h-1.5 rounded-full bg-current opacity-70" />
        {p.status.replace('_', ' ')}
    </span>
</td>
</tr>
)}}
</tbody>
</table>
</div>
{valItems.length > 20 && (
    <div className="py-3 text-center text-xs text-on-surface-variant border-t
border-surface-container-low">
        Showing 20 of {valItems.length} products. Export CSV for full report.
    </div>
)}
</div>

{deadItems.length > 0 && (

```

```

<div className="bg-surface-container-lowest rounded-xl editorial-shadow
overflow-hidden">

  <div className="px-6 py-5 border-b border-surface-container-low flex items-
center gap-3">

    <span className="material-symbols-outlined text-
tertiary">inventory</span>

    <div>

      <h3 className="font-bold text-on-surface">Dead Stock (60+ days no
sales)</h3>

      <p className="text-xs text-on-surface-variant mt-0.5">

        {deadItems.length} items at risk:
        {formatCurrencyINR(deadItems.reduce((sum, item) => sum + item.value, 0))}

      </p>
    </div>
  </div>

  <div className="divide-y divide-surface-container-low">

    {deadItems.map(p => (

      <div key={p.id} className="flex items-center gap-4 px-6 py-4 hover:bg-
surface-container-low/50 transition-colors">

        <div className="flex-1">

          <p className="text-sm font-bold text-on-surface">{p.name}</p>

          <p className="text-xs text-on-surface-variant font-mono">{p.sku} -
{p.category}</p>

        </div>

        <div className="text-right">

          <p className="text-sm font-bold text-on-surface">{p.current_stock}
units</p>

```

```
<p className="text-xs text-tertiary font-  
semibold">{formatCurrencyINR(p.value)} at risk</p>
```

```
</div>
```

```
</div>
```

```
}}}
```

```
</div>
```

```
</div>
```

```
}}
```

```
</div>
```

```
)
```

```
}
```



ResetPassword.jsx

```
import { useMemo, useState } from 'react'

import { Link, useNavigate, useSearchParams } from 'react-router-dom'

import { useMutation } from '@tanstack/react-query'

import toast from 'react-hot-toast'

import { authApi } from '../services/api'

export default function ResetPassword() {
  const [searchParams] = useSearchParams()

  const navigate = useNavigate()

  const token = searchParams.get('token') || ''

  const [form, setForm] = useState({ newPassword: '', confirmPassword: '' })

  const hasToken = useMemo(() => Boolean(token), [token])

  const mutation = useMutation({
    mutationFn: () => authApi.resetPassword({ token, ...form }),
    onSuccess: (res) => {
      toast.success(res.data?.message || 'Password updated successfully')
      navigate('/login')
    },
    onError: (err) => {
      const firstValidationError = err.response?.data?.errors?.[0]?.msg

      toast.error(firstValidationError || err.response?.data?.error || 'Could not reset password')
    }
  })
}
```

```
}  
})
```

```
const handleSubmit = (e) => {  
  e.preventDefault()  
  mutation.mutate()  
}
```

```
return (  
  <div className="min-h-screen bg-gradient-to-br from-slate-100 via-indigo-50 to-slate-100 flex items-center justify-center p-4">  
    <div className="w-full max-w-md">  
      <div className="bg-white rounded-2xl shadow-editorial p-8">  
        <h1 className="text-2xl font-bold text-on-surface mb-1">Set a new password</h1>  
        <p className="text-sm text-on-surface-variant mb-8">Choose a strong password for your account.</p>  
  
        {!hasToken ? (  
          <div className="rounded-2xl border border-tertiary/20 bg-tertiary-fixed/10 p-5">  
            <p className="text-sm text-tertiary font-medium">This reset link is invalid or incomplete.</p>  
  
            <Link to="/login" className="mt-4 inline-flex text-sm font-semibold text-primary hover:underline">  
              Back to login  
</Link>
```

```

</div>

): {
  <form onSubmit={handleSubmit} className="space-y-5">

    <div>

      <label className="block text-xs font-bold text-on-surface-variant
uppercase tracking-wider mb-2">New Password</label>

      <input
        type="password"
        required
        minLength={8}
        value={form.newPassword}
        onChange={(e) => setForm({ ...form, newPassword: e.target.value })}
        className="w-full bg-surface-container-low border-none rounded-xl px-4
py-3 text-sm focus:ring-2 focus:ring-primary/30 outline-none transition-all"
        placeholder="Minimum 8 characters"
      />
    </div>

    <div>

      <label className="block text-xs font-bold text-on-surface-variant
uppercase tracking-wider mb-2">Confirm Password</label>

      <input
        type="password"
        required
        minLength={8}
        value={form.confirmPassword}

```



```

      onChange={{e} => setForm({ ...form, confirmPassword: e.target.value })}

      className="w-full bg-surface-container-low border-none rounded-xl px-4
py-3 text-sm focus:ring-2 focus:ring-primary/30 outline-none transition-all"

      placeholder="Re-enter your new password"

    />
  </div>

```

```

    <button

      type="submit"

      disabled={mutation.isPending}

      className="w-full bg-gradient-to-br from-primary-container to-primary
text-white py-3 rounded-xl font-semibold text-sm shadow-editorial hover:opacity-
90 active:scale-95 transition-all disabled:opacity-60"

    >

      {mutation.isPending ? 'Updating...' : 'Update Password'}

    </button>

  </form>

)}

```

```

<p className="text-center text-sm text-on-surface-variant mt-6">

  Remembered it?{' '}

  <Link to="/login" className="text-primary font-semibold hover:underline">

    Back to login

  </Link>

</p>

</div>

```

</div>

</div>

)

}



Settings.jsx

```
import { useState } from 'react'

import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'

import toast from 'react-hot-toast'

import { authApi } from '../services/api'

import { useAuthStore } from '../store/authStore'

export default function Settings() {

  const { user, updateUser } = useAuthStore()

  const isAdmin = user?.role === 'admin'

  const queryClient = useQueryClient()

  const [profileForm, setProfileForm] = useState({ name: user?.name || '' })

  const [passwordForm, setPasswordForm] = useState({ current: '', password: '' })

  const [inviteForm, setInviteForm] = useState({ name: '', email: '', role: 'staff', password: '' })

  const [activeTab, setActiveTab] = useState('profile')

  const inviteMutation = useMutation({

    mutationFn: () => authApi.invite(inviteForm),

    onSuccess: () => {

      toast.success(`${inviteForm.name} has been added to your workspace!`)

      setInviteForm({ name: '', email: '', role: 'staff', password: '' })

      queryClient.invalidateQueries({ queryKey: ['team'] })

    },

  })
```

```

    onError: (err) => toast.error(err.response?.data?.error || 'Invite failed'),
  })

  const { data: teamData, isLoading: isTeamLoading, isError: isTeamError, refetch:
  refetchTeam } = useQuery({
    queryKey: ['team'],
    queryFn: () => authApi.team().then(r => r.data),
    enabled: activeTab === 'team' && isAdmin
  })

  const reactivateMutation = useMutation({
    mutationFn: (id) => authApi.reactivate(id),
    onSuccess: () => { toast.success('User reactivated successfully!'); refetchTeam() },
    onError: (err) => toast.error(err.response?.data?.error || 'Failed to reactivate')
  })

  const deactivateMutation = useMutation({
    mutationFn: (id) => authApi.deactivate(id),
    onSuccess: () => { toast.success('User deactivated successfully'); refetchTeam() },
    onError: (err) => toast.error(err.response?.data?.error || 'Failed to deactivate')
  })

  const TABS = [
    { key: 'profile', label: 'Profile', icon: 'person' },
    { key: 'team', label: 'Team', icon: 'group', adminOnly: true },
  ]

```

```

    { key: 'about', label: 'About', icon: 'info' },
  ].filter(t => !t.adminOnly || isAdmin)

const getReactivationDaysLeft = (deactivatedAt) => {
  if (!deactivatedAt) return null

  const cutoff = new Date(deactivatedAt).getTime() + (45 * 24 * 60 * 60 * 1000)
  const remaining = cutoff - Date.now()
  if (remaining <= 0) return 0
  return Math.ceil(remaining / (24 * 60 * 60 * 1000))
}

return (
  <div className="px-8 py-8 max-w-3xl">
    <div className="mb-8">
      <h2 className="text-3xl font-bold tracking-tight text-on-surface">Settings</h2>
      <p className="text-on-surface-variant mt-1 text-sm">Manage your profile and workspace</p>
    </div>

    { /* Tabs */ }

    <div className="flex gap-1 mb-8 bg-surface-container-low rounded-xl p-1 w-fit">
      {TABS.map(t => (
        <button key={t.key} onClick={() => setActiveTab(t.key)}

```

```

        className={`flex items-center gap-2 px-4 py-2 rounded-lg text-sm font-
semibold transition-all ${
          activeTab === t.key
            ? 'bg-surface-container-highest text-on-surface shadow-sm'
            : 'text-on-surface-variant hover:bg-surface-container hover:text-on-surface'
        }}>
        <span className="material-symbols-outlined text-[16px]">{t.icon}</span>
        {t.label}
      </button>
    )}
  </div>

```

```

    {/* Profile Tab */}
    {activeTab === 'profile' && (
      <div className="space-y-6">
        <div className="bg-surface-container-lowest rounded-2xl editorial-shadow
p-6">
          <h3 className="font-bold text-on-surface mb-5">Profile Information</h3>

```

```

          {/* Avatar */}
          <div className="flex items-center gap-4 mb-6">
            <div className="w-16 h-16 rounded-2xl bg-primary-container flex items-
center justify-center text-white text-2xl font-bold">
              {user?.name?.charAt(0)?.toUpperCase()}
            </div>
            <div>

```

```

    <p className="font-bold text-on-surface">{user?.name}</p>
    <p className="text-sm text-on-surface-variant">{user?.email}</p>
    <span className="inline-block mt-1 px-2.5 py-0.5 rounded-full bg-
primary-fixed/30 text-primary text-[10px] font-bold uppercase tracking-wider">
      {user?.role}
    </span>
  </div>
</div>

<div className="space-y-4">
  <div>
    <label className="block text-xs font-bold text-on-surface-variant
uppercase tracking-wider mb-1.5">Display Name</label>
    <input value={profileForm.name} onChange={e => setProfileForm({ name:
e.target.value })}
      className="w-full bg-surface-container-low border-none rounded-xl px-4
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none" />
  </div>
  <div>
    <label className="block text-xs font-bold text-on-surface-variant
uppercase tracking-wider mb-1.5">Email Address</label>
    <input value={user?.email} readOnly
      className="w-full bg-surface-container border-none rounded-xl px-4 py-
2.5 text-sm outline-none text-on-surface-variant cursor-not-allowed" />
  </div>
</div>

```



```

<div className="mt-5 flex justify-end">

  <button

    onClick={() => { updateUser({ name: profileForm.name });
toast.success('Profile updated!') }}

    className="px-6 py-2.5 bg-gradient-to-br from-primary-container to-
primary text-white rounded-xl text-sm font-semibold editorial-shadow
hover:opacity-90 active:scale-95 transition-all">

      Save Changes

    </button>

  </div>
</div>

{/* Password section */}

<div className="bg-surface-container-lowest rounded-2xl editorial-shadow
p-6">

  <h3 className="font-bold text-on-surface mb-5">Change Password</h3>

  <div className="space-y-4">

    {[

      { key: 'current', label: 'Current Password' },

      { key: 'password', label: 'New Password' },

    ].map(f => {

      <div key={f.key}>

        <label className="block text-xs font-bold text-on-surface-variant
uppercase tracking-wider mb-1.5">{f.label}</label>

        <input type="password" value={passwordForm[f.key]}

          onChange={e => setPasswordForm({ ...passwordForm, [f.key]:
e.target.value }}}


```

```
className="w-full bg-surface-container-low border-none rounded-xl px-4 py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none"
```

```
placeholder="••••••••" />
```

```
</div>
```

```
)))
```

```
</div>
```

```
<div className="mt-5 flex justify-end">
```

```
<button
```

```
onClick={() => toast('Password change requires re-authentication. Feature coming soon.', { icon: 'i' })}
```

```
className="px-6 py-2.5 bg-surface-container-highest text-on-surface rounded-xl text-sm font-semibold hover:bg-surface-container-high transition-colors">
```

```
Update Password
```

```
</button>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
})
```

```
{/* Team Tab */}
```

```
{activeTab === 'team' && isAdmin && (
```

```
<div className="space-y-6">
```

```
<div className="bg-surface-container-lowest rounded-2xl editorial-shadow p-6">
```

```
<h3 className="font-bold text-on-surface mb-1">Invite Team Member</h3>
```

```
<p className="text-xs text-on-surface-variant mb-5">Add managers and staff  
to your workspace</p>
```

```
<form onSubmit={e => { e.preventDefault(); inviteMutation.mutate() }}  
className="space-y-4">
```

```
<div className="grid grid-cols-2 gap-4">
```

```
<div>
```

```
<label className="block text-xs font-bold text-on-surface-variant  
uppercase tracking-wider mb-1.5">Full Name *</label>
```

```
<input required value={inviteForm.name} onChange={e =>  
setInviteForm({ ...inviteForm, name: e.target.value })}
```

```
className="w-full bg-surface-container-low border-none rounded-xl px-  
4 py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none"
```

```
placeholder="Jane Smith" />
```

```
</div>
```

```
<div>
```

```
<label className="block text-xs font-bold text-on-surface-variant  
uppercase tracking-wider mb-1.5">Role *</label>
```

```
<select value={inviteForm.role} onChange={e => setInviteForm({  
...inviteForm, role: e.target.value })}
```

```
className="w-full bg-surface-container-low border-none rounded-xl px-  
4 py-2.5 text-sm focus:ring-0 outline-none">
```

```
<option value="staff">Staff</option>
```

```
<option value="manager">Manager</option>
```

```
</select>
```

```
</div>
```

```
</div>
```

```

    <label className="block text-xs font-bold text-on-surface-variant
uppercase tracking-wider mb-1.5">Email *</label>

    <input required type="email" value={inviteForm.email} onChange={e =>
setInviteForm({ ...inviteForm, email: e.target.value })}

    className="w-full bg-surface-container-low border-none rounded-xl px-
4 py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none"

    placeholder="jane@company.com" />

</div>

<div>

    <label className="block text-xs font-bold text-on-surface-variant
uppercase tracking-wider mb-1.5">Temp Password *</label>

    <input required value={inviteForm.password} onChange={e =>
setInviteForm({ ...inviteForm, password: e.target.value })}

    className="w-full bg-surface-container-low border-none rounded-xl px-
4 py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none"

    placeholder="They should change this" />

</div>

</div>

<div className="bg-primary-fixed/20 rounded-xl p-3 text-xs text-on-
surface-variant">

    <strong>Role permissions:</strong><br />

    <span className="font-semibold">Staff</span> — Can view and record
stock movements<br />

    <span className="font-semibold">Manager</span> — Can add/edit
products and suppliers<br />

    <span className="font-semibold">Admin</span> — Full access including
deletion and team management

```

```

</div>

<div className="flex justify-end">

  <button type="submit" disabled={inviteMutation.isPending}

    className="flex items-center gap-2 px-6 py-2.5 bg-gradient-to-br from-
    primary-container to-primary text-white rounded-xl text-sm font-semibold
    editorial-shadow hover:opacity-90 active:scale-95 transition-all disabled:opacity-
    60">

    <span className="material-symbols-outlined text-
    [18px]">person_add</span>

    {inviteMutation.isPending ? 'Adding...' : 'Add Team Member'}

  </button>
</div>
</form>
</div>

<div className="bg-surface-container-lowest rounded-2xl editorial-shadow
overflow-hidden">

  <h3 className="font-bold text-on-surface p-6 pb-2">Team Members</h3>

  <div className="divide-y divide-surface-container-low border-t border-
  surface-container-low mt-4">

    {isTeamLoading && (

      <div className="p-6 text-center text-sm text-on-surface-variant">Loading
      team members...</div>

    )}

    {isTeamError && (

      <div className="p-6 text-center text-sm text-tertiary">Could not load team
      members. Please try again.</div>
    )}
  </div>
</div>

```

```

    })

    {teamData?.team?.map(member => (

        <div key={member.id} className="p-6 flex items-center justify-between
        hover:bg-surface-container-low/50 transition-colors">

            <div className="flex items-center gap-4">

                <div className="w-10 h-10 rounded-xl bg-primary-container/20 flex
                items-center justify-center text-primary font-bold">

                    {member.name.charAt(0).toUpperCase()}

                </div>

                <div>

                    <p className="font-bold text-on-surface flex items-center gap-2">

                        {member.name}

                        {!member.is_active && <span className="px-2 py-0.5 rounded-full bg-
                        tertiary-fixed/30 text-tertiary text-[10px] uppercase font-bold tracking-
                        wider">Deactivated</span>}

                    </p>

                    <p className="text-xs text-on-surface-variant font-
                    mono">{member.email} · {member.role}</p>

                    {!member.is_active && member.deactivated_at && (

                        <>

                            <p className="text-[10px] text-tertiary mt-1 font-medium">

                                Deactivated on {new
                                Date(member.deactivated_at).toLocaleDateString()}

                            </p>

                            <p className="text-[10px] text-on-surface-variant mt-1">

                                {getReactivationDaysLeft(member.deactivated_at) > 0

```

```

      ? `${getReactivationDaysLeft(member.deactivated_at)} day(s) left to
reactivate`

      : 'Reactivation window expired'}

    </p>
  </>
)}
</div>
</div>
<div className="flex gap-2">
  {member.id !== user.id && member.is_active && {
    <button onClick={() => deactivateMutation.mutate(member.id)}
disabled={deactivateMutation.isPending}
    className="px-3 py-1.5 bg-surface-container text-tertiary rounded-lg
text-xs font-semibold hover:bg-tertiary hover:text-white transition-colors">
      Deactivate
    </button>
  }}
  {!member.is_active && {
    <button onClick={() => reactivateMutation.mutate(member.id)}
disabled={reactivateMutation.isPending}
    className="px-3 py-1.5 bg-secondary-container/30 text-secondary
rounded-lg text-xs font-semibold hover:bg-secondary hover:text-white transition-
colors">
      Reactivate
    </button>
  }}
</div>

```



```

    </div>

  )})

  {!isTeamLoading && !isTeamError && !teamData?.team?.length && (
    <div className="p-6 text-center text-sm text-on-surface-variant">No team
members found</div>

  })

</div>

</div>

</div>
})

{/* About Tab */}
{activeTab === 'about' && (
  <div className="space-y-6">
    <div className="bg-surface-container-lowest rounded-2xl editorial-shadow
p-6">
      <div className="flex items-center gap-4 mb-6">
        <div className="w-16 h-16 rounded-2xl bg-primary-container flex items-
center justify-center text-white">
          <span className="material-symbols-outlined text-
3xl">inventory_2</span>
        </div>
        <div>
          <h3 className="text-xl font-bold text-on-surface">StockGrid</h3>
          <p className="text-xs text-on-surface-variant uppercase tracking-widest
font-bold">Inventory Intelligence</p>
        </div>

```

```
</div>
```

```
<div className="space-y-4 text-sm">
```

```
{[
```

```
{ label: 'Version', value: '1.0.0' },
```

```
{ label: 'AI Engine', value: 'Google Gemini 2.0 Flash Lite' },
```

```
{ label: 'Database', value: 'PostgreSQL via Supabase' },
```

```
{ label: 'Frontend', value: 'React + Vite + Tailwind CSS' },
```

```
{ label: 'Backend', value: 'Node.js + Express' },
```

```
{ label: 'Your Role', value: user?.role?.charAt(0).toUpperCase() +  
user?.role?.slice(1) },
```

```
].map(item => (
```

```
<div key={item.label} className="flex items-center justify-between py-2  
border-b border-surface-container-low last:border-0">
```

```
<span className="text-on-surface-variant font-  
medium">{item.label}</span>
```

```
<span className="font-semibold text-on-surface">{item.value}</span>
```

```
</div>
```

```
)))
```

```
</div>
```

```
<div className="mt-6 p-4 bg-surface-container-low rounded-xl text-xs text-  
on-surface-variant leading-relaxed">
```

```
<strong className="text-on-surface">Architecture Note:</strong> Stock  
levels are never stored directly — they are always calculated as the sum of IN  
transactions minus OUT transactions. This ensures a complete, auditable history of  
all inventory movements.
```

```

    </div>

</div>

{/* Danger Zone */}

{isAdmin && (
    <div className="bg-primary-fixed/10 border border-primary/20 rounded-
2xl p-6">
        <div className="flex items-center gap-2 text-primary mb-4">
            <span className="material-symbols-outlined text-
[20px]">warning</span>
            <h3 className="font-bold">Danger Zone</h3>
        </div>
        <div className="flex items-center justify-between">
            <div>
                <p className="font-bold text-on-surface text-sm">Delete Workspace</p>
                <p className="text-xs text-on-surface-variant mt-1 max-w-[400px]">
                    Permanently delete this organization and all associated data. This action
is irreversible.
                </p>
            </div>
            <button
                onClick={() => {
                    const confirmName = window.prompt("Type 'DELETE' to confirm
permanent workspace deletion.");
                    if (confirmName === 'DELETE') {
                        authApi.deleteOrganization()

```

```

        .then(() => {
            toast.success('Workspace deleted. Goodbye!');
            setTimeout(() => {
                useAuthStore.getState().logout();
                window.location.href = '/login';
            }, 2000);
        })
        .catch(err => toast.error(err.response?.data?.error || 'Deletion failed'));
    }
}

    className="px-4 py-2 bg-primary text-white rounded-xl text-xs font-bold
editorial-shadow hover:bg-primary/90 transition-all"
    >
        Delete Workspace
    </button>
</div>
</div>
    })
</div>
})
</div>
)
}

```

Suppliers.jsx

```
import { useState } from 'react'

import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'

import toast from 'react-hot-toast'

import { suppliersApi, importExportApi } from '../services/api'

import { useAuthStore } from '../store/authStore'

import SupplierImportModal from '../components/modals/SupplierImportModal'

function SupplierModal({ supplier, onClose, onSave }) {

  const isEdit = !!supplier

  const [form, setForm] = useState({

    name: supplier?.name || "",

    contact_email: supplier?.contact_email || "",

    phone: supplier?.phone || "",

    address: supplier?.address || "",

    lead_time_days: supplier?.lead_time_days?.toString() || '7',

    notes: supplier?.notes || "",

  })

  const mutation = useMutation({

    mutationFn: () => isEdit

      ? suppliersApi.update(supplier.id, form)

      : suppliersApi.create(form),

    onSuccess: () => { toast.success(isEdit ? 'Supplier updated!' : 'Supplier added!');

    onSave() },

  })
```

```

    onError: (err) => toast.error(err.response?.data?.error || 'Save failed'),
  })

  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center">
      <div className="absolute inset-0 bg-black/30 backdrop-blur-sm"
        onClick={onClose} />
      <div className="relative bg-white rounded-2xl shadow-editorial-lg w-full max-
        w-md mx-4">
        <div className="px-6 pt-6 pb-4 border-b border-surface-container-low flex
          items-center justify-between">
          <h2 className="text-xl font-bold text-on-surface">{isEdit ? 'Edit Supplier' :
            'Add Supplier'}</h2>
          <button onClick={onClose} className="p-2 rounded-lg hover:bg-surface-
            container-low">
            <span className="material-symbols-outlined text-on-surface-
              variant">close</span>
          </button>
        </div>
        <form onSubmit={e => { e.preventDefault(); mutation.mutate() }}
          className="p-6 space-y-4">
          {[
            { key: 'name', label: 'Supplier Name', required: true, placeholder: 'Acme
              Suppliers Inc.' },
            { key: 'contact_email', label: 'Contact Email', type: 'email', placeholder:
              'orders@supplier.com' },
            { key: 'phone', label: 'Phone', placeholder: '+1 555 000 0000' },
            { key: 'address', label: 'Address', placeholder: '123 Warehouse Rd, City' },

```

```

].map(f => (
  <div key={f.key}>

    <label className="block text-xs font-bold text-on-surface-variant uppercase
tracking-wider mb-1.5">

      {f.label}{f.required && <span className="text-tertiary ml-0.5">*</span>}

    </label>

    <input required={f.required} type={f.type || 'text'} value={form[f.key]}

      onChange={e => setForm({ ...form, [f.key]: e.target.value })}

      placeholder={f.placeholder}

      className="w-full bg-surface-container-low border-none rounded-xl px-4
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none" />

    </div>
  )
)}

<div>

  <label className="block text-xs font-bold text-on-surface-variant uppercase
tracking-wider mb-1.5">Lead Time (days)</label>

  <input type="number" min="1" value={form.lead_time_days}

    onChange={e => setForm({ ...form, lead_time_days: e.target.value })}

    className="w-full bg-surface-container-low border-none rounded-xl px-4
py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none" />

  </div>

  <div>

    <label className="block text-xs font-bold text-on-surface-variant uppercase
tracking-wider mb-1.5">Notes</label>

    <textarea value={form.notes} onChange={e => setForm({ ...form, notes:
e.target.value })}

```



```

      rows={2} className="w-full bg-surface-container-low border-none
rounded-xl px-4 py-2.5 text-sm focus:ring-2 focus:ring-primary/30 outline-none
resize-none" />

    </div>

    <div className="flex gap-3 pt-2">

      <button type="button" onClick={onClose}

        className="flex-1 py-3 bg-surface-container-highest rounded-xl text-sm
font-semibold hover:bg-surface-container-high transition-colors">Cancel</button>

        <button type="submit" disabled={mutation.isPending}

          className="flex-1 py-3 bg-gradient-to-br from-primary-container to-
primary text-white rounded-xl text-sm font-semibold editorial-shadow
hover:opacity-90 disabled:opacity-60 active:scale-95 transition-all">

            {mutation.isPending ? 'Saving...' : isEdit ? 'Save Changes' : 'Add Supplier'}

          </button>

        </div>

      </form>

    </div>

  </div>

)
}

```

```

export default function Suppliers() {
  const { user } = useAuthStore()
  const qc = useQueryClient()
  const canEdit = ['admin', 'manager'].includes(user?.role)
  const [modal, setModal] = useState(null)
  const [importModal, setImportModal] = useState(false)

```

```
const [selected, setSelected] = useState([])
```

```
const { data, isLoading } = useQuery({  
  queryKey: ['suppliers'],  
  queryFn: () => suppliersApi.list().then(r => r.data.suppliers),  
})
```

```
const deleteMutation = useMutation({  
  mutationFn: (id) => suppliersApi.delete(id),  
  onSuccess: () => { qc.invalidateQueries(['suppliers']); toast.success('Supplier  
deleted'); setSelected([]) },  
  onError: () => toast.error('Delete failed'),  
})
```

```
const bulkDeleteMutation = useMutation({  
  mutationFn: (ids) => suppliersApi.bulkDelete(ids),  
  onSuccess: (res) => {  
    qc.invalidateQueries(['suppliers'])  
    toast.success(res.data.message)  
    setSelected([])  
  },  
  onError: () => toast.error('Bulk delete failed'),  
})
```

```
const handleBulkDelete = () => {
```

```

    if (confirm(`Delete ${selected.length} suppliers?`)) {
      bulkDeleteMutation.mutate(selected)
    }
  }

  const suppliers = data || []

  const toggleSelect = (id) => {
    setSelected(prev => prev.includes(id) ? prev.filter(x => x !== id) : [...prev, id])
  }

  const handleExport = async () => {
    try {
      const p = selected.length > 0 ? { ids: selected.join(',') } : {}
      const res = await importExportApi.exportSuppliers(p)
      const url = URL.createObjectURL(new Blob([res.data], { type: 'text/csv' }))
      const a = document.createElement('a')
      a.href = url
      a.download = `suppliers_export.csv`
      a.click()
    } catch { toast.error('Export failed') }
  }

  return (
    <div className="px-8 py-8">

```

```

<div className="flex items-end justify-between mb-8">
  <div>
    <nav className="flex items-center gap-1 text-xs font-bold tracking-widest
text-on-surface-variant uppercase mb-2">
      <span>Network</span>
      <span className="material-symbols-outlined text-
[14px]">chevron_right</span>
      <span className="text-primary">Suppliers</span>
    </nav>
    <h2 className="text-3xl font-bold tracking-tight text-on-
surface">Suppliers</h2>
    <p className="text-on-surface-variant mt-1 text-sm">{suppliers.length}
supplier{suppliers.length !== 1 ? 's' : ''} in your network</p>
  </div>

  <div className="flex items-center gap-3">
    {canEdit && (
      <button
        onClick={() => setImportModal(true)}

        className="flex items-center gap-2 px-5 py-2.5 rounded-xl text-sm font-
semibold text-on-surface-variant bg-surface-container-highest hover:bg-surface-
container-high transition-colors"

        >
          <span className="material-symbols-outlined text-lg">upload_file</span>

          Import
        </button>
      )}
  </div>

```

```

<button
  onClick={() => handleExport()}

  className="flex items-center gap-2 px-5 py-2.5 rounded-xl text-sm font-
semibold text-on-surface-variant bg-surface-container-highest hover:bg-surface-
container-high transition-colors"

  >

  <span className="material-symbols-outlined text-lg">download</span>

  Export

</button>

{canEdit && (
  <button onClick={() => setModal('new')}

    className="flex items-center gap-2 px-6 py-2.5 rounded-xl text-sm font-
semibold text-white bg-gradient-to-br from-primary-container to-primary editorial-
shadow active:scale-95 transition-transform">

      <span className="material-symbols-outlined text-lg">add</span>

      Add Supplier

    </button>

  )}

</div>

</div>

{selected.length > 0 && (
  <div className="bg-secondary-container/20 text-on-secondary-container
rounded-xl px-5 py-3.5 mb-6 flex items-center justify-between border border-
secondary-container/50">

    <div className="flex items-center gap-3 font-semibold text-sm">

```

```

      <span className="material-symbols-outlined text-
secondary">library_add_check</span>

      {selected.length} supplier(s) selected

    </div>

    <div className="flex gap-2 items-center">

      <button onClick={() => handleExport()} className="px-4 py-2 bg-secondary
text-white rounded-lg text-sm font-bold shadow hover:bg-secondary/90 transition-
all active:scale-95">

        Export Selected

      </button>

      <button onClick={() => setSelected([])} className="px-4 py-2 bg-white text-
on-surface-variant border border-outline-variant shadow-sm rounded-lg text-sm
font-bold hover:bg-surface-container-low transition-colors">

        Clear

      </button>

      {canEdit && (

        <button

          onClick={handleBulkDelete}

          className="w-10 h-10 flex items-center justify-center bg-tertiary/10 text-
tertiary rounded-lg hover:bg-tertiary hover:text-white transition-all active:scale-95"

          title="Delete Selected"

        >

          <span className="material-symbols-outlined">delete</span>

        </button>

      )}

    </div>

  </div>

```

```

    })

    {isLoading ? (
      <div className="grid grid-cols-3 gap-6">
        {[...Array(6)].map((_, i) => <div key={i} className="skeleton h-48 rounded-xl"
        />)}
      </div>
    ) : suppliers.length === 0 ? (
      <div className="text-center py-24">
        <span className="material-symbols-outlined text-5xl text-on-surface-variant
        mb-4 block">local_shipping</span>
        <p className="text-lg font-bold text-on-surface mb-1">No suppliers yet</p>
        <p className="text-sm text-on-surface-variant mb-6">Add your first supplier
        to link them to products</p>
        {canEdit && (
          <button onClick={() => setModal('new')}
            className="px-6 py-2.5 bg-gradient-to-br from-primary-container to-
            primary text-white rounded-xl text-sm font-semibold editorial-shadow">
              Add Supplier
            </button>
          )}
        </div>
      ) : (
        <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
          {suppliers.map(supplier => (
            <div key={supplier.id} className="bg-surface-container-lowest rounded-xl
            editorial-shadow p-6 group">

```



```

<div className="flex items-start justify-between mb-4">
  <div className="flex items-center gap-3">
    <input
      type="checkbox"
      checked={selected.includes(supplier.id)}
      onChange={() => toggleSelect(supplier.id)}
      className="rounded border-outline-variant text-primary focus:ring-
primary -mt-1"
    />
    <div className="w-12 h-12 rounded-xl bg-primary-fixed/30 flex items-
center justify-center text-primary flex-shrink-0">
      <span className="material-symbols-outlined text-
2xl">local_shipping</span>
    </div>
  </div>
  {canEdit && (
    <div className="flex gap-1 opacity-0 group-hover:opacity-100 transition-
opacity">
      <button onClick={() => setModal(supplier)}
        className="p-1.5 rounded-lg text-on-surface-variant hover:bg-surface-
container-high transition-colors">
        <span className="material-symbols-outlined text-[18px]">edit</span>
      </button>
      <button onClick={() => { if (confirm(`Delete "${supplier.name}"`))
deleteMutation.mutate(supplier.id) }}
        className="p-1.5 rounded-lg text-tertiary hover:bg-tertiary-fixed/20
transition-colors">

```

```
        <span className="material-symbols-outlined text-[18px]">delete</span>
```

```
    </button>
```

```
  </div>
```

```
}}
```

```
</div>
```

```
    <h3 className="text-base font-bold text-on-surface mb-1">{supplier.name}</h3>
```

```
    <div className="space-y-2 mt-3">
```

```
      {supplier.contact_email && (
```

```
        <div className="flex items-center gap-2 text-xs text-on-surface-variant">
```

```
          <span className="material-symbols-outlined text-[14px]">mail</span>
```

```
          <a href={`mailto:${supplier.contact_email}`} className="hover:text-primary transition-colors truncate">
```

```
            {supplier.contact_email}
```

```
          </a>
```

```
        </div>
```

```
      )}
```

```
      {supplier.phone && (
```

```
        <div className="flex items-center gap-2 text-xs text-on-surface-variant">
```

```
          <span className="material-symbols-outlined text-[14px]">call</span>
```

```
          <span>{supplier.phone}</span>
```

```
        </div>
```

```
      )}
```

```

    {supplier.address && (
      <div className="flex items-center gap-2 text-xs text-on-surface-variant">
        <span className="material-symbols-outlined text-[14px]">location_on</span>
        <span className="truncate">{supplier.address}</span>
      </div>
    )}
  </div>

  <div className="mt-4 pt-4 border-t border-surface-container-low flex items-center justify-between">
    <div className="text-xs text-on-surface-variant">
      Lead time: <span className="font-bold text-on-surface">{supplier.lead_time_days || 7} days</span>
    </div>
    <div className="text-xs text-on-surface-variant">
      Products: <span className="font-bold text-on-surface">{supplier.products?.length || 0}</span>
    </div>
  </div>

  {supplier.notes && (
    <p className="text-xs text-on-surface-variant mt-3 italic leading-relaxed line-clamp-2">{supplier.notes}</p>
  )}
</div>
)}}
</div>

```

```

    }}

    {modal !== null && (
      <SupplierModal
        supplier={modal === 'new' ? null : modal}
        onClose={() => setModal(null)}
        onSave={() => { qc.invalidateQueries(['suppliers']); setModal(null) }}
      />
    )}
    {importModal && (
      <SupplierImportModal
        onClose={() => setImportModal(false)}
        onImported={() => { qc.invalidateQueries(['suppliers']); setImportModal(false) }}
      />
    )}
  </div>
)
}

```

Transactions.jsx

```
import { useState } from 'react'

import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query'

import { format, parseISO } from 'date-fns'

import toast from 'react-hot-toast'

import { transactionsApi, productsApi, alertsApi } from '../services/api'

import TransactionModal from '../components/modals/TransactionModal'

export default function Transactions() {
  const qc = useQueryClient()

  const [filters, setFilters] = useState({ type: '', start_date: '', end_date: '', product_id: '' })

  const [page, setPage] = useState(1)

  const [txModal, setTxModal] = useState(null)

  const [tab, setTab] = useState('movements') // movements / alerts

  const { data, isLoading } = useQuery({
    queryKey: ['transactions', filters, page],
    queryFn: () => transactionsApi.list({ ...filters, page, limit: 25 }).then(r => r.data),
  })

  const { data: alertsData, isLoading: alertsLoading } = useQuery({
    queryKey: ['alerts', 'all'],
    queryFn: () => alertsApi.list({ resolved: 'false', limit: 50 }).then(r => r.data.alerts),
  })
```

```

const { data: summaryData } = useQuery({
  queryKey: ['tx-summary'],
  queryFn: () => transactionsApi.summary({ days: 30 }).then(r => r.data),
})

const { data: productsData } = useQuery({
  queryKey: ['products-list'],
  queryFn: () => productsApi.list({ limit: 200 }).then(r => r.data.products),
})

const resolveMutation = useMutation({
  mutationFn: (id) => alertsApi.resolve(id),
  onSuccess: () => { qc.invalidateQueries(['alerts']); toast.success('Alert resolved') },
})

const transactions = data?.transactions || []
const pagination = data?.pagination || {}
const alerts = alertsData || []

const handleExportCSV = async () => {
  try {
    const params = new URLSearchParams({ format: 'csv', ...filters })
    window.open(`/api/export/transactions?${params}`, '_blank')
  } catch { toast.error('Export failed') }
}

```

```
}
```

```
return (
```

```
<div className="px-8 py-8">
```

```
  { /* Header */ }
```

```
<div className="flex items-end justify-between mb-8">
```

```
<div>
```

```
<h2 className="text-3xl font-bold tracking-tight text-on-surface">Stock  
Movements</h2>
```

```
<p className="text-on-surface-variant mt-1 text-sm">
```

```
  {summaryData ? `${summaryData.total_transactions} transactions in last 30  
days` : 'Loading...'}
```

```
</p>
```

```
</div>
```

```
<div className="flex gap-3">
```

```
<button onClick={handleExportCSV}
```

```
  className="flex items-center gap-2 px-5 py-2.5 bg-surface-container-highest  
text-on-surface rounded-xl text-sm font-semibold hover:bg-surface-container-high  
transition-colors">
```

```
<span className="material-symbols-outlined text-  
[18px]">download</span>
```

```
  Export Logs
```

```
</button>
```

```
<button onClick={() => setTxModal(true)}
```

```
  className="flex items-center gap-2 px-5 py-2.5 bg-gradient-to-br from-  
primary-container to-primary text-white rounded-xl text-sm font-semibold  
editorial-shadow hover:opacity-90 active:scale-95 transition-all">
```



```
<span className="material-symbols-outlined text-[18px]">add</span>
```

```
Manual Adjustment
```

```
</button>
```

```
</div>
```

```
</div>
```

```
{/* Summary cards */}
```

```
<div className="grid grid-cols-3 gap-6 mb-8">
```

```
<div className="bg-surface-container-lowest rounded-xl editorial-shadow p-5">
```

```
<p className="text-xs font-bold text-on-surface-variant uppercase tracking-wider mb-2">Stock In (30d)</p>
```

```
<div className="flex items-center gap-3">
```

```
<div className="w-10 h-10 rounded-xl bg-secondary-container/20 flex items-center justify-center text-secondary">
```

```
<span className="material-symbols-outlined">trending_up</span>
```

```
</div>
```

```
<span className="text-3xl font-bold text-on-surface">+{summaryData?.total_in?.toLocaleString() || '0'}</span>
```

```
</div>
```

```
</div>
```

```
<div className="bg-surface-container-lowest rounded-xl editorial-shadow p-5">
```

```
<p className="text-xs font-bold text-on-surface-variant uppercase tracking-wider mb-2">Stock Out (30d)</p>
```

```
<div className="flex items-center gap-3">
```

```

<div className="w-10 h-10 rounded-xl bg-primary-fixed/40 flex items-
center justify-center text-primary">

  <span className="material-symbols-outlined">trending_down</span>

</div>

<span className="text-3xl font-bold text-on-surface">-
{summaryData?.total_out?.toLocaleString() || '0'}</span>

</div>

</div>

<div className="bg-surface-container-lowest rounded-xl editorial-shadow p-
5">

  <p className="text-xs font-bold text-on-surface-variant uppercase tracking-
wider mb-2">Active Alerts</p>

  <div className="flex items-center gap-3">

    <div className={`w-10 h-10 rounded-xl flex items-center justify-center
${alerts.length > 0 ? 'bg-tertiary-fixed/30 text-tertiary' : 'bg-secondary-container/20
text-secondary'}}>

      <span className="material-symbols-outlined">{alerts.length > 0 ? 'warning'
: 'check_circle'}</span>

    </div>

    <span className="text-3xl font-bold text-on-surface">{alerts.length}</span>

  </div>

</div>

</div>

{ /* Tab selector */ }

<div className="flex gap-1 mb-6 bg-surface-container-low rounded-xl p-1 w-
fit">

  {[{ key: 'movements', label: 'Movements', icon: 'swap_vert' },

```

```

    { key: 'alerts', label: `Alerts ${alerts.length > 0 ? `(${alerts.length})` : ''}`, icon:
'notifications' }

    ].map(t => (

      <button key={t.key} onClick={() => setTab(t.key)}

        className={`flex items-center gap-2 px-4 py-2 rounded-lg text-sm font-
semibold transition-all ${

          tab === t.key

            ? 'bg-surface-container-highest text-on-surface shadow-sm'

            : 'text-on-surface-variant hover:bg-surface-container hover:text-on-surface'

          }}>

        <span className="material-symbols-outlined text-[16px]">{t.icon}</span>

        {t.label}

      </button>

    )))
</div>

{/* Movements Tab */}

{tab === 'movements' && (

  <>

    {/* Filters */}

    <div className="grid grid-cols-5 gap-4 mb-6">

      <div className="flex items-center gap-2 bg-surface-container-low rounded-
xl px-3 col-span-2">

        <select value={filters.product_id} onChange={e => setFilters({ ...filters,
product_id: e.target.value })}>

          <div className="bg-transparent border-none text-sm focus:ring-0 outline-none
py-2.5 w-full">

```

```

        <option value="">All Products</option>

        {(productsData || []).map(p => <option key={p.id}
value={p.id}>{p.name}</option>)}

    </select>

</div>

<div className="flex items-center gap-2 bg-surface-container-low rounded-
xl px-3">

    <select value={filters.type} onChange={e => setFilters({ ...filters, type:
e.target.value })}

        className="bg-transparent border-none text-sm focus:ring-0 outline-none
py-2.5 w-full">

        <option value="">All Types</option>

        <option value="IN">Stock In</option>

        <option value="OUT">Stock Out</option>

    </select>

</div>

<input type="date" value={filters.start_date}

    onChange={e => setFilters({ ...filters, start_date: e.target.value })}

    className="bg-surface-container-low border-none rounded-xl px-4 py-2.5
text-sm focus:ring-2 focus:ring-primary/30 outline-none" />

<input type="date" value={filters.end_date}

    onChange={e => setFilters({ ...filters, end_date: e.target.value })}

    className="bg-surface-container-low border-none rounded-xl px-4 py-2.5
text-sm focus:ring-2 focus:ring-primary/30 outline-none" />

</div>

{ /* Table */}

```

```

<div className="bg-surface-container-lowest rounded-xl editorial-shadow
overflow-hidden">

  <table className="w-full text-left">

    <thead>

      <tr className="bg-surface-container-low">

        <th className="py-3.5 px-5 text-xs font-bold uppercase tracking-wider
text-on-surface-variant">Date & Time</th>

        <th className="py-3.5 px-4 text-xs font-bold uppercase tracking-wider
text-on-surface-variant">Type</th>

        <th className="py-3.5 px-4 text-xs font-bold uppercase tracking-wider
text-on-surface-variant">Product</th>

        <th className="py-3.5 px-4 text-xs font-bold uppercase tracking-wider
text-on-surface-variant">Quantity</th>

        <th className="py-3.5 px-4 text-xs font-bold uppercase tracking-wider
text-on-surface-variant">Reason</th>

        <th className="py-3.5 px-4 text-xs font-bold uppercase tracking-wider
text-on-surface-variant">Reference</th>

        <th className="py-3.5 px-4 text-xs font-bold uppercase tracking-wider
text-on-surface-variant">By</th>

      </tr>

    </thead>

    <tbody className="divide-y divide-surface-container-low">

      {isLoading ? (

        [...Array(10)].map((_, i) => (

          <tr key={i}>

            {[...Array(7)].map((_, j) => (

              <td key={j} className="py-4 px-4"><div className="skeleton h-4
rounded" /></td>

```

```

    }}}
  </tr>

  ))
) : transactions.length === 0 ? (
  <tr>

    <td colSpan={7} className="py-16 text-center text-on-surface-variant
text-sm">

      <span className="material-symbols-outlined text-4xl block mb-3 text-
outline">swap_vert</span>

      No transactions found

    </td>
  </tr>
) : transactions.map(tx => (
  <tr key={tx.id} className="hover:bg-surface-container-low/50 transition-
colors">
    <td className="py-4 px-5">

      <p className="text-sm font-medium text-on-
surface">{format(parseISO(tx.created_at), 'MMM d, yyyy')}</p>

      <p className="text-xs text-on-surface-
variant">{format(parseISO(tx.created_at), 'h:mm a')}</p>

    </td>

    <td className="py-4 px-4">

      <span className={`inline-flex items-center gap-1.5 px-3 py-1 rounded-
full text-xs font-bold ${
        tx.type === 'IN'
          ? 'bg-secondary-container/20 text-on-secondary-container'
          : 'bg-primary-fixed/30 text-primary'
      }`}>

```



```

    }}>

    <span className="material-symbols-outlined text-[12px]">{tx.type
=== 'IN' ? 'add' : 'remove'}</span>

    {tx.type === 'IN' ? 'Stock In' : 'Stock Out'}

  </span>

</td>

<td className="py-4 px-4">

  <p className="text-sm font-semibold text-on-
surface">{tx.product?.name || '—'}</p>

  <p className="text-xs text-on-surface-variant font-
mono">{tx.product?.sku}</p>

</td>

<td className="py-4 px-4">

  <span className={`text-sm font-bold ${tx.type === 'IN' ? 'text-
secondary' : 'text-primary'}`}>

    {tx.type === 'IN' ? '+' : '-'}{tx.quantity}

  </span>

</td>

<td className="py-4 px-4 text-sm text-on-surface-
variant">{tx.reason}</td>

<td className="py-4 px-4 text-xs text-on-surface-variant font-
mono">{tx.reference_no || '—'}</td>

<td className="py-4 px-4 text-xs text-on-surface-
variant">{tx.user?.name || 'System'}</td>

</tr>

)}}

</tbody>

```



```
</table>
```

```
{pagination.pages > 1 && {
```

```
  <div className="flex items-center justify-between px-6 py-4 border-t border-surface-container-low">
```

```
    <p className="text-xs text-on-surface-variant">
```

```
      Showing {(page - 1) * 25 + 1}-{Math.min(page * 25, pagination.total)} of {pagination.total}
```

```
    </p>
```

```
    <div className="flex items-center gap-1">
```

```
      <button onClick={() => setPage(p => Math.max(1, p - 1))} disabled={page === 1}
```

```
        className="p-1.5 rounded-lg text-on-surface-variant hover:bg-surface-container-high disabled:opacity-40">
```

```
        <span className="material-symbols-outlined text-[18px]">chevron_left</span>
```

```
      </button>
```

```
      <span className="text-xs text-on-surface-variant px-2">Page {page} of {pagination.pages}</span>
```

```
      <button onClick={() => setPage(p => Math.min(pagination.pages, p + 1))} disabled={page >= pagination.pages}
```

```
        className="p-1.5 rounded-lg text-on-surface-variant hover:bg-surface-container-high disabled:opacity-40">
```

```
        <span className="material-symbols-outlined text-[18px]">chevron_right</span>
```

```
      </button>
```

```
    </div>
```

```
</div>
```

```

    }}
  </div>
</>
}}

{/* Alerts Tab */}
{tab === 'alerts' && (
  <div className="bg-surface-container-lowest rounded-xl editorial-shadow overflow-hidden">
    {alertsLoading ? (
      <div className="p-6 space-y-3">
        {[...Array(5)].map((_, i) => <div key={i} className="skeleton h-16 rounded-xl" />)}
      </div>
    ) : alerts.length === 0 ? (
      <div className="py-16 text-center text-on-surface-variant">
        <span className="material-symbols-outlined text-4xl block mb-3 text-secondary" style={{ fontVariationSettings: '"FILL' 1" }}>check_circle</span>
        <p className="text-sm font-semibold text-on-surface">All clear!</p>
        <p className="text-xs mt-1">No active stock alerts</p>
      </div>
    ) : (
      <div className="divide-y divide-surface-container-low">
        {alerts.map(alert => (
          <div key={alert.id} className="flex items-center gap-4 p-5 hover:bg-surface-container-low/50 transition-colors">

```

```

<div className={`w-10 h-10 rounded-xl flex items-center justify-center
flex-shrink-0 ${
  alert.type === 'out_of_stock' ? 'bg-tertiary-fixed/30 text-tertiary' : 'bg-
yellow-100 text-yellow-700'
}}`>
  <span className="material-symbols-outlined text-[20px]">
    {alert.type === 'out_of_stock' ? 'do_not_disturb_on' : 'warning'}
  </span>
</div>

<div className="flex-1 min-w-0">
  <p className="text-sm font-bold text-on-
surface">{alert.product?.name}</p>
  <p className="text-xs text-on-surface-variant mt-
0.5">{alert.message}</p>
  <p className="text-[10px] text-outline mt-
0.5">{format(parseISO(alert.created_at), 'MMM d, yyyy • h:mm a')}</p>
</div>

<div className="flex items-center gap-2 flex-shrink-0">
  <span className={`px-2.5 py-1 rounded-full text-[10px] font-bold
uppercase tracking-wide ${
    alert.type === 'out_of_stock' ? 'bg-tertiary text-white' : 'bg-yellow-100
text-yellow-800'
  }}`>
    {alert.type === 'out_of_stock' ? 'Out of Stock' : 'Low Stock'}
  </span>
  <button onClick={() => resolveMutation.mutate(alert.id)}
    disabled={resolveMutation.isPending}

```

```
className="text-xs font-semibold text-primary hover:underline px-2  
py-1 rounded-lg hover:bg-primary-fixed/20 transition-colors">
```

```
Resolve
```

```
</button>
```

```
</div>
```

```
</div>
```

```
}}}
```

```
</div>
```

```
}}
```

```
</div>
```

```
}}
```

```
/* Quick transaction modal (when no specific product chosen) */
```

```
{txModal === true && {
```

```
<QuickTxModal
```

```
products={productsData || []}
```

```
onClose={() => setTxModal(null)}
```

```
onSaved={() => { qc.invalidateQueries(['transactions']);  
qc.invalidateQueries(['alerts']); setTxModal(null) }}
```

```
/>
```

```
}}
```

```
</div>
```

```
)
```

```
}
```

```

function QuickTxModal({ products, onClose, onSave }) {
  const [selectedProduct, setSelectedProduct] = useState(null)
  const [productSearch, setProductSearch] = useState("")

  const filtered = products.filter(p =>
    p.name.toLowerCase().includes(productSearch.toLowerCase()) ||
    p.sku.toLowerCase().includes(productSearch.toLowerCase())
  ).slice(0, 8)

  if (selectedProduct) {
    return <TransactionModal product={selectedProduct} onClose={onClose}
onSaved={onSave} />
  }

  return (
    <div className="fixed inset-0 z-50 flex items-center justify-center">
      <div className="absolute inset-0 bg-black/30 backdrop-blur-sm"
onClick={onClose} />
      <div className="relative bg-white rounded-2xl shadow-editorial-lg w-full max-
w-md mx-4">
        <div className="px-6 pt-6 pb-4 border-b border-surface-container-low flex
items-center justify-between">
          <h2 className="text-xl font-bold text-on-surface">Select Product</h2>
          <button onClick={onClose} className="p-2 rounded-lg hover:bg-surface-
container-low">
            <span className="material-symbols-outlined text-on-surface-
variant">close</span>

```

```

</button>

</div>

<div className="p-6">

  <div className="flex items-center gap-2 bg-surface-container-low rounded-xl
px-4 mb-4">

    <span className="material-symbols-outlined text-on-surface-variant text-
[18px]">search</span>

    <input type="text" placeholder="Search products..." value={productSearch}
    onChange={e => setProductSearch(e.target.value)}

    className="bg-transparent border-none py-2.5 text-sm focus:ring-0 outline-
none flex-1" />

  </div>

  <div className="space-y-1 max-h-72 overflow-y-auto scrollbar-thin">

    {filtered.map(p => {

      <button key={p.id} onClick={() => setSelectedProduct(p)}

        className="w-full flex items-center gap-3 px-4 py-3 rounded-xl hover:bg-
surface-container-low transition-colors text-left">

        <div className="w-8 h-8 rounded-lg bg-surface-container flex items-center
justify-center flex-shrink-0">

          <span className="material-symbols-outlined text-[16px] text-on-surface-
variant">inventory_2</span>

        </div>

        <div className="flex-1 min-w-0">

          <p className="text-sm font-semibold text-on-surface
truncate">{p.name}</p>

          <p className="text-xs text-on-surface-variant">SKU: {p.sku} · Stock:
{p.current_stock}</p>

        </div>

      }
    )}

  </div>

```

```

        <span className="material-symbols-outlined text-on-surface-variant text-[18px]">chevron_right</span>

        </button>

    )}

    {filtered.length === 0 && (

        <p className="text-sm text-on-surface-variant text-center py-8">No
        products found</p>

    )}

</div>

</div>

</div>

)
}

```

```

    api.js

import axios from 'axios'
import { useAuthStore } from '../store/authStore'

const api = axios.create({

  baseURL: import.meta.env.VITE_API_URL ?
  `${import.meta.env.VITE_API_URL}/api` : '/api',

  timeout: 30000,

})

```

```

// Request interceptor: attach token

```



```
api.interceptors.request.use((config) => {  
  const token = useAuthStore.getState().token  
  if (token) {  
    config.headers.Authorization = `Bearer ${token}`  
  }  
  return config  
})
```

// Response interceptor: handle auth errors

```
api.interceptors.response.use(  
  (response) => response,  
  (error) => {  
    if (error.response?.status === 401) {  
      useAuthStore.getState().logout()  
    }  
    return Promise.reject(error)  
  }  
)
```

```
export default api
```

// Auth

```
export const authApi = {  
  login: (data) => api.post('/auth/login', data),  
  register: (data) => api.post('/auth/register', data),
```

```
forgotPassword: (data) => api.post('/auth/forgot-password', data),
resetPassword: (data) => api.post('/auth/reset-password', data),
me: () => api.get('/auth/me'),
invite: (data) => api.post('/auth/invite', data),
team: () => api.get('/auth/team'),
reactivate: (id) => api.patch(`/auth/reactivate/${id}`),
deactivate: (id) => api.patch(`/auth/deactivate/${id}`),
deleteOrganization: () => api.delete('/auth/organization'),
}
```

// Products

```
export const productsApi = {
  list: (params) => api.get('/products', { params }),
  get: (id) => api.get(`/products/${id}`),
  create: (data) => api.post('/products', data),
  update: (id, data) => api.put(`/products/${id}`, data),
  delete: (id) => api.delete(`/products/${id}`),
  bulkDelete: (ids) => api.post('/products/bulk-delete', { ids }),
  categories: () => api.get('/products/meta/categories'),
}
```

// Transactions

```
export const transactionsApi = {
  list: (params) => api.get('/transactions', { params }),
  create: (data) => api.post('/transactions', data),
}
```

```
summary: (params) => api.get('/transactions/summary', { params }),  
}
```

// Suppliers

```
export const suppliersApi = {  
  list: () => api.get('/suppliers'),  
  get: (id) => api.get(`/suppliers/${id}`),  
  create: (data) => api.post('/suppliers', data),  
  update: (id, data) => api.put(`/suppliers/${id}`, data),  
  delete: (id) => api.delete(`/suppliers/${id}`),  
  bulkDelete: (ids) => api.post('/suppliers/bulk-delete', { ids }),  
}
```

// Reports

```
export const reportsApi = {  
  dashboard: () => api.get('/reports/dashboard'),  
  valuation: () => api.get('/reports/inventory-valuation'),  
  topProducts: (params) => api.get('/reports/top-products', { params }),  
  deadStock: (params) => api.get('/reports/dead-stock', { params }),  
}
```

// AI

```
export const aiApi = {  
  chat: (messages) => api.post('/ai/chat', { messages }),  
  predict: (product_id) => api.post('/ai/predict', { product_id }),  
}
```

```

reorderSuggestions: () => api.post('/ai/reorder-suggestions'),

deadStock: (days) => api.post('/ai/dead-stock', { days }),

columnMap: (headers, sample_data) => api.post('/ai/column-map', { headers,
sample_data }),

generateSKU: (name, category) => api.post('/ai/generate-sku', { name, category }),

dashboardInsights: (data) => api.post('/ai/dashboard-insights', { dashboard_data:
data }),

}

```

// Import / Export

```

export const importExportApi = {

  import: (formData) => api.post('/import', formData, {

    headers: { 'Content-Type': 'multipart/form-data' },

    timeout: 120000,

  }),

  importPreview: (formData) => api.post('/import?dry_run=true', formData, {

    headers: { 'Content-Type': 'multipart/form-data' },

    timeout: 60000,

  }),

  exportProducts: (format) => api.get(`/export/products?format=${format}`, {
responseType: 'blob' }),

  exportTransactions: (params) => api.get('/export/transactions', { params,
responseType: 'blob' }),

  importSuppliers: (formData) => api.post('/import/suppliers', formData, {

    headers: { 'Content-Type': 'multipart/form-data' }

  }),

}

```

```
exportSuppliers: (params) => api.get('/export/suppliers', { params, responseType:
'blob' })),
}
```

// Alerts

```
export const alertsApi = {
  list: (params) => api.get('/alerts', { params }),
  resolve: (id) => api.patch(`/alerts/${id}/resolve`),
}
```



authStore.js

```
import { create } from 'zustand'

import { persist } from 'zustand/middleware'

export const useAuthStore = create(
  persist(
    (set, get) => ({
      user: null,
      token: null,
      isAuthenticated: false,

      login: (user, token) => set({ user, token, isAuthenticated: true }),

      logout: () => {
        set({ user: null, token: null, isAuthenticated: false })
        window.location.href = '/login'
      },

      updateUser: (updates) => set(state => ({
        user: { ...state.user, ...updates }
      })),
    }),
    {
      name: 'stitch-auth',
    }
  )
)
```

```
    partialize: (state) => ({ user: state.user, token: state.token, isAuthenticated:
state.isAuthenticated })),
```

```
  }
```

```
  )
```

```
)
```



currency.js

```
const inrCurrencyFormatter = new Intl.NumberFormat('en-IN', {  
  style: 'currency',  
  currency: 'INR',  
  minimumFractionDigits: 2,  
  maximumFractionDigits: 2,  
})
```

```
const inrCompactFormatter = new Intl.NumberFormat('en-IN', {  
  style: 'currency',  
  currency: 'INR',  
  notation: 'compact',  
  maximumFractionDigits: 1,  
})
```

```
export function formatCurrencyINR(value) {  
  return inrCurrencyFormatter.format(Number(value) || 0)  
}
```

```
export function formatCompactCurrencyINR(value) {  
  return inrCompactFormatter.format(Number(value) || 0)  
}
```

App.jsx

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom'

import { useAuthStore } from './store/authStore'

import AppLayout from './components/layout/AppLayout'

import Login from './pages/Login'

import Register from './pages/Register'

import ResetPassword from './pages/ResetPassword'

import Dashboard from './pages/Dashboard'

import Products from './pages/Products'

import Transactions from './pages/Transactions'

import AllInsights from './pages/AllInsights'

import Suppliers from './pages/Suppliers'

import Reports from './pages/Reports'

import Settings from './pages/Settings'

function ProtectedRoute({ children }) {
  const isAuthenticated = useAuthStore(s => s.isAuthenticated)
  return isAuthenticated ? children : <Navigate to="/login" replace />
}

function PublicRoute({ children }) {
  const isAuthenticated = useAuthStore(s => s.isAuthenticated)
  return isAuthenticated ? <Navigate to="/" replace /> : children
}
```

```

export default function App() {
  return (
    <BrowserRouter future={{ v7_startTransition: true, v7_relativeSplatPath: true }}>
      <Routes>
        <Route path="/login" element={<PublicRoute><Login /></PublicRoute>} />
        <Route path="/register" element={<PublicRoute><Register /></PublicRoute>} />
        <Route path="/reset-password" element={<ResetPassword />} />
        <Route path="/" element={<ProtectedRoute><AppLayout /></ProtectedRoute>}>
          <Route index element={<Dashboard />} />
          <Route path="products" element={<Products />} />
          <Route path="transactions" element={<Transactions />} />
          <Route path="ai-insights" element={<AIInsights />} />
          <Route path="suppliers" element={<Suppliers />} />
          <Route path="reports" element={<Reports />} />
          <Route path="settings" element={<Settings />} />
        </Route>
        <Route path="*" element={<Navigate to="/" replace />} />
      </Routes>
    </BrowserRouter>
  )
}

```

index.css

@tailwind base;

@tailwind components;

@tailwind utilities;

@layer base {

:root {

/ Original Stitch Palette (Light) */*

--color-on-surface: 25 28 30;

--color-secondary-container: 108 248 187;

--color-on-background: 25 28 30;

--color-inverse-on-surface: 240 241 243;

--color-primary-fixed-dim: 195 192 255;

--color-surface-bright: 248 249 251;

--color-primary-container: 79 70 229;

--color-on-error: 255 255 255;

--color-on-error-container: 147 0 10;

--color-tertiary-fixed-dim: 255 179 173;

--color-on-tertiary: 255 255 255;

--color-on-secondary: 255 255 255;

--color-error-container: 255 218 214;

--color-on-primary-fixed-variant: 51 35 204;

--color-on-secondary-fixed: 0 33 19;

--color-tertiary: 150 0 20;

--color-primary-fixed: 226 223 255;

--color-tertiary-fixed: 255 218 215;
--color-primary: 53 37 205;
--color-background: 248 249 251;
--color-on-primary-container: 218 215 255;
--color-surface-container-lowest: 255 255 255;
--color-outline: 119 117 135;
--color-surface-container-low: 243 244 246;
--color-on-tertiary-fixed-variant: 147 0 19;
--color-inverse-primary: 195 192 255;
--color-inverse-surface: 46 49 50;
--color-on-tertiary-container: 255 208 204;
--color-surface-variant: 225 226 228;
--color-on-surface-variant: 70 69 85;
--color-surface-dim: 217 218 220;
--color-error: 186 26 26;
--color-on-secondary-container: 0 113 77;
--color-secondary: 0 108 73;
--color-on-primary-fixed: 15 0 105;
--color-outline-variant: 199 196 216;
--color-surface-container: 237 238 240;
--color-secondary-fixed: 111 251 190;
--color-on-primary: 255 255 255;
--color-secondary-fixed-dim: 78 222 163;
--color-surface: 248 249 251;
--color-tertiary-container: 188 29 37;

```
--color-surface-container-high: 231 232 234;  
--color-on-tertiary-fixed: 65 0 4;  
--color-on-secondary-fixed-variant: 0 82 54;  
--color-surface-tint: 77 68 227;  
--color-surface-container-highest: 225 226 228;  
}
```

```
.dark {  
  /* True Dark Aesthetic (Modern Slate/Indigo) */  
  --color-on-surface: 248 250 252;  
  --color-secondary-container: 45 212 191;  
  --color-on-background: 248 250 252;  
  --color-inverse-on-surface: 15 23 42;  
  --color-primary-fixed-dim: 99 102 241;  
  --color-surface-bright: 30 41 59;  
  --color-primary-container: 67 56 202;  
  --color-on-error: 255 255 255;  
  --color-on-error-container: 254 226 226;  
  --color-tertiary-fixed-dim: 244 63 94;  
  --color-on-tertiary: 255 255 255;  
  --color-on-secondary: 255 255 255;  
  --color-error-container: 153 27 27;  
  --color-on-primary-fixed-variant: 199 210 254;  
  --color-on-secondary-fixed: 204 253 241;  
  --color-tertiary: 225 29 72;
```

--color-primary-fixed: 199 210 254;
--color-tertiary-fixed: 255 228 230;
--color-primary: 129 140 248;
--color-background: 15 23 42;
--color-on-primary-container: 224 231 255;
--color-surface-container-lowest: 15 23 42;
--color-outline: 71 85 105;
--color-surface-container-low: 30 41 59;
--color-on-tertiary-fixed-variant: 255 204 213;
--color-inverse-primary: 129 140 248;
--color-inverse-surface: 241 245 249;
--color-on-tertiary-container: 255 241 242;
--color-surface-variant: 51 65 85;
--color-on-surface-variant: 148 163 184;
--color-surface-dim: 15 23 42;
--color-error: 239 68 68;
--color-on-secondary-container: 20 184 166;
--color-secondary: 16 185 129;
--color-on-primary-fixed: 238 242 255;
--color-outline-variant: 51 65 85;
--color-surface-container: 30 41 59;
--color-secondary-fixed: 17 94 89;
--color-on-primary: 255 255 255;
--color-secondary-fixed-dim: 20 184 166;
--color-surface: 15 23 42;


```
--color-tertiary-container: 159 18 57;
--color-surface-container-high: 51 65 85;
--color-on-tertiary-fixed: 255 241 242;
--color-on-secondary-fixed-variant: 204 253 241;
--color-surface-tint: 99 102 241;
--color-surface-container-highest: 71 85 105;
}
```

```
* { box-sizing: border-box; }
```

```
body {
  font-family: 'Inter', system-ui, sans-serif;
  background-color: rgb(var(--color-background));
  color: rgb(var(--color-on-surface));
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  transition: background-color 0.3s ease, color 0.3s ease;
}
}
```

```
* {
  transition: background-color 0.3s ease, border-color 0.3s ease;
}
```

```
@layer components {
  .material-symbols-outlined {
```

```
font-variation-settings: 'FILL' 0, 'wght' 400, 'GRAD' 0, 'opsz' 24;
vertical-align: middle;
line-height: 1;
flex-shrink: 0;
}
```

```
.editorial-shadow {
  box-shadow: 0px 12px 32px rgba(15, 23, 42, 0.2);
}
```

```
.dark .editorial-shadow {
  box-shadow: 0px 12px 32px rgba(0, 0, 0, 0.5);
}
```

```
.glass-panel {
  background: rgba(var(--color-surface-container) / 0.8);
  backdrop-filter: blur(24px);
  -webkit-backdrop-filter: blur(24px);
}
```

```
.scrollbar-thin {
  scrollbar-width: thin;
  scrollbar-color: rgb(var(--color-outline-variant)) transparent;
}
```

```
.scrollbar-thin::-webkit-scrollbar { width: 4px; }  
.scrollbar-thin::-webkit-scrollbar-track { background: transparent; }  
.scrollbar-thin::-webkit-scrollbar-thumb {  
  background-color: rgb(var(--color-outline-variant));  
  border-radius: 999px;  
}
```

```
.animate-pulse-slow {  
  animation: pulse 3s cubic-bezier(0.4, 0, 0.6, 1) infinite;  
}
```

```
/* Smooth page transitions */  
.page-enter {  
  animation: pageEnter 0.2s ease-out;  
}
```

```
@keyframes pageEnter {  
  from { opacity: 0; transform: translateY(8px); }  
  to { opacity: 1; transform: translateY(0); }  
}
```

```
/* Skeleton shimmer */  
.skeleton {
```

```
background: linear-gradient(90deg, rgb(var(--color-surface-container-high))
25%, rgb(var(--color-surface-container)) 50%, rgb(var(--color-surface-container-
high)) 75%);
```

```
background-size: 200% 100%;
```

```
animation: shimmer 1.5s infinite;
```

```
}
```

```
@keyframes shimmer {
```

```
0% { background-position: 200% 0; }
```

```
100% { background-position: -200% 0; }
```

```
}
```

```
}
```



main.jsx

```
import React from 'react'

import ReactDOM from 'react-dom/client'

import { QueryClient, QueryClientProvider } from '@tanstack/react-query'

import { Toaster } from 'react-hot-toast'

import App from './App.jsx'

import './index.css'
```

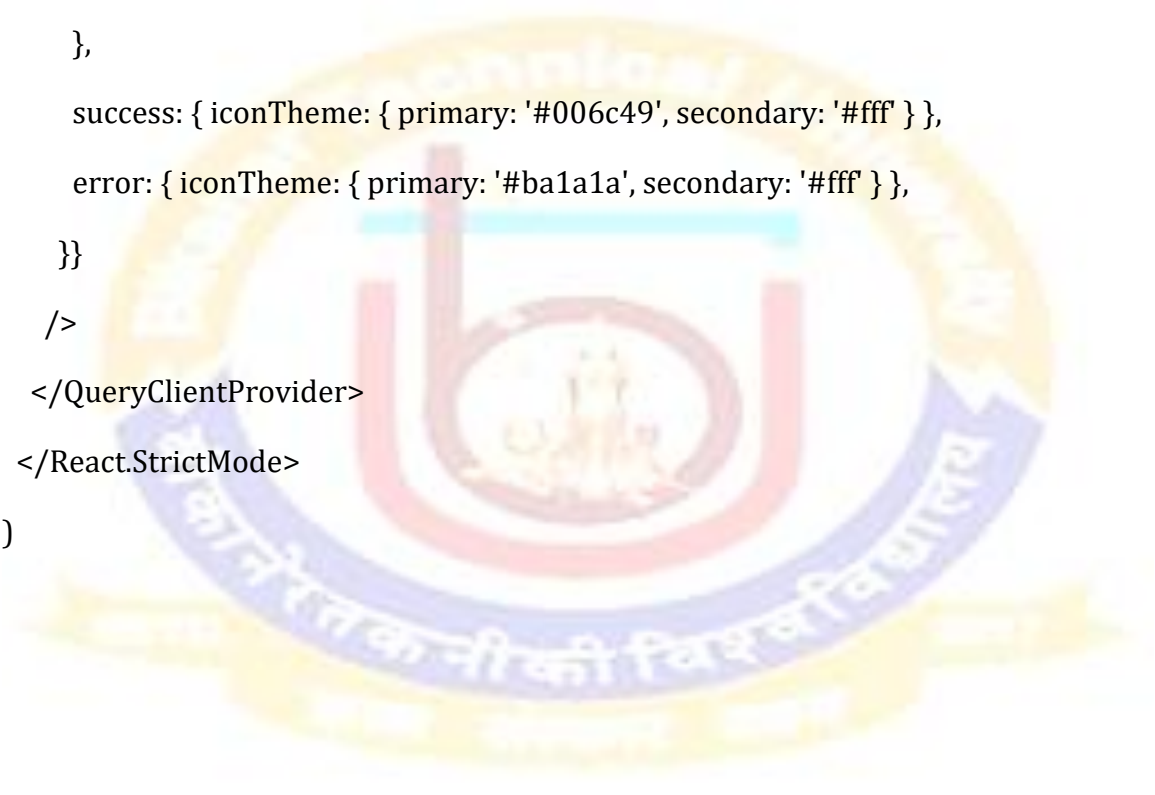
```
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      retry: 1,
      staleTime: 30 * 1000,
      refetchOnWindowFocus: false,
    },
  },
})
```

```
ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <QueryClientProvider client={queryClient}>
      <App />
      <Toaster
        position="top-right"
        toastOptions={{
```

```

duration: 4000,
style: {
  background: '#191c1e',
  color: '#f0f1f3',
  borderRadius: '0.75rem',
  fontSize: '0.875rem',
  fontWeight: '500',
},
success: { iconTheme: { primary: '#006c49', secondary: '#fff' } },
error: { iconTheme: { primary: '#ba1a1a', secondary: '#fff' } },
}}
/>
</QueryClientProvider>
</React.StrictMode>
)

```



index.html

```
<!DOCTYPE html>

<html lang="en">

  <head>

    <meta charset="UTF-8" />

    <link rel="icon" type="image/x-icon" href="/icon.ico" />

    <link rel="shortcut icon" href="/icon.ico" />

    <meta name="application-name" content="StockGrid" />

    <meta name="apple-mobile-web-app-title" content="StockGrid" />

    <meta name="viewport" content="width=device-width, initial-scale=1.0" />

    <title>StockGrid</title>

    <link rel="preconnect" href="https://fonts.googleapis.com" />

    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin />

    <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700;800
&display=swap" rel="stylesheet" />

    <link
href="https://fonts.googleapis.com/css2?family=Material+Symbols+Outlined:opsz,
wght,FILL,GRAD@20..48,100..700,0..1,-50..200" rel="stylesheet" />

    <script>

      (function() {

        try {

          var localTheme = localStorage.getItem('theme_preference');

          if (localTheme === 'light') {

            document.documentElement.classList.remove('dark');

          } else {
```



```
        document.documentElement.classList.add('dark');  
    }  
    } catch (e) {}  
    })();  
</script>  
</head>  
<body>  
    <div id="root"></div>  
    <script type="module" src="/src/main.jsx"></script>  
</body>  
</html>
```



tailwind.config.js

```
/** @type {import('tailwindcss').Config} */  
  
export default {  
  content: ['./index.html', './src/**/*.{js,ts,jsx,tsx}'],  
  darkMode: 'class',  
  theme: {  
    extend: {  
      colors: {  
        'on-surface': 'rgb(var(--color-on-surface) / <alpha-value>)',  
        'secondary-container': 'rgb(var(--color-secondary-container) / <alpha-value>)',  
        'on-background': 'rgb(var(--color-on-background) / <alpha-value>)',  
        'inverse-on-surface': 'rgb(var(--color-inverse-on-surface) / <alpha-value>)',  
        'primary-fixed-dim': 'rgb(var(--color-primary-fixed-dim) / <alpha-value>)',  
        'surface-bright': 'rgb(var(--color-surface-bright) / <alpha-value>)',  
        'primary-container': 'rgb(var(--color-primary-container) / <alpha-value>)',  
        'on-error': 'rgb(var(--color-on-error) / <alpha-value>)',  
        'on-error-container': 'rgb(var(--color-on-error-container) / <alpha-value>)',  
        'tertiary-fixed-dim': 'rgb(var(--color-tertiary-fixed-dim) / <alpha-value>)',  
        'on-tertiary': 'rgb(var(--color-on-tertiary) / <alpha-value>)',  
        'on-secondary': 'rgb(var(--color-on-secondary) / <alpha-value>)',  
        'error-container': 'rgb(var(--color-error-container) / <alpha-value>)',  
        'on-primary-fixed-variant': 'rgb(var(--color-on-primary-fixed-variant) / <alpha-value>)',  
        'on-secondary-fixed': 'rgb(var(--color-on-secondary-fixed) / <alpha-value>)',  
        'tertiary': 'rgb(var(--color-tertiary) / <alpha-value>)',  
      }  
    }  
  }  
}
```

'primary-fixed': 'rgb(var(--color-primary-fixed) / <alpha-value>)',
'tertiary-fixed': 'rgb(var(--color-tertiary-fixed) / <alpha-value>)',
'primary': 'rgb(var(--color-primary) / <alpha-value>)',
'background': 'rgb(var(--color-background) / <alpha-value>)',
'on-primary-container': 'rgb(var(--color-on-primary-container) / <alpha-value>)',
'surface-container-lowest': 'rgb(var(--color-surface-container-lowest) / <alpha-value>)',
'outline': 'rgb(var(--color-outline) / <alpha-value>)',
'surface-container-low': 'rgb(var(--color-surface-container-low) / <alpha-value>)',
'on-tertiary-fixed-variant': 'rgb(var(--color-on-tertiary-fixed-variant) / <alpha-value>)',
'inverse-primary': 'rgb(var(--color-inverse-primary) / <alpha-value>)',
'inverse-surface': 'rgb(var(--color-inverse-surface) / <alpha-value>)',
'on-tertiary-container': 'rgb(var(--color-on-tertiary-container) / <alpha-value>)',
'surface-variant': 'rgb(var(--color-surface-variant) / <alpha-value>)',
'on-surface-variant': 'rgb(var(--color-on-surface-variant) / <alpha-value>)',
'surface-dim': 'rgb(var(--color-surface-dim) / <alpha-value>)',
'error': 'rgb(var(--color-error) / <alpha-value>)',
'on-secondary-container': 'rgb(var(--color-on-secondary-container) / <alpha-value>)',
'secondary': 'rgb(var(--color-secondary) / <alpha-value>)',
'on-primary-fixed': 'rgb(var(--color-on-primary-fixed) / <alpha-value>)',
'outline-variant': 'rgb(var(--color-outline-variant) / <alpha-value>)',
'surface-container': 'rgb(var(--color-surface-container) / <alpha-value>)',

```
'secondary-fixed': 'rgb(var(--color-secondary-fixed) / <alpha-value>)',
'on-primary': 'rgb(var(--color-on-primary) / <alpha-value>)',
'secondary-fixed-dim': 'rgb(var(--color-secondary-fixed-dim) / <alpha-value>)',
'surface': 'rgb(var(--color-surface) / <alpha-value>)',
'tertiary-container': 'rgb(var(--color-tertiary-container) / <alpha-value>)',
'surface-container-high': 'rgb(var(--color-surface-container-high) / <alpha-value>)',
'on-tertiary-fixed': 'rgb(var(--color-on-tertiary-fixed) / <alpha-value>)',
'on-secondary-fixed-variant': 'rgb(var(--color-on-secondary-fixed-variant) / <alpha-value>)',
'surface-tint': 'rgb(var(--color-surface-tint) / <alpha-value>)',
'surface-container-highest': 'rgb(var(--color-surface-container-highest) / <alpha-value>)',
},
borderRadius: {
  DEFAULT: '0.25rem',
  lg: '0.5rem',
  xl: '0.75rem',
  full: '9999px',
},
fontFamily: {
  sans: ['Inter', 'system-ui', 'sans-serif'],
  headline: ['Inter', 'sans-serif'],
  body: ['Inter', 'sans-serif'],
  label: ['Inter', 'sans-serif'],
},
```

```
boxShadow: {  
  editorial: '0px 12px 32px rgba(25, 28, 30, 0.06)',  
  'editorial-lg': '0px 24px 48px rgba(25, 28, 30, 0.10)',  
},  
},  
},  
plugins: [],  
}
```



vite.config.js

```
import { defineConfig } from 'vite'

import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:5000',
        changeOrigin: true,
      }
    }
  }
})
```

README.md

📦 StockGrid

> AI-powered inventory management system with smart insights, stock tracking, and automation.

🚀 Overview

StockGrid is a full-stack SaaS application designed to help businesses manage inventory efficiently, reduce stock issues, and make data-driven decisions using AI.

It replaces messy spreadsheets with a structured, scalable, and intelligent system.

✨ Features

📦 Inventory Management

- Product CRUD (Create, Read, Update, Delete)
- SKU-based product tracking
- Category management

🔄 Stock Tracking

- IN / OUT transactions
- Real-time stock calculation
- Complete audit trail

🚨 Alerts

- Low stock notifications
- Smart threshold handling

📊 Analytics

- Sales insights
- Top products
- Dead stock detection

🤖 AI Features

- Demand prediction
- Reorder suggestions
- AI-powered insights

📁 Import / Export

- CSV / Excel import
- Smart column mapping (AI-assisted)
- Data export for backup

🗝️ Authentication

- JWT-based auth
- Role-based access (Admin / Manager / Staff)
- Secure password reset system

🛠️ Tech Stack

Frontend

- React (Vite)
- Tailwind CSS

Backend

- Node.js
- Express.js

Database

- PostgreSQL (Supabase)

Other

- Gmail SMTP (Email service)
- JWT (Authentication)

📁 Project Structure

```text

StockGrid/

```
├── backend/ # Express.js API
| ├── src/
| | ├── config/ # Env and DB configuration
| | ├── controllers/ # Request handlers
| | ├── middleware/ # Auth and validation
| | ├── routes/ # API endpoints
| | ├── services/ # Business logic (AI, Email, etc.)
| | └── index.js # App entry point
├── frontend/ # React + Vite Application
| ├── src/
| | ├── components/ # Reusable UI components
| | ├── pages/ # View components
| | ├── services/ # API communication
| | ├── store/ # State management
| | └── App.jsx # Root component
└── database/ # SQL scripts and schema
```

```

🛠️ Getting Started

Prerequisites

- [Node.js](https://nodejs.org/) (v18+)
- [npm](https://www.npmjs.com/) or [yarn](https://yarnpkg.com/)
- [Supabase](https://supabase.com/) account
- [Google Gemini API Key](https://aistudio.google.com/)

Installation

1. ****Clone the repository:****

```
``bash
git clone https://github.com/Shubham06032003/stockgrid.git
cd stockgrid
``
```

2. ****Setup Backend:****

```
``bash
cd backend
npm install
``
```

Create a `.env` file in the `backend/` directory (see [Environment Variables](#-environment-variables)).

3. ****Setup Frontend:****

```
```bash
cd ../frontend
npm install
```
```

Running Locally

1. ****Start the Backend:****

```
```bash
cd backend
npm run dev
```
```

2. ****Start the Frontend:****

```
```bash
cd frontend
npm run dev
```
```

🔑 Environment Variables

Backend (`backend/.env`)

| Variable | Description |

| :--- | :--- |

| `PORT` | Server port (default: 5000) |

| `SUPABASE\_URL` | Your Supabase project URL |

| `SUPABASE\_SERVICE\_KEY` | Your Supabase service role key |

| `JWT\_SECRET` | Secret for JWT signing |

| `JWT\_EXPIRES\_IN` | Token expiration (e.g., `7d`) |

| `GEMINI\_API\_KEY` | Google AI API Key |

| `GEMINI\_MODEL` | AI Model (e.g., `gemini-3-flash-preview`) |

| `EMAIL\_USER` | Gmail address for SMTP |

| `EMAIL\_PASS` | Gmail App Password |

| `FRONTEND\_URL` | URL of the frontend (for CORS) |

🤝 Contributing

1. Fork the Project
2. Create your Feature Branch (`git checkout -b feature/AmazingFeature`)
3. Commit your Changes (`git commit -m 'Add some AmazingFeature'`)
4. Push to the Branch (`git push origin feature/AmazingFeature`)
5. Open a Pull Request

📄 License

Distributed under the MIT License. See `LICENSE` for more information.

<p align="center">Made with ❤️ for smart businesses</p>

